



Mahidol University

ITCS 213 Computer Organization and Architecture

Class: 1-2026

Sec 2 Fri 9:00-12:00 IT312 (Aj. Joke)

Sec 3 Fri 13:00-16:00 IT312 (Aj. Joke)

Sec 1 Fri 13:00-16:00 IT311 (Aj. Peter)

Unit 2: **ALU** & Registers & **Instruction Sets**

Lecturer: Dr. Peter M.D. Scully
Faculty of ICT, Mahidol University
peter.scu@mahidol.edu

Slides adapted from Subject Coordinator:
Dr. Kanoksak Wattanachote (Ph.D) (金功勳 博士)
kanoksak.wat@mahidol.ac.th

Announcement: ITCS213

Unit 2: ALU & Register

Announcements

Assembly Programming Tools links available:

ARM: https://cpulator.01xz.net/?sys=arm

MIPS: https://tio.run/ -> MIPS -> Hello World

x86: https://tutorialspoint.com/compile_assembly_online.php

Today we aim to introduce R-Type and I-Type instructions. What are these? We'll find out!

Agenda

Part A: IEEE 754 Floating-Point Format (***Part 2***)

Part B: CPU, GPU & NPU — processor types

Part C: CPU Major Structural Components

Part D: ALU — Arithmetic-Logic Unit

Part E: Instruction Set Architectures (ISAs)

Part F: Programming: ISAs with Assembly (ARM, MIPS, x86)

Part G: Pipelined Processors

Computer Organization Study Process:

How we build understanding — from number formats to processor design

- **Step 1: Number Representation**
  - IEEE 754, binary conversion, data formats
- **Step 2: Processor Architecture**
  - CPU/GPU/NPU, ALU, control unit, registers
- **Step 3: Instruction Set Architecture**
  - RISC vs CISC, ARM/MIPS/x86, opcode formats
- **Step 4: Assembly Programming**
  - Writing and decoding assembly (ARM, MIPS, x86)
- **Step 5: Pipeline Processing**
  - Instruction pipelining, hazards, performance
- **Step 6: Memory & Advanced Topics**
  - Cache hierarchy, memory management (future units)

Steps 1-5 are covered in this unit (Unit 2). Step 6 continues in Units 5-14.

PART A

IEEE 754 Floating-point Representation (32-bit + 64-bit)

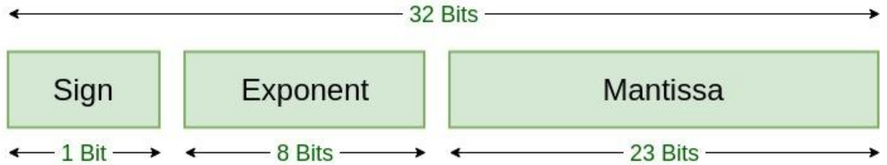
How IEEE 754 encodes real numbers in 64-bit double precision.

IEEE 754 Floating-Point Format

IEEE 754 Floating-Point Format (32-bit **Single Precision**)

- Structure:

1 Bit	8 Bits	23 Bits
Sign (S)	**Exponent** (E)	**Mantissa** (M)

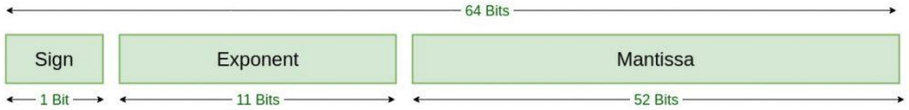


Single Precision
IEEE 754 Floating-Point Standard

- Example:

1	10000010	10110100000000000000000
Sign	**Exponent**	**Mantissa**

Also : 64-bit **Single Precision**



Double Precision
IEEE 754 Floating-Point Standard

source:
https://www.geeksforgeeks.org/computer-organization-architecture/ieee-standard-754-floating-point-numbers/

IEEE 754 Floating-Point Format

WORKED EXAMPLE

IEEE 754 Floating-Point Format (32-bit Single Precision)

1. Example: **85.125**

2. **Integer part (85):**
   85 = 1010101

3. **Fractional part :**
   0.125 = 001

4. **Combined binary (85.125):**
   85.125 = 1010101.001

We already know:

- *Sign bit* is Positive = *0*

2 Binary Conversion (Base 2) Refresher

$5_{10} \rightarrow 101_2$

Binary Conversion

- Example
  - **Convert $(37)_{10}$ to binary: Using integer division:**

$37 \div 2 = 18 + 1$ remainder $= 1$

$18 \div 2 = 9 + 0$ remainder $= 0$

$9 \div 2 = 4 + 1$ remainder $= 1$

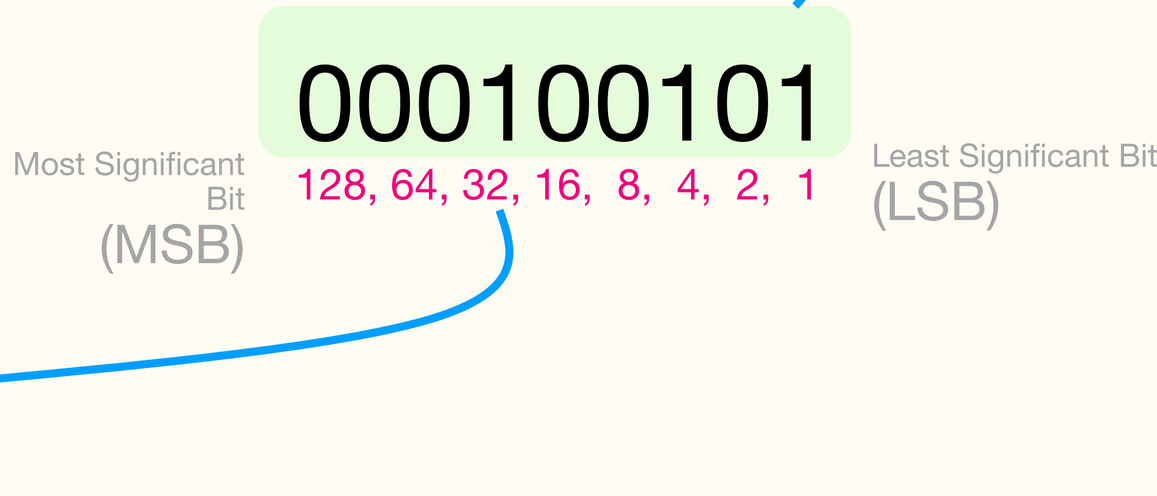
$4 \div 2 = 2 + 0$ remainder $= 0$

$2 \div 2 = 1 + 0$ remainder $= 0$

$1 \div 2 = 0 + 1$ remainder $= 1$

Least Significant Bit (LSB)

Most Significant Bit (MSB)



  - The answer is $(100101)_2 = (37)_{10}$

3 Binary Conversion (Base 2) Refresher

$0.5_{10} \rightarrow 0.1_{2}$

Binary Conversion

- Converting a decimal number to binary.
- Example
  - **Convert $(0.4375)_{10}$ to binary.**

Multiplication result		The result of integers
0.4375 x 2 = 0.875	→	0
0.8750 x 2 = 1.750	→	1
0.7500 x 2 = 1.500	→	1
0.5000 x 2 = 1.000	→	1

  - The answer is $(0.4375)_{10} = (0.0111)_{2}$.

IEEE 754 Floating-Point Format

WORKED EXAMPLE

IEEE 754 Floating-Point Format (32-bit Single Precision)

- Example: 85.125

5 Normalize the Binary Number

85.125 = 1010101.001

= 1.010101001 x 2^6 → normalization to "1.xxxxxxx"



IEEE 754 Floating-Point Format

WORKED EXAMPLE

Now we can allocate some portions of the IEEE 754 bits:

```
1.010101001 x 2^6
```

6. The Sign bit is positive, so: 0 (positive number).

7. The Exponent is: 6 *(since we shifted the binary point 6 places left).*

8. We can allocate a portion of the Mantissa: *<Implicit **1.** is assumed>*
   - 010101001 (the fractional part after normalization).

IEEE 754 Floating-Point Format

WORKED EXAMPLE

- Now, we can insert those binary numbers into the table for **32-Bit**:

32-bit Single Precision

Field	Size (bits)	Calculation	Binary Value
Sign (S)	1	0 (positive)	0
Exponent (E)	8	127 (bias) + 6 = 133	10000101 (133 in binary)
Mantissa (M)	23	010101001 (padded with zeros)	01010100100000000000000

6

7.1

7.2

8.1

8.2

8.3

*Remember: **1.** is implicit. This mantissa is relative to 1.0. So padded zeros are smaller fractional digits.*

32-Bit Mantissa = 23 bits total.

IEEE 754 Floating-Point Format

WORKED EXAMPLE

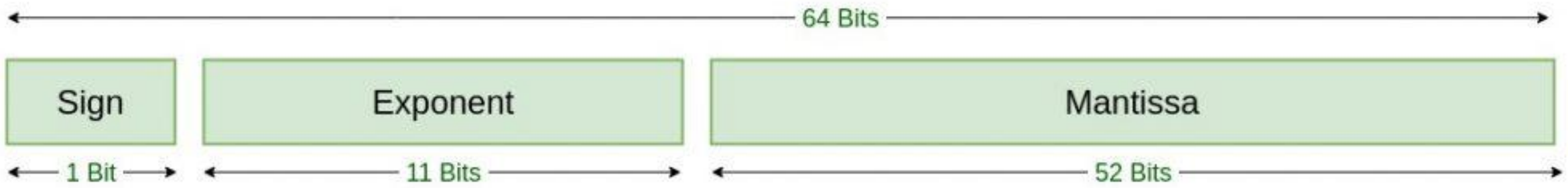
- Example: 85.125 on IEEE 754 **Double-Precision (64-bit)**

64-bit Double Precision

Field	Size (bits)	Calculation	Binary Value
Sign (S)	1	0 (positive)	0
Exponent (E)	11	1023 (bias) + 6 = **1029**	10000000101 **(1029 in binary)**
Mantissa (M)	52	010101001 (padded with zeros)	01010100100…00(52 bits)

6 7.1 7.2 8.1 8.2 8.3

64 Bit → Mantissa = 52 bits total.



IEEE 754 Floating-Point Format



IEEE 754 Floating-Point Format (32-bit **Single Precision**)

- Key Features:
  - Sign (S): 0 = Positive, 1 = Negative
  - **Exponent** (E): Bias = 127 (Stored as E + 127)
  - **Mantissa** (M): Implicit leading 1 (**Normalize**d)

1

Normalize example:

- The binary number **101.1101 is normalized as 1.011101×2^2**

IEEE 754 Floating-Point Format



IEEE 754 Floating-Point Format (32-bit **Single Precision**)

- To Represent Signed **Exponent**s
  - The exponent can be positive or negative
    - e.g., 2^{6} or 2^{-3}
  - Instead of using two's complement (which complicates hardware design), IEEE 754 uses a biased exponent (**always stored as a positive number**).

Fixed Exponent Bias Values

Precision	Exponent Bits	Bias Value	Formula for Bias
Single (32-bit)	8 bits	**127**	$2^{(8-1)} - 1 = 127$
Double (64-bit)	11 bits	**1023**	$2^{(11-1)} - 1 = 1023$

IEEE 754 Floating-Point Format



Mantissa:

- When converting a decimal number to IEEE 754 binary floating-point, the fractional part often produces a repeating or infinite binary sequence.
- Since the IEEE 754 standard allocates a fixed number of bits for the mantissa
  - 23 bits for single-precision
  - 52 bits for double-precision
- We must truncate or round the repeating sequence.

IEEE 754 Floating-Point Format



Mantissa Rounding:

- Rules for Determining the Mantissa:
  1. Stop at the 24th Bit (Single-Precision) or 53rd Bit (Double-Precision)
     - The mantissa is stored as a fixed-length field:
     - Single-precision (32-bit): 23 bits (but we compute 24 bits for rounding).
     - Double-precision (64-bit): 52 bits (compute 53 bits).

If the binary fraction repeats or continues infinitely, calculate up to the 24th/53rd bit and then apply rounding.

IEEE 754 Floating-Point Format



Mantissa Rounding: *(32-bit **Single Precision**)*

- Rules for Determining the Mantissa using IEEE 754 **(Round to Nearest, Ties to Even)**:

  - There are four cases.
    - **Bits 24 - 26**, are determine the rounding decision.
    - In each case, we have a **mantissa** of **more than 23 bits**.

      For example **00000000000000000000000** **(23 bits of zeros)**
      Plus three extra bits **111** **(Bits 24, 25, 26)**

      These 3 extra bits are called:
      - Guard (G), Round (R) and Sticky (S)

IEEE 754 Floating-Point Format



Mantissa Rounding: *(32-bit **Single Precision**)*

For example **00000000000000000000000** **(23 bits of zeros)**
Plus three extra bits **111** **(Bits 24, 25, 26)**

	G R S			Bits [22, 23]
1. Bits: **... 01**	0 1 1	→	Truncate	→ **... 01**
2. Bits: **... 00**	1 0 1	→	Add 1 to Bit 23.	→ **... 01**
3. Bits: **... 00**	1 0 0	→	Round to even. Bit 23 is already 0.	→ **... 00**
4. Bits: **... 01**	1 0 0	→	Round Bit 23 to even (carry 1 to left)	→ **... 10**

IEEE 754 Floating-Point Format



Mantissa Rounding: *(32-bit **Single Precision**)*

Case	Bit 23	Guard (G)	Round (R)	Sticky (S)	*Trailing Value*	Action	Final Bit 23
1. Less Than Half	**1**	**0**	**1**	**1**	*< 0.5*	Truncate	**1**
2. Greater Than Half	**0**	**1**	**0**	**1**	*> 0.5*	+1	**1**
3. Tie (Already Even)	**0**	**1**	**0**	**0**	*= 0.5*	Keep Even	**0**
4. Tie (Was Odd)	**1**	**1**	**0**	**0**	*= 0.5*	Make Even (+1)	**0**

IEEE 754 Floating-Point Format - Key Takeaways

EXAM AND QUIZ PRACTICE

- **Normalization:** The binary number is expressed as $1.xxxx \times 2^n$
- Exponent Bias:
  - **Single**-precision: Add **127** to the exponent.
  - **Double**-precision: Add **1023** to the exponent.
- Mantissa:
  - The **mantissa is padded with trailing zeros** to fill the required bits (**23 for single**, **52 for double**).
  - **Assumed 1.xxxx**.
  - **Rounding bit**.
- Different bit size structures for 32-Bit and 64-bit IEEE 754 formats. Same principles.

Exercise 1: IEEE 754 Floating-Point Format

EXERCISE

IEEE 754 Floating-Point Format

- Question 1: Convert the decimal number -12.625 to its IEEE 754 64-bit binary representation.
- Question 2: What is the 64-bit representation of 0.0? S
- Question 3: What is the 32-bit representation of 0.1? P
- Question 4: Convert -26.375 to its 64-bit IEEE 754 binary representation.
- Question 5: Convert -1/3 to its 32-bit IEEE 754 binary representation. B
- Question 6: Whose ID, does the mantissa fraction repeat?
- Question 7: Whose ID, does the mantissa fraction continue infinitely?





Solutions: IEEE 754 Floating-Point Format

EXERCISE

IEEE 754 Floating-Point Format

- −0.333... (−1/3, repeating)
  - Sign bit: Negative → 1
  - Convert fraction to binary:
    1/3 = 0.010101010101... (repeating 01)
  - **Normalize**: $1.010101010101... \times 2^{-2}$
  - **Exponent**: -2 + 127 = 125 → 01111101
  - **Mantissa**: 01010101010101010101011 (rounded)
  - Final:
    1 | 01111101 | 01010101010101010101011
    Binary: 10111111010101010101010101011011
    Hex: 0xBEAAAAAB



Solutions: IEEE 754 Floating-Point Format

EXERCISE

IEEE 754 Floating-Point Format

- 0.1 (Repeating)
  - Sign bit: Positive → 0
  - Convert to binary (fraction only):
    $0.1 \times 2 = 0.2 \rightarrow 0$
    $0.2 \times 2 = 0.4 \rightarrow 0$
    $0.4 \times 2 = 0.8 \rightarrow 0$
    $0.8 \times 2 = 1.6 \rightarrow 1$ (0.6 left)
    ... → repeats 0001100110011...
    → 0.0001100110011001100110011...
  - **Normalize**: $1.1001100110011... \times 2^{-4}$
  - **Exponent**: $-4 + 127 = 123 \rightarrow 01111011$
  - **Mantissa**: 10011001100110011001101 (rounded to 23 bits)
  - Final:
    0 | 01111011 | 10011001100110011001101
    Binary: 00111101111001100110011001101101
    Hex: 0x3DCCCCCD



Solutions: IEEE 754 Floating-Point Format

EXERCISE

IEEE 754 Floating-Point Format

- 0.0 (Repeating)
  - 0 00000000 00000000000000000000000
  - Sign bit (1 bit): 0 (positive)
  - **Exponent** (8 bits): 00000000 (all zeros, special case for zero/subnormal numbers)
  - **Mantissa** (23 bits): 000...000 (all zeros)

  - 0 00000000000 0000000000000000000000000000000000000000000000000000
  - Sign bit (1 bit): 0 (positive)
  - Exponent (11 bits): 00000000000 (all zeros, special case for zero/subnormal numbers)
  - Mantissa (52 bits): 000...000 (all zeros)





Key Points: IEEE 754 Floating-point Representation (64-bit)

- **IEEE 754 Standard**
  - Defines binary floating-point representation
  - Used universally in computing
- **Single Precision (32-bit)**
  - 1 sign bit, 8 exponent bits, 23 mantissa bits
  - Exponent bias: 127
- **Double Precision (64-bit)**
  - 1 sign bit, 11 exponent bits, 52 mantissa bits
  - Exponent bias: 1023
- **Normalization**
  - Express as $1.xxxx \times 2^n$
  - Implicit leading 1
- **Mantissa Rules**
  - Stop at 24th bit (single) or 53rd bit (double)
  - Round-to-Nearest-Even (IEEE 754 rounding)
    - Four Cases. Look for half-way point.

PART B

CPU GPU NPU

Comparing processors: CPUs for control, GPUs for parallel work, NPUs for AI.

CPU: Central Processing Unit

- **CPU**
  - A **CPU** is a **Central Processing Unit** — the main, general-purpose processor in a computer that runs the operating system and applications, and controls all other components.
  - Executes a wide range of instructions (**CISC** or **RISC**).
  - Sequential, strong at branching, and mixed workloads.
  - Small number of powerful cores (often 4–16 in modern desktop CPUs).

CPU: Central Processing Unit

- **CPU** Primary Role
  - Acts as the “**brain**” of the computer.
  - Executes general-purpose instructions from programs.
  - Coordinates work between memory, storage, and peripheral devices.

- **CPU**: How It Works
  - Executes instructions in a fetch → decode → execute → write-back cycle.
  - Supports branching, loops, logic, arithmetic, and a wide range of data types.
  - Can run many different types of workloads — from spreadsheets to video games to simulations.

CPU: Central Processing Unit

- **Key** Characteristics
  - Few powerful cores (e.g., 4–16 in modern desktops/laptops, more in servers).
  - High clock speed (often 3–5 GHz).
  - Large caches for quick access to frequently used data.
  - Good at single-threaded performance and tasks with lots of branching.

GPU: Graphics Processing Unit

- **GPU** :
  - A **GPU** is a **Graphics Processing Unit** — a processor originally designed to accelerate graphics rendering but now widely used for many parallel-computation tasks, especially AI and scientific workloads.

- **GPU** Original Role
  - Rendering images & video: Converts 3D models into the pixels you see on a screen.
  - Handles geometry processing, drawing, texture mapping, shading, etc.
  - Optimized for floating-point and vector math.

GPU: Graphics Processing Unit

- **GPU** Modern Role
  - GPUs aren't just for graphics anymore.
  - Their thousands of simple cores make them ideal for:
    - Deep learning training and inference
    - Physics simulations
    - Video processing
    - Cryptocurrency mining

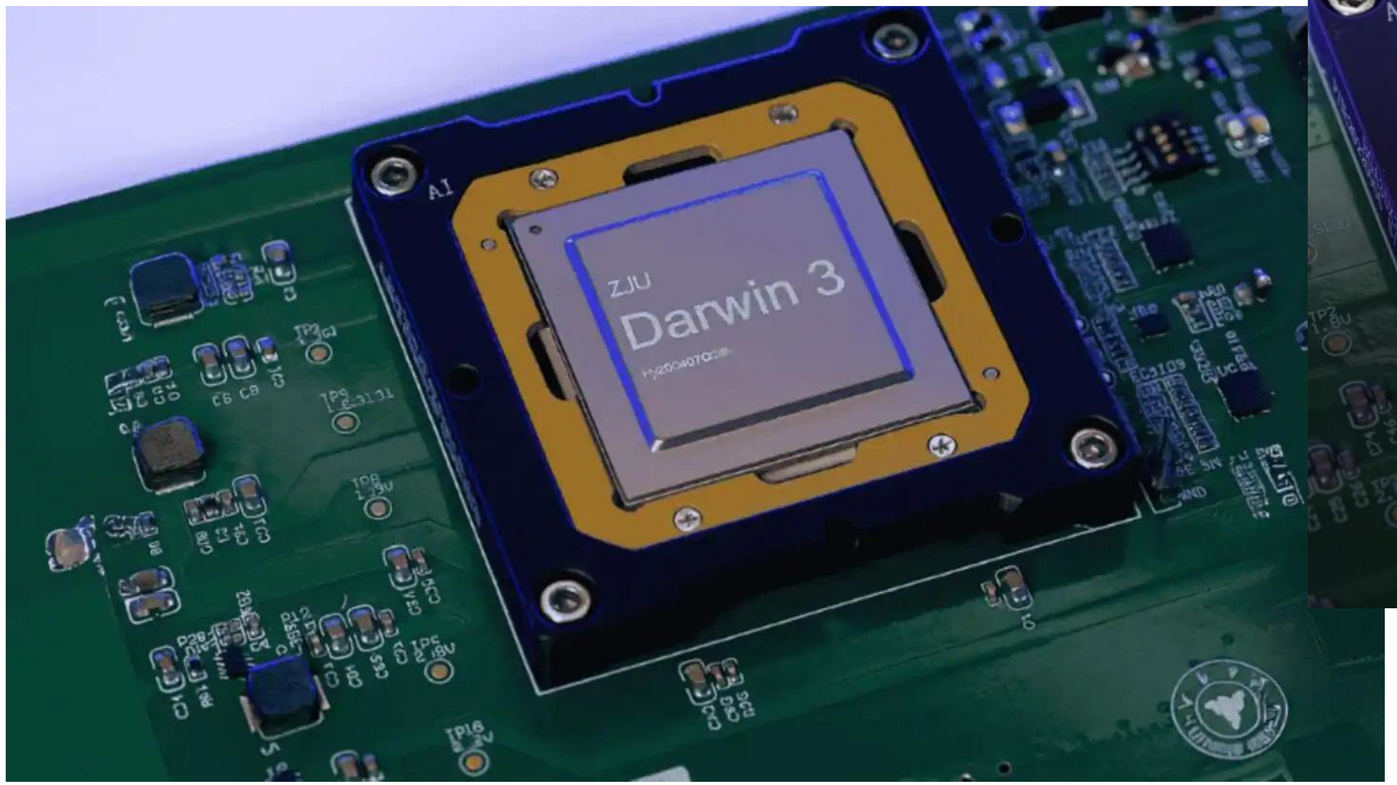
inference (n) - a guess that you make or an opinion that you form based on the information that you have.

GPU: Graphics Processing Unit

- **GPU**: How It Works
  - Uses **SIMD**/**SIMT** (Single Instruction, Multiple Data/Thread) —
    - The same instruction is applied to many data elements in **parallel**.
    - Extremely good at matrix multiplications and vector operations (core operations in both graphics and AI).
  - Relies on very high memory bandwidth to feed data to many cores at once.

NPU: Neural Processing Unit

- An **NPU** is a **Neural Processing Unit**.



source:
https://www.newsbytesapp.com/news/science/darwin-monkey-china-s-brain-like-ai-system-nears-macaque-neuron-count/story



source: South China Morning Post

Darwin 3

- **Chinese** engineers have unveiled a groundbreaking artificial intelligence (AI) system, dubbed the "**Darwin Monkey**."
- The innovative **computing system** features over two billion artificial neurons, with a neuron count approaching that of a macaque.
- Developed by researchers at Zhejiang University, this revolutionary machine marks a major milestone in the field of brain-inspired AI.



source: South China Morning Post

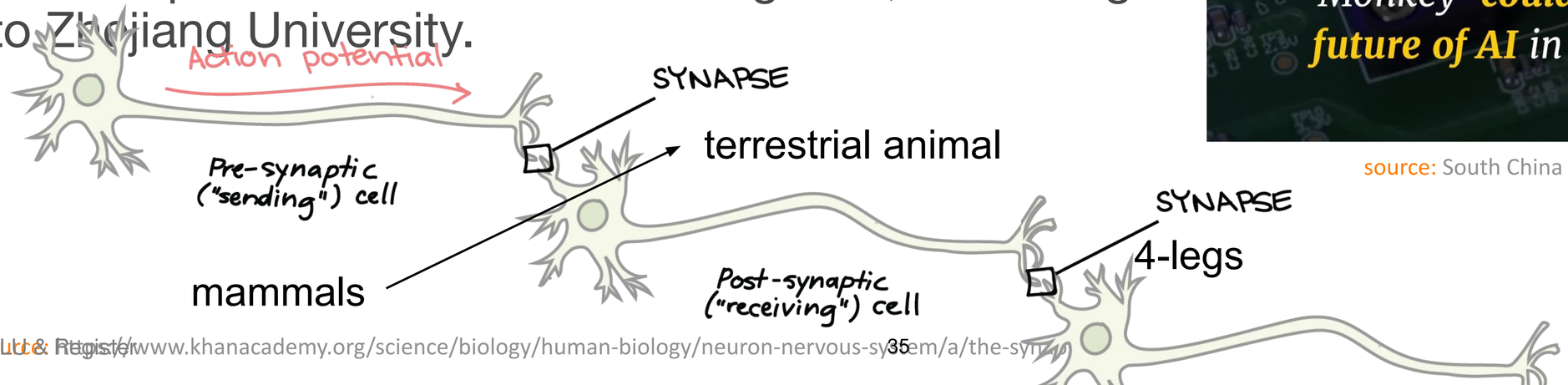
macaque (n) - a medium-sized monkey from Africa or Asia, with a long face.

Darwin 3

- The "**Darwin Monkey**" is a highly advanced brain-inspired computer.
  - It comprises 960 Darwin 3 brain-inspired computing chips and can create over **100 billion synapses**.
- This massive network of artificial neurons, as well as synapses, can bring us a step closer to achieving more sophisticated brain-like intelligence, according to Zhejiang University.



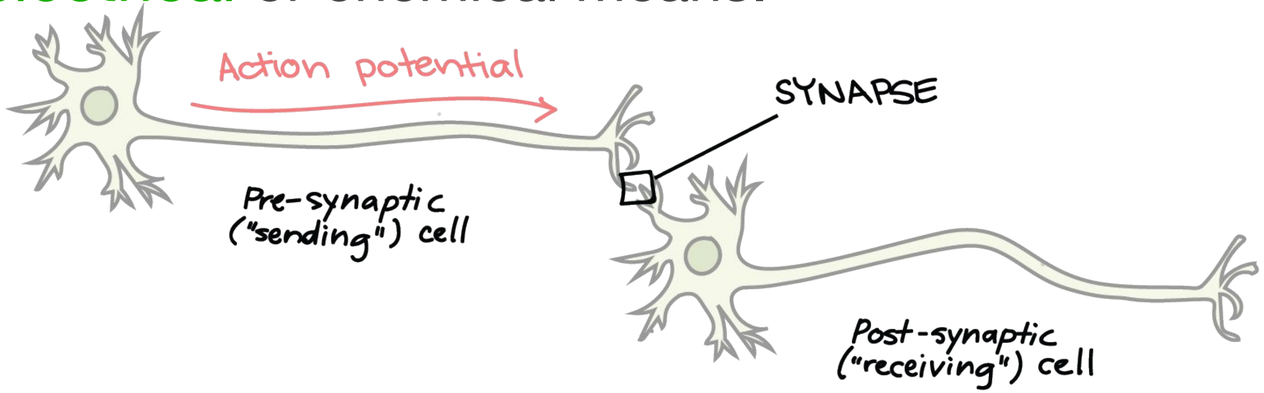
source: South China Morning Post



source: Khan Academy: https://www.khanacademy.org/science/biology/human-biology/neuron-nervous-system/a/the-synapse

Synapses

- **Synapses** are specialized junctions where nerve cells (neurons) communicate with each other or with other types of cells, like muscle or gland cells.
- They act as the communication links in the **nervous system**, enabling the transmission of signals and the integration of information.
- This communication can occur through either electrical or chemical means.





source: South China Morning Post

source: Khan Academy: https://www.khanacademy.org/science/biology/human-biology/neuron-nervous-system/a/the-synapse

Darwin 3

- It can perform
  - **logical reasoning**,
  - **mathematical calculations**.
- The “Darwin Monkey” has already been put to the test and has proven its capabilities in performing complex tasks.
  - These include content generation, logical reasoning, as well as mathematical calculations.
- Chinese AI company **DeepSeek**’s brain-like large language model was used in conjunction with the “Darwin Monkey,” showcasing the potential of this cutting-edge technology in real-world applications.

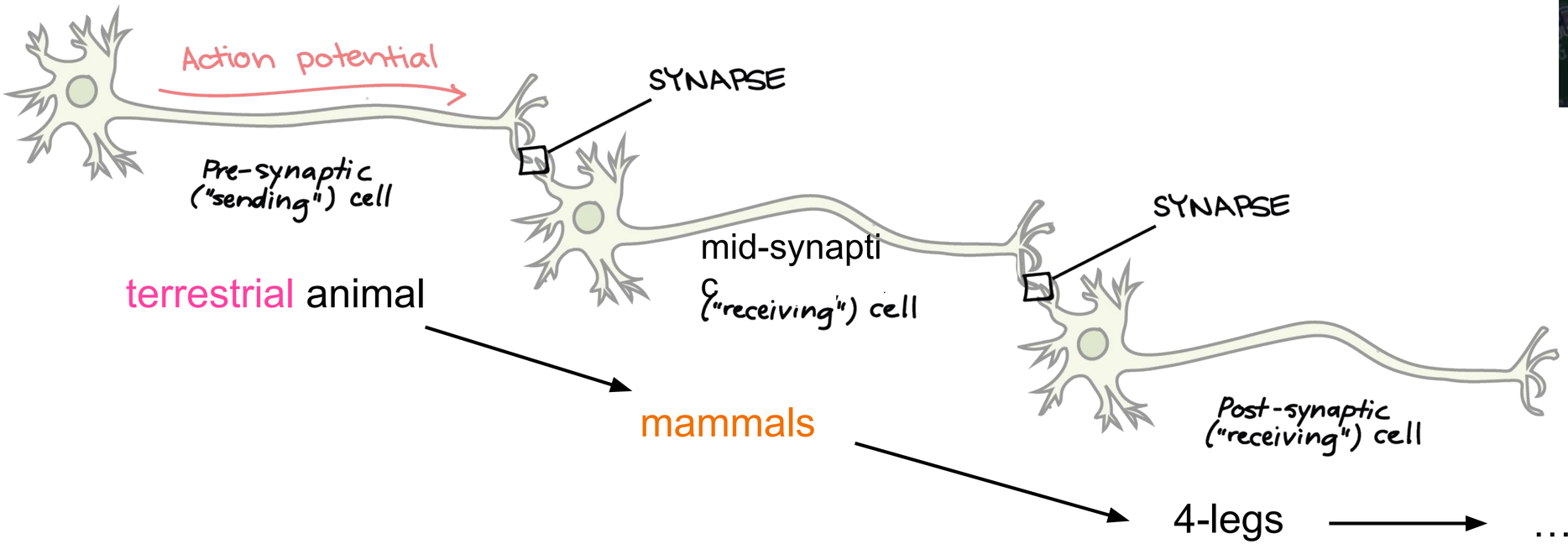


source: South China Morning Post

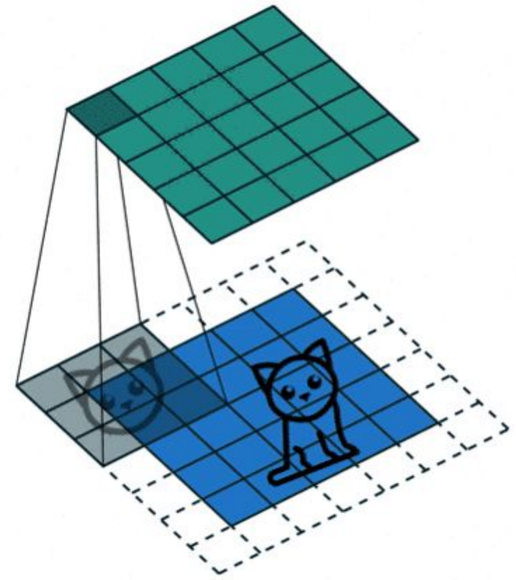
Darwin 3



source: South China Morning Post



source: https://www.khanacademy.org/science/biology/human-biology/neuron-nervous-system/a/the-synapse



source: https://wikidocs.net/177768

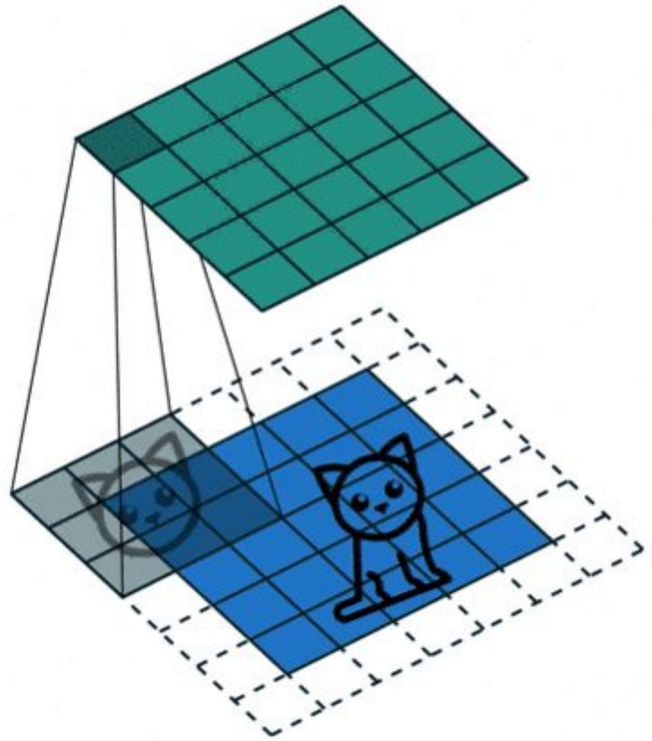
terrestrial (adj) - (of animals) living on the land rather than in the water or air.

mammal (n) - any animal of which the female feeds her young on milk from her own body.

NPU: Neural Processing Unit

- An **NPU** is a **Neural Processing Unit** — a processor specialized for accelerating **artificial neural network (ANN)**computations, especially **deep learning** tasks.

- **NPU** Primary Role
  - Execute AI workloads faster and more power-efficiently than a **CPU** or **GPU**.
  - Optimized for tensor and matrix operations common in AI models, e.g.,
    - convolutions,
    - dot products,
    - activation functions.
  - Often built into smartphones, edge AI devices, and some servers.



source: https://wikidocs.net/177768

NPU: Neural Processing Unit

- **NPU**: How It Works
  - Uses many small AI-specific cores connected by high-speed data paths.
  - Often uses a dataflow architecture — moves data through hardware "**pipelines**" instead of fetching it back and forth from memory.
  - Supports quantization (e.g., INT8, FP16) to speed up inference and reduce power use.

- **NPU**: Where It's Used
  - On-device AI (smartphones, cameras, IoT)
    - Face recognition
    - Voice assistants
    - Real-time translation
  - Cloud AI accelerators (servers with NPUs for inference at scale).

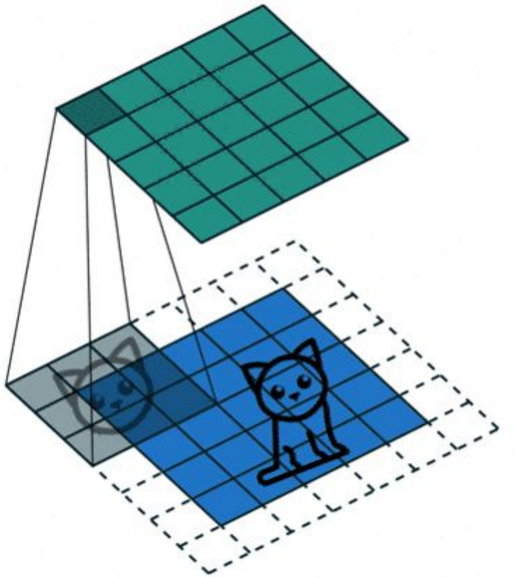
Darwin NPU

- **Darwin** is a **neuromorphic** **co-processor** — not a traditional **CPU** or AI chip per se.
    - The phrase “chip per se” here means “chip in the strict sense of the word” or “chip as normally defined.”
- Darwin is still a silicon processor, but not a “CPU” or “**GPU**” as people normally mean — it’s a special-purpose neuromorphic processor.

CPU GPU NPU

- Core **Purpose**

Processor	Main Role	Example Workloads
CPU	General-purpose computing — runs OS, apps, logic	Word processing, web browsing, running databases
GPU	Parallel numeric processing — designed for graphics but also great for AI	3D rendering, deep learning matrix ops
Darwin3	Brain-inspired computing — runs spiking neural networks (SNNs)	Event-based vision, robotics control, real-time sensory processing



source: https://wikidocs.net/177768

CPU GPU NPU

- Core **Purpose**

Feature	CPU	GPU	NPU
Purpose	General computing	Parallel math (graphics, AI)	AI-specific
Core Type	Few, complex	Many, simple	Many, AI-optimized
Workload	Flexible, any task	Parallel math, graphics	Neural network ops
Power Efficiency	Moderate	High usage	Very high for AI
Programming	General-purpose languages	CUDA/OpenCL	AI frameworks (TensorFlow Lite, ONNX, etc.)



Key Points: CPU, GPU & NPU

- **CPU (Central Processing Unit)**
  - General-purpose, sequential processing
  - Few powerful cores (4–16), high clock speed
- **GPU (Graphics Processing Unit)**
  - Parallel processing, thousands of cores
  - SIMD/SIMT architecture
  - Modern use: AI, scientific computing
- **NPU (Neural Processing Unit)**
  - AI-specific acceleration
  - Dataflow architecture
- **Darwin 3 (Neuromorphic)**
  - Brain-inspired computing
  - 960 chips, 2 billion artificial neurons

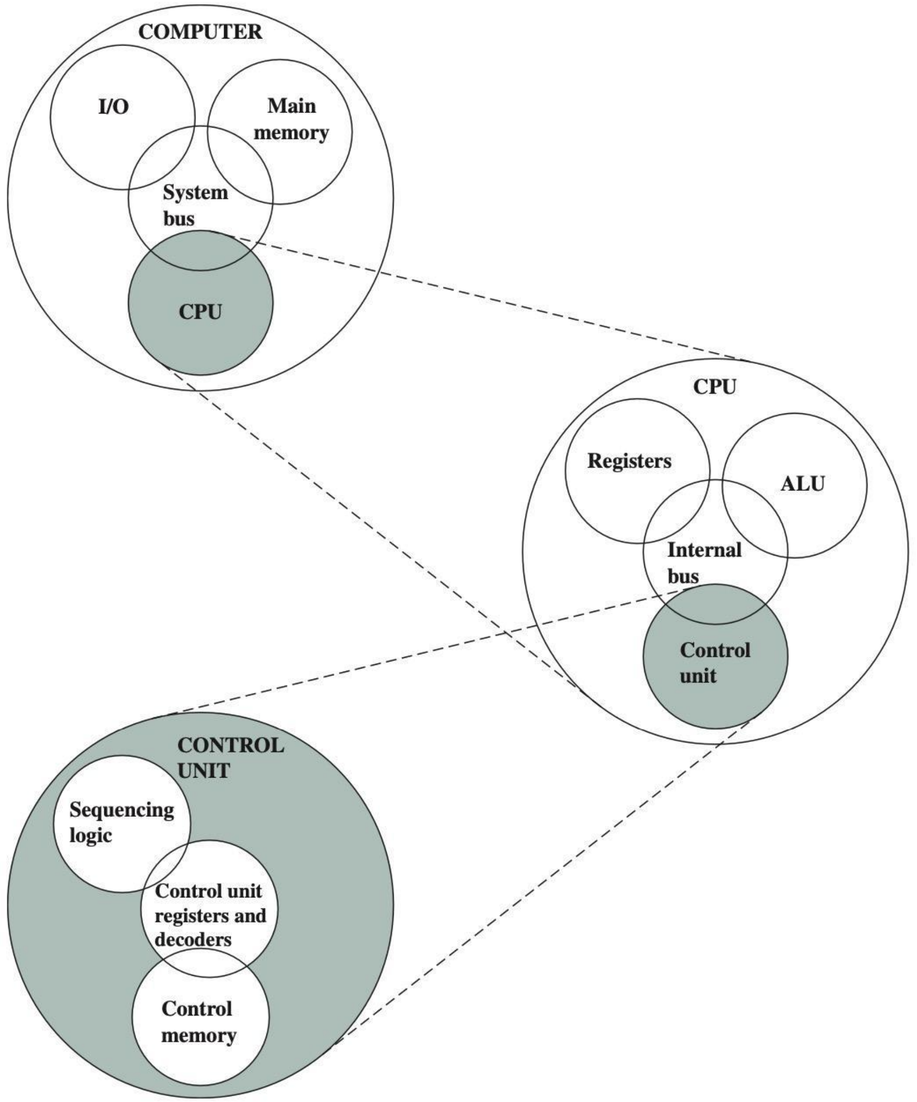
PART C

CPU Major Structural Components

Inside the CPU: the ALU, control unit, registers and interconnections.

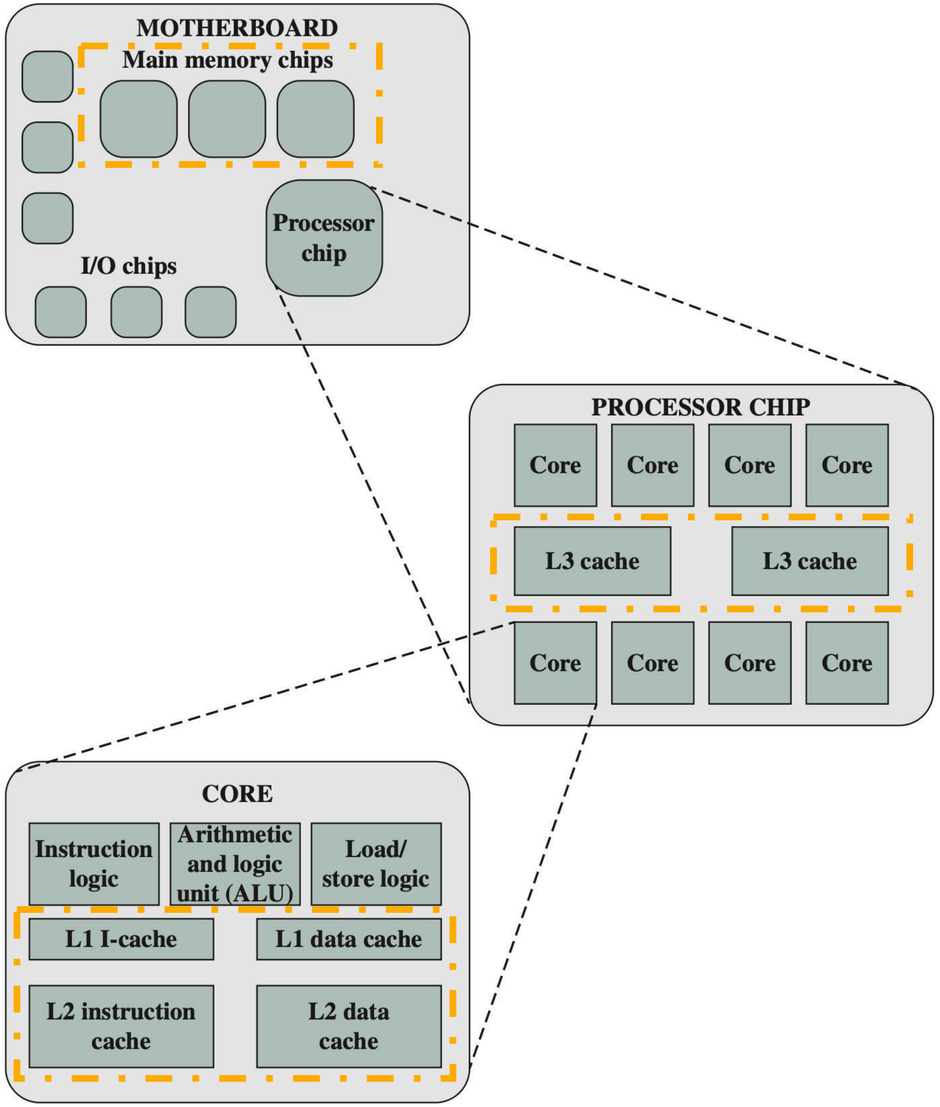
CPU Major Structural Components

- **Control unit:** controls the operation of the CPU and hence the computer.
- **Arithmetic-Logic Unit (ALU):** performs the computer's data processing functions.
- **Registers**: provide storage internal to the CPU.
- **CPU interconnection:** a mechanism that provides for communication among the control unit, ALU, and registers.



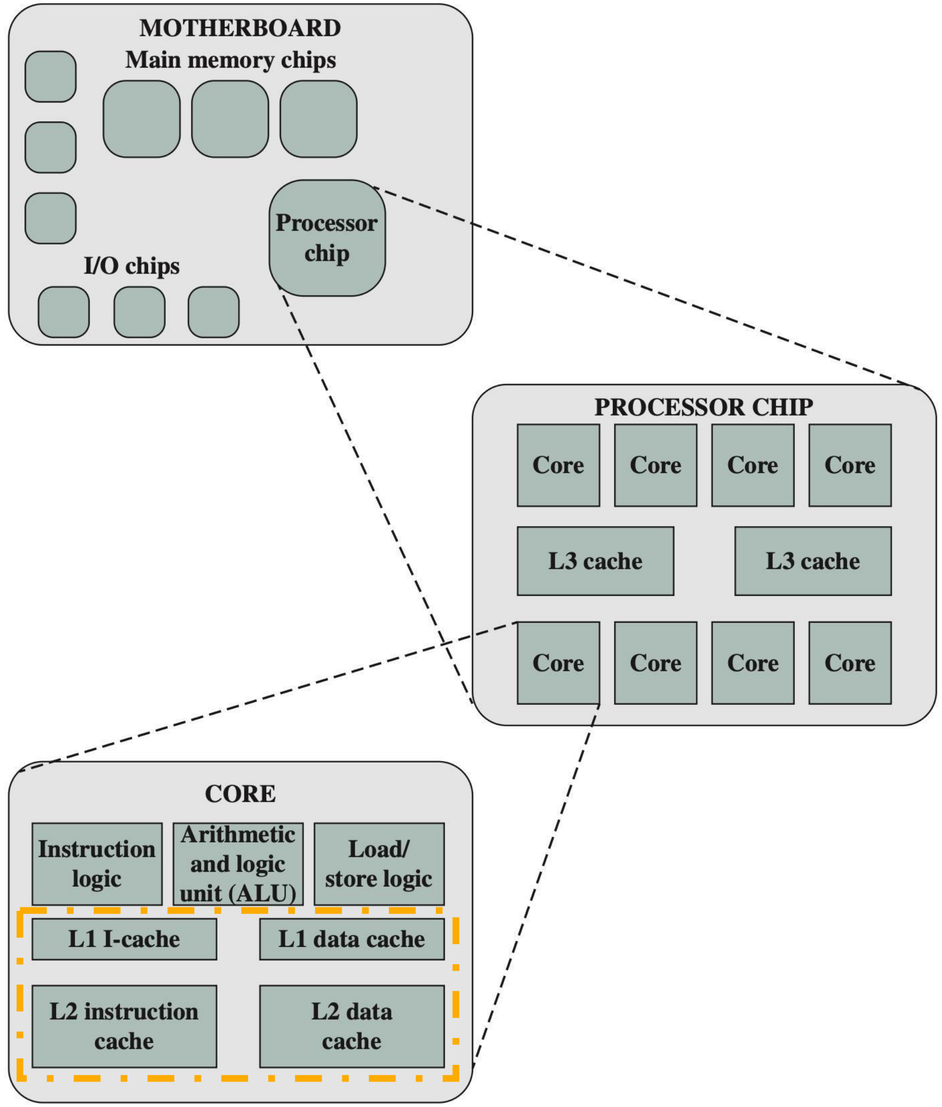
Functional Elements of Core

- **Instruction logic:** This includes the tasks involved in
  - fetching instructions and
  - decoding each instruction to determine the instruction operation and the memory locations of any operands.
- **Arithmetic-Logic Unit (ALU):** performs the operation specified by an instruction.
- **Load/store logic:** manages the transfer of data to and from main memory via cache.



Functional Elements of Core

- The core also contains an **L1 cache**, split between an **instruction cache** (**I-cache**) and an **L1 data cache**.
  - **L1 cache** (**I-cache**) is used for the transfer of instructions to and from main memory.
  - **L1 data cache**, for the transfer of operands and results.
- Today's processor chips also include an **L2 cache** as part of the core.
  - **L2 cache** is also split between **instruction** and **data** caches.
  - Note: Although a combined, single L2 cache is also used.

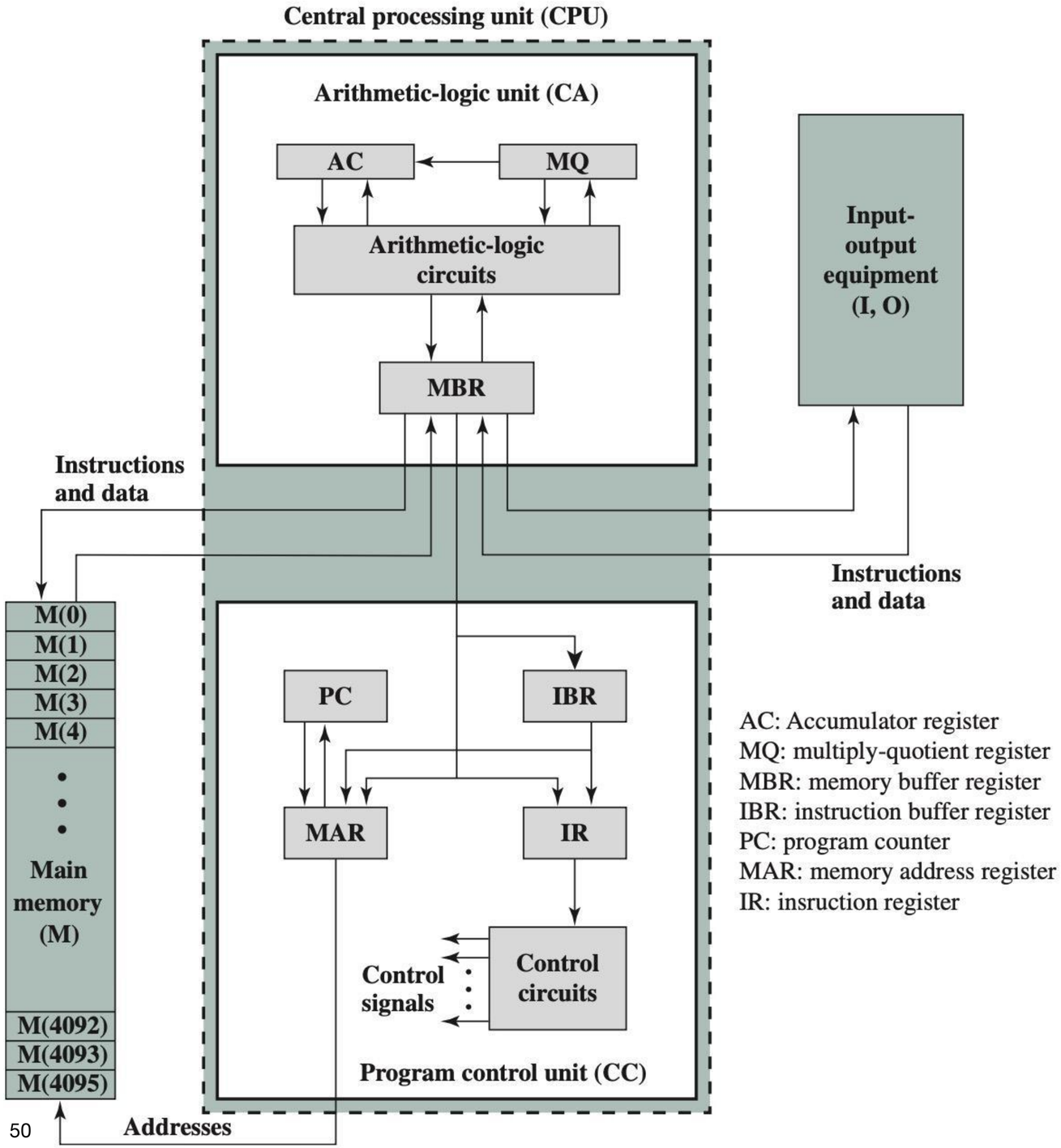


Arithmetic-Logic Unit (ALU)

- The Arithmetic-Logic Unit (**ALU**) is a fundamental component of a computer's Central Processing Unit (**CPU**).
- It is responsible for performing:
  - Arithmetic operations (addition, subtraction, multiplication, division).
  - Logical operations (AND, OR, NOT, XOR, bit shifts, and comparisons).
- The ALU is the "**calculator**" of the CPU, handling all mathematical and decision-making tasks required by software.

Role of the ALU in the CPU

- The **ALU** works in coordination with other **CPU** components to execute instructions.





Key Points: CPU Major Structural Components

- **Control Unit**
  - Controls CPU operations
  - Manages instruction execution flow
- **Arithmetic-Logic Unit (ALU)**
  - Arithmetic: add, subtract, multiply, divide
  - Logical: AND, OR, NOT, XOR
- **Registers**
  - Internal CPU storage
  - Ultra-fast access
- **L1 Cache**
  - Instruction cache (I-cache)
  - Data cache
- **Instruction Logic**
  - Fetching and decoding instructions
  - Determines operation and operand locations

Which of these circuits / functions do not have?

- 2 inputs
- 1 output



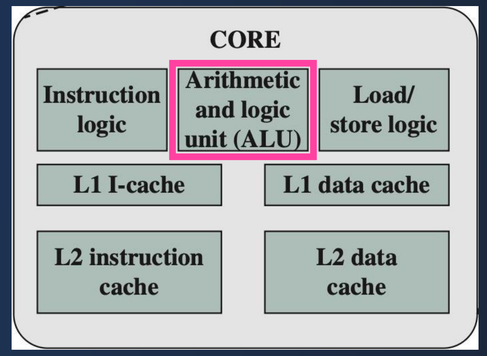
PART D

Arithmetic circuits: + − × ÷

Logic Circuits: == > < ≥ ≤ && || !

Bit Shift Circuits, ….

ALU



The **Arithmetic-Logic Unit:** where the computer actually computes in circuits, inside each core.

Role of the ALU in the CPU

print 2 + 2 ?



Here's how it fits into the CPU's workflow:

1. Inputs to the ALU:
    - **Operands**: The data to be processed (usually from registers or memory).
        - Example: If the instruction is
            - **ADD R1, R2, R3**,
            - ALU takes **R2** and **R3** as **inputs**.

ADD R1, R2, R3

<func> <out> <args>

<Control Signal> <Operands>

    - **Control Signals**: Tell the ALU which operation to perform.
        - Example: **ADD**, **SUB**, **AND**.
        - These signals come from the **Control Unit** (**CU**) after decoding an instruction.

Role of the ALU in the CPU

- Here’s how it fits into the CPU’s workflow:

2. Operations Performed by the ALU

Type	Operations	Example (Assembly)
Arithmetic	Addition (ADD), Subtraction (SUB)	ADD R1, R2, R3
	Multiplication (MUL), Division (DIV)	SUB R4, R5, #10
Logical	AND, OR, NOT, XOR	AND R6, R7, R8
Bit Shifts	Left Shift (LSL), Right Shift (LSR)	LSL R1, R2, #2
Comparison	Compare (CMP), Test (TST)	CMP R1, R2

Role of the ALU in the CPU

- Here's how it fits into the CPU's workflow:

3. Outputs of the ALU
    - **Result:** **Stored back in a register or memory**.
        - Example: **ADD R1, R2, R3** → Result goes to **R1**.

    - **Status Flags:** The **ALU sets flags** in the **status register**, e.g.,
        - **Zero**,
        - **Carry**,
        - **Overflow**.
    - Example:
        - After **CMP R1, R2**, the **Zero Flag (ZF)** is **set if R1 == R2**.

Role of the ALU in the CPU

- Here's how it fits into the CPU's workflow:
  4. Interaction with Other CPU Components
     - **Registers:** Provide input data and store results.
     - **Control Unit (CU):** Decodes instructions and sends control signals to the ALU.
     - **Memory:** If operands come from RAM, they are loaded into registers first.

Role of the ALU in the CPU

- Without the ALU, the CPU **couldn’t perform calculations** or **make decisions**.
- It directly affects **performance**; a faster ALU results in faster computations.
- Modern CPUs have **multiple ALUs** for parallel processing, e.g., in superscalar architectures.



Key Points: ALU

- **ALU Role in CPU**
  - Core computational component
  - Works with registers and control unit
- **Inputs to the ALU**
  - Operands from registers or memory
  - Operation code from control unit
- **Operations**
  - Arithmetic: add, subtract, multiply, divide
  - Logic: AND, OR, NOT, comparison
- **Outputs of the ALU**
  - Result stored in register or memory
  - Status flags (zero, carry, overflow)
- **Performance Impact**
  - Faster ALU = faster computations
  - Multiple ALUs in superscalar architectures

PART E



2 Main Types & 3 Flavours: Instruction Set Architectures (ISAs)

Introduction to what instruction sets are, and how they define the hardware-software contract.

Instruction Set Architectures (ISAs)

- **MIPS**, **ARM**, and **x86** are all instruction set architectures (**ISAs**) that define how a computer processor executes commands.
  - **MIPS** and **ARM** are **RISC** (Reduced Instruction Set Computing) architectures.
  - **x86** is a **CISC** (Complex Instruction Set Computing) architecture.
- Each architecture has its own strengths and weaknesses, influencing its suitability for different applications.
- Quick Analogy:
  - MIPS = simple, clean textbook example of how a **CPU** should be designed.
  - ARM = RISC done for mass-market commercial products.
  - x86 = legacy-rich, complex powerhouse that dominates PCs and servers.

ARM

- Reduced **Instruction Set** Computing (**RISC**) Architecture:
  - ARM is an RISC architecture with simpler, fixed-length instructions.
- **Low Power Consumption:**
  - ARM processors are renowned for their energy efficiency, making them ideal for
    - mobile devices,
    - embedded systems, and
    - other power-sensitive applications.
- **Examples:**
  - Found in smartphones, tablets, embedded systems, and increasingly in laptops and servers.

MIPS

- Reduced **Instruction Set** Computing (**RISC**) Architecture:
  - MIPS is also a RISC architecture, known for its simplicity and efficiency.
- **Embedded Systems Focus:**
  - Historically, MIPS has been popular in
    - embedded systems,
    - networking equipment, and
    - other specialized applications.
- **Examples:**
  - Used in routers, networking devices, and some embedded systems.

x86

- Complex **Instruction Set** Computing (**CISC**) Architecture:
  - x86 utilizes a complex instruction set, with instructions that can perform multiple operations in a single step.
- **Variable Length Instructions:**
  - x86 instructions can have varying lengths, making them more complex to decode and execute.
- **High Performance:**
  - Historically, x86 has been dominant in high-performance computing, such as desktops, laptops, and servers, due to its focus on raw processing power.
- **Examples:**
  - Intel and AMD processors.

ISAs

- Design Philosophy: **RISC** vs **CISC**

Feature	ARM	MIPS	x86
Type	**RISC (Reduced Instruction Set Computer)**	**RISC**	**CISC (Complex Instruction Set Computer)**
Goal	Fewer, simpler instructions that execute quickly	Same as ARM — simple, load/store model	Many complex instructions, some doing multiple steps in one opcode
Instruction Length	Fixed (usually 32-bit, some 16-bit)	Fixed 32-bit	**Variable** (1–15 bytes per instruction)
Registers	16 general-purpose (32-bit in ARMv7, 64-bit in ARMv8)	32 general-purpose	**Few general-purpose** (x86-64 has 16, but original x86 had only 8)



Key Points: Instruction Set Architectures (ISAs)

- **ISA Comparison**
  - RISC:
    - Example Instruction Set Architecture:
      - ARM,
      - MIPS
    - simple
    - fixed-length instructions
  - CISC:
    - Example Instruction Set Architecture:
      - x86
    - complex
    - variable-length instructions
    - general purpose

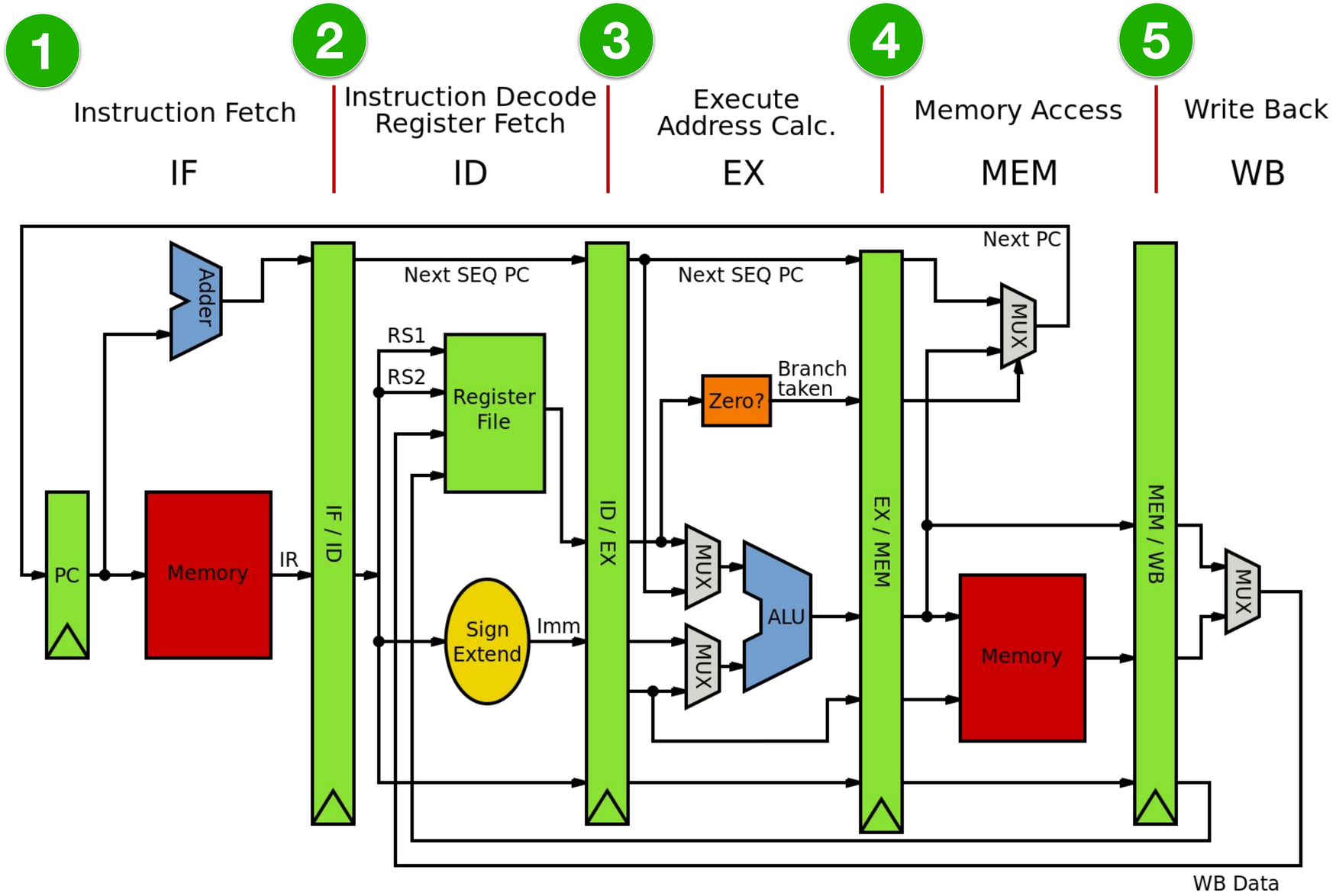
PART E

Stages:
1. Instruction Fetch
2. Instruction Decode & Register Fetch
3. Execute
4. Memory Access
5. Write Back

Pipeline Stages of: Instruction Set Architectures (ISAs)

What instruction sets are, and how they define the hardware-software contract.

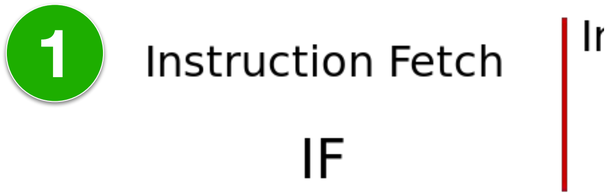
Pipeline Stages of Instruction Set Architectures: MIPS



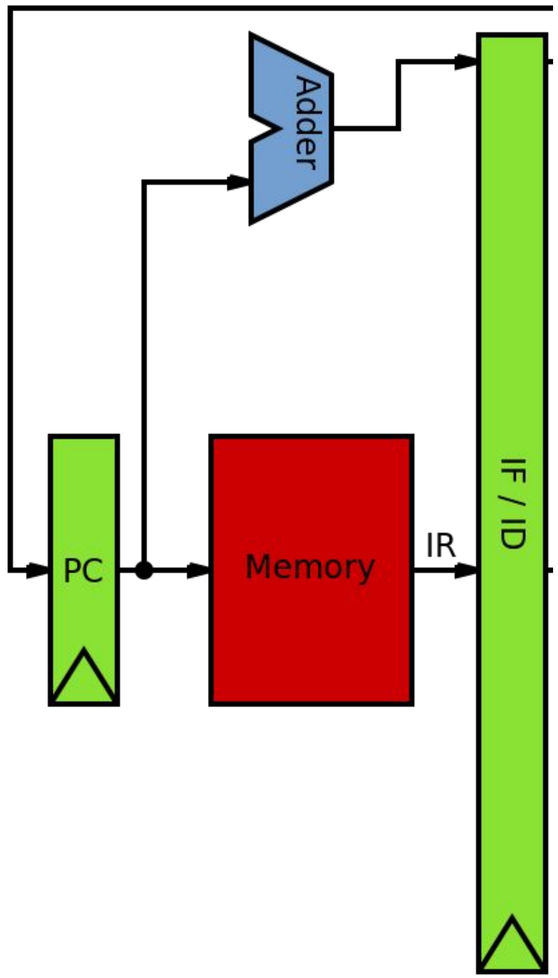
Stages:

1. Instruction Fetch
2. Instruction Decode & Register Fetch
3. Execute
4. Memory Access
5. Write Back

ISA: MIPS



- **Instruction Fetch** (IF) Stage
  - **PC** (**Program Counter**) – holds the address of the current instruction being fetched from memory.
  - **Adder** – adds 4 to the PC to get the address of the next sequential instruction
    - (since MIPS instructions are 4 bytes).
  - **Instruction Memory** (Memory in red) – stores the program's instructions; outputs the instruction corresponding to the PC.
  - **IF/ID Pipeline Register** – stores the
    - fetched instruction and
    - PC+4 for use in the next stage.



ISA: MIPS

2 Instruction Decode Register Fetch

ID

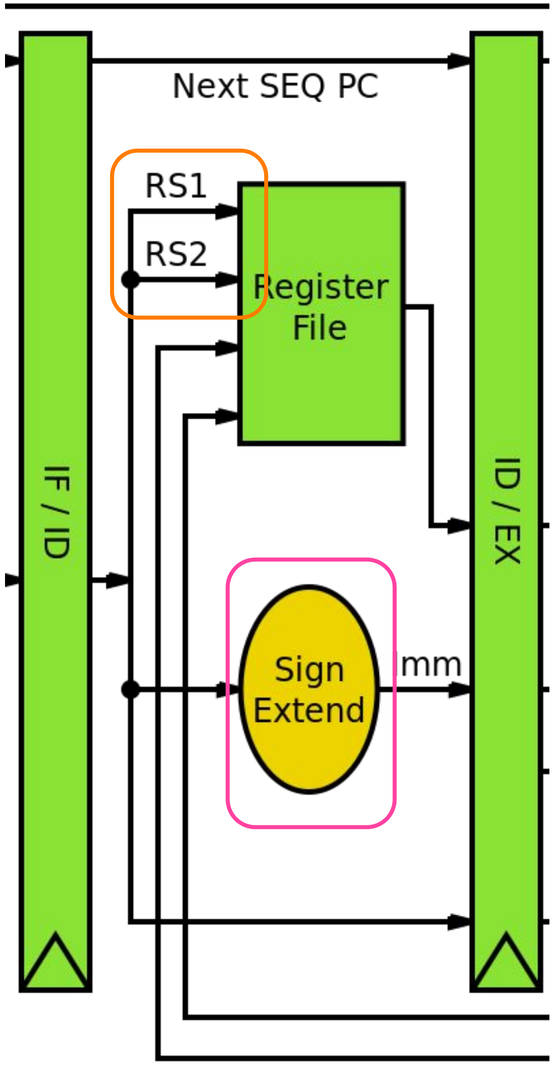
- **Instruction Decode** / **Register Fetch** (ID) Stage

  - **Register File** – contains 32 general-purpose registers.
    - Two registers are read (rs, rt), and
    - One register can be written (rd).

  IN: rs, rt
  OUT: rd

  - Sign Extend:
    - Extends a 16-bit immediate value from the instruction
    - into a 32-bit signed value
      (needed for arithmetic or address calculations).

  - **ID/EX Pipeline Register** – holds
    - register values,
    - sign-extended immediate,
    - control signals, and
    - PC+4 for the Execute stage.

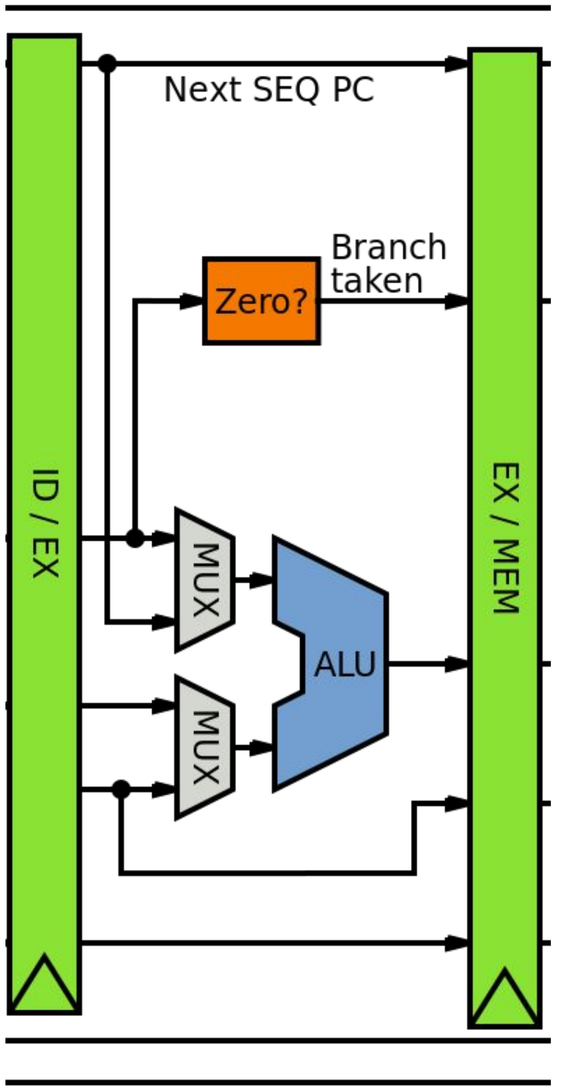


ISA: MIPS



Execute / **Address Calculation** (EX) Stage

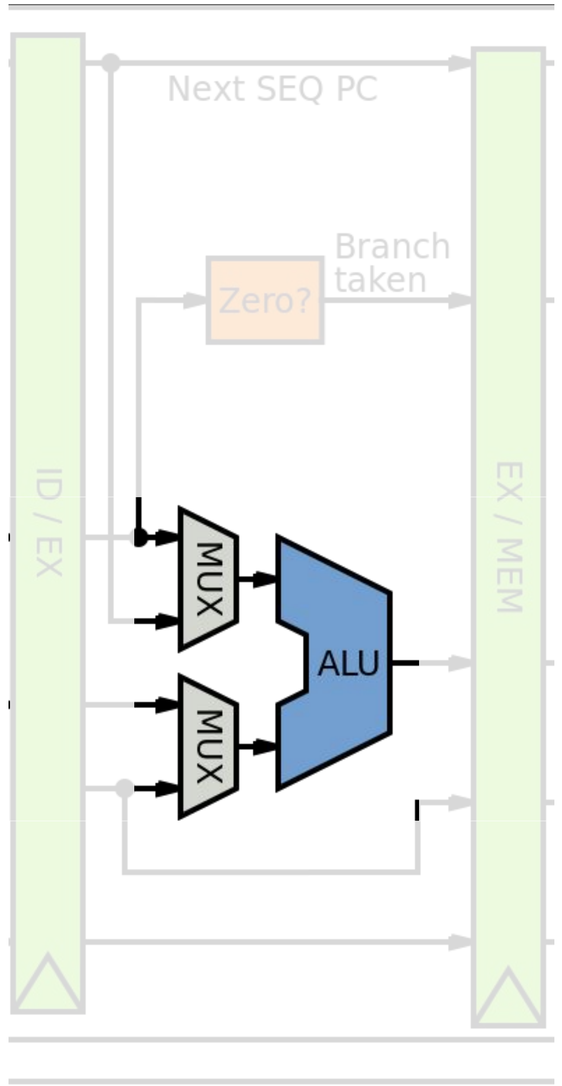
- **MUX** (**Multiplexers**) – select between inputs, e.g., between register value or sign-extended immediate for ALU input (for **R-type** vs. **I-type** instructions).
- **ALU** (**Arithmetic-Logic Unit**) – performs arithmetic or logical operations, e.g., addition, subtraction, AND, OR, and set less than.
- **Zero?** (**Zero Detector**) – checks if ALU output is zero (used for branch decisions).
- **EX/MEM Pipeline** Register – stores
  - ALU results,
  - Zero flag,
  - control signals, and
  - data to be written to memory.



ISA: MIPS

3 Execute Address Calc.

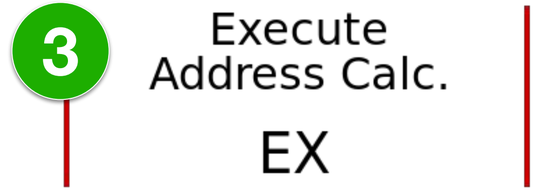
EX



3 R-type vs. I-type

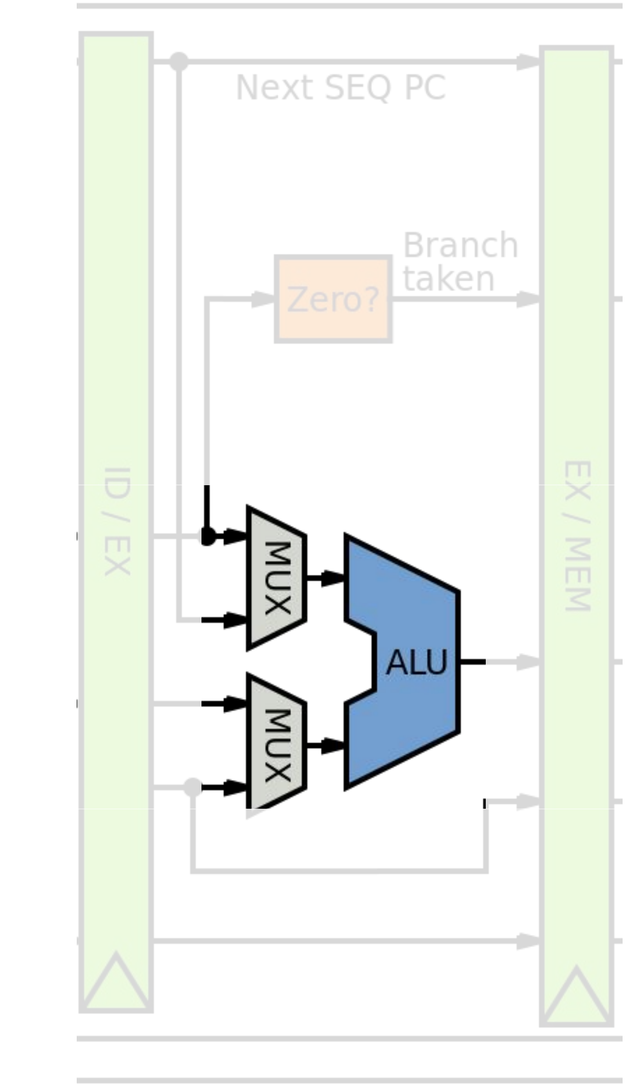
Feature	R-type	I-type
Operands	All from registers	One register + immediate
Immediate value	No	Yes (16-bit)
Memory access	No	Yes (for lw, sw)
Destination	rd	rt
Example	**add $t0, $t1, $t2**	**addi $t0, $t1, 5**

ISA: MIPS



Execute / **Address Calculation** (EX) Stage

- **R-type** (**Register-type**) Instructions
  - Purpose: Operations that use only registers (no immediate values or memory addresses).



ISA: MIPS

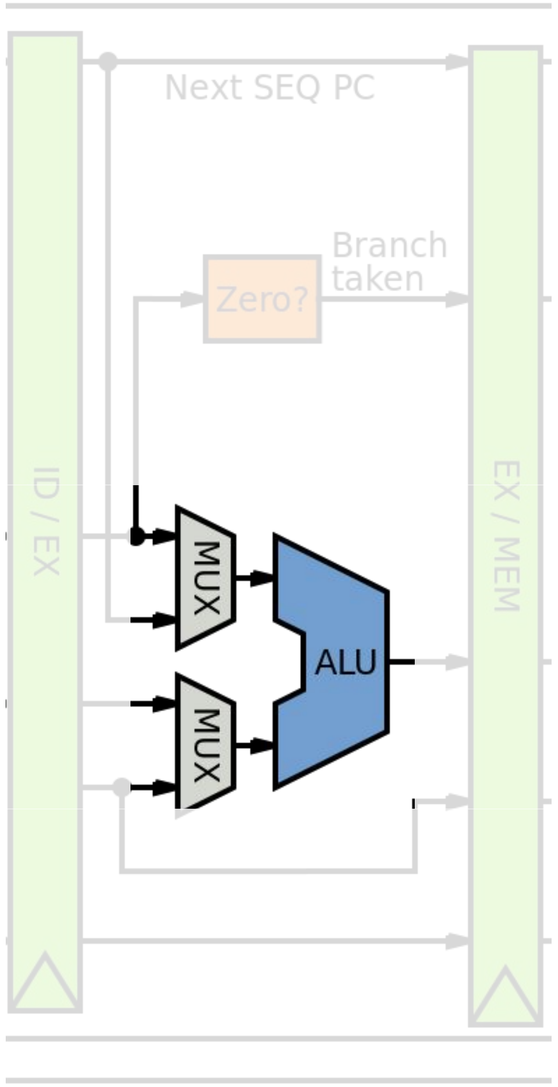


Execute / **Address Calculation** (EX) Stage

- **R-type** (**Register-type**) Instructions
  - Purpose: Operations that use only registers (no immediate values or memory addresses).
  - Format:

    | opcode (6 bits) | rs (5 bits) | rt (5 bits) | rd (5 bits) | shamt (5 bits) | funct (6 bits) |

  - **Fields:**
    - **opcode**: Usually 000000 for **R-type** (the specific operation is in funct).
    - rs: Source register 1.
    - rt: Source register 2.
    - rd: Destination register.
    - shamt: Shift amount (used for shift instructions, else 0)



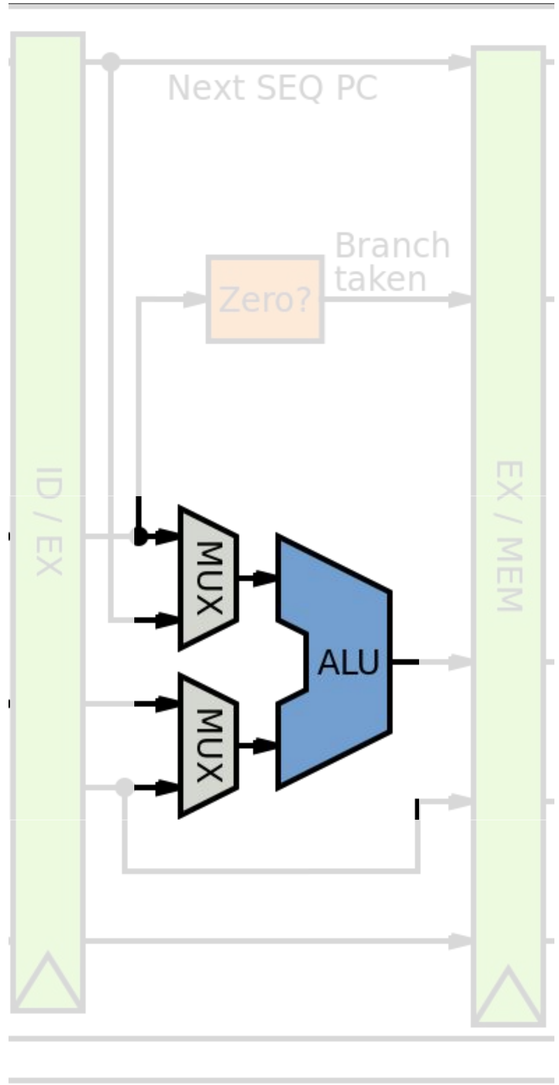
ISA: MIPS



- Execute / **Address Calculation** (EX) Stage
  - **R-type** (**Register-type**) Instructions
    - Purpose: Operations that use only registers (no immediate values or memory addresses).
    - Format:

| **opcode** (6 bits) | rs (5 bits) | rt (5 bits) | rd (5 bits) | shamt (5 bits) | funct (6 bits) |

- **Fields:**
  - …
  - funct: Function code that specifies the exact ALU operation.



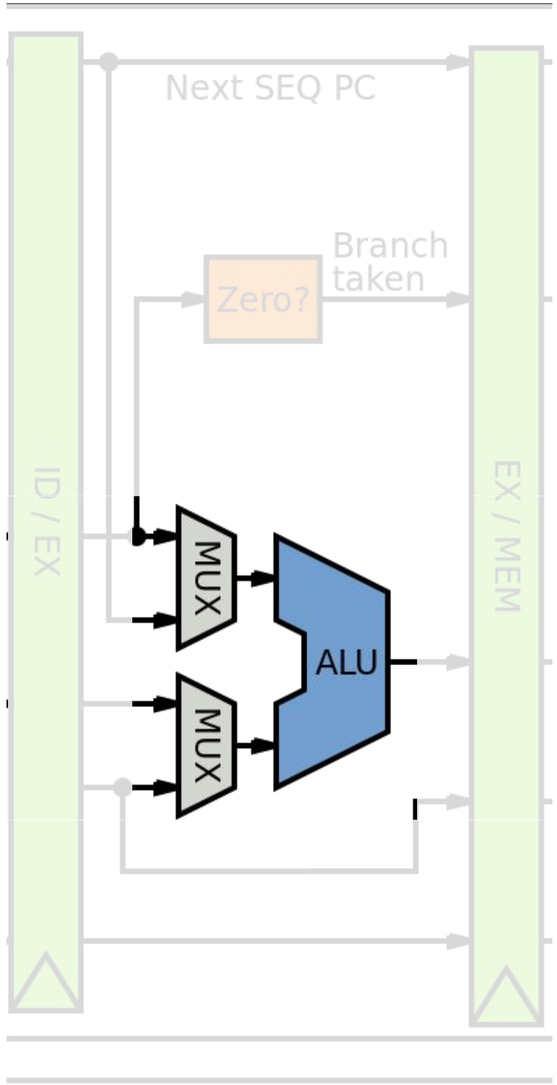
ISA: MIPS



- Execute / **Address Calculation** (EX) Stage
  - **R-type** (**Register-type**) Instructions
    - Purpose: Operations that use only registers (no immediate values or memory addresses).
    - Format:

| **opcode** (6 bits) | rs (5 bits) | rt (5 bits) | rd (5 bits) | shamt (5 bits) | funct (6 bits) |

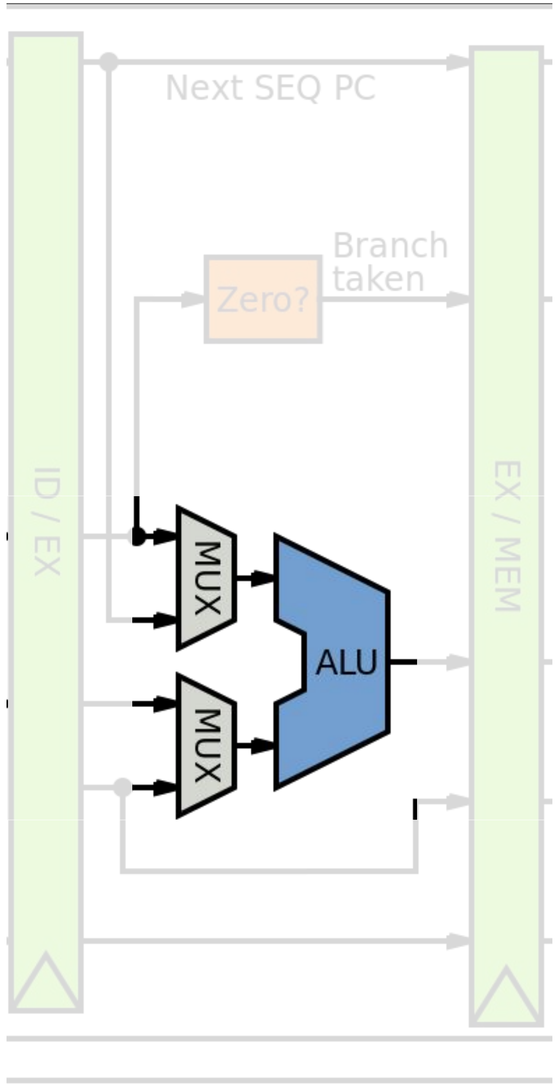
    - **Examples:**
      - add $t0, $t1, $t2 → $t0 = $t1 + $t2
      - sub $s0, $s1, $s2
      - and $a0, $a1, $a2
      - sll $t0, $t1, 2 (shift left logical)



ISA: MIPS



- Execute / **Address Calculation** (EX) Stage
  - **I-type (Immediate-type)** Instructions
    - Purpose: Instructions that use an immediate constant (or a short offset for memory/branching).
    - Format:
      | opcode (6 bits) | rs (5 bits) | rt (5 bits) | immediate (16 bits) |
    - **Fields:**
      - **opcode**: Specifies the operation.
      - rs: Source register.
      - rt: Destination register (or source for store).
      - immediate: 16-bit constant or offset (sign-extended to 32 bits).

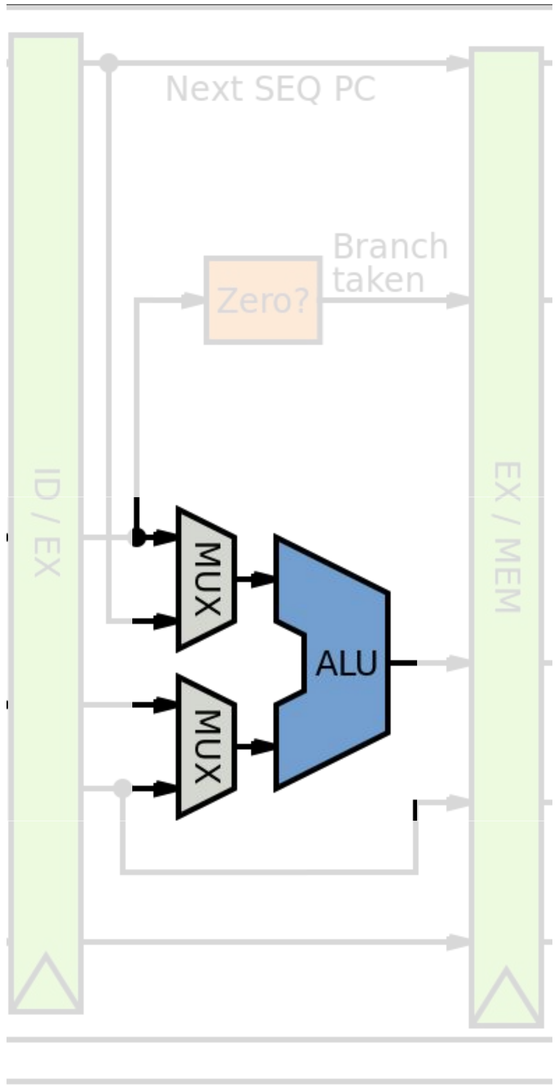


ISA: MIPS



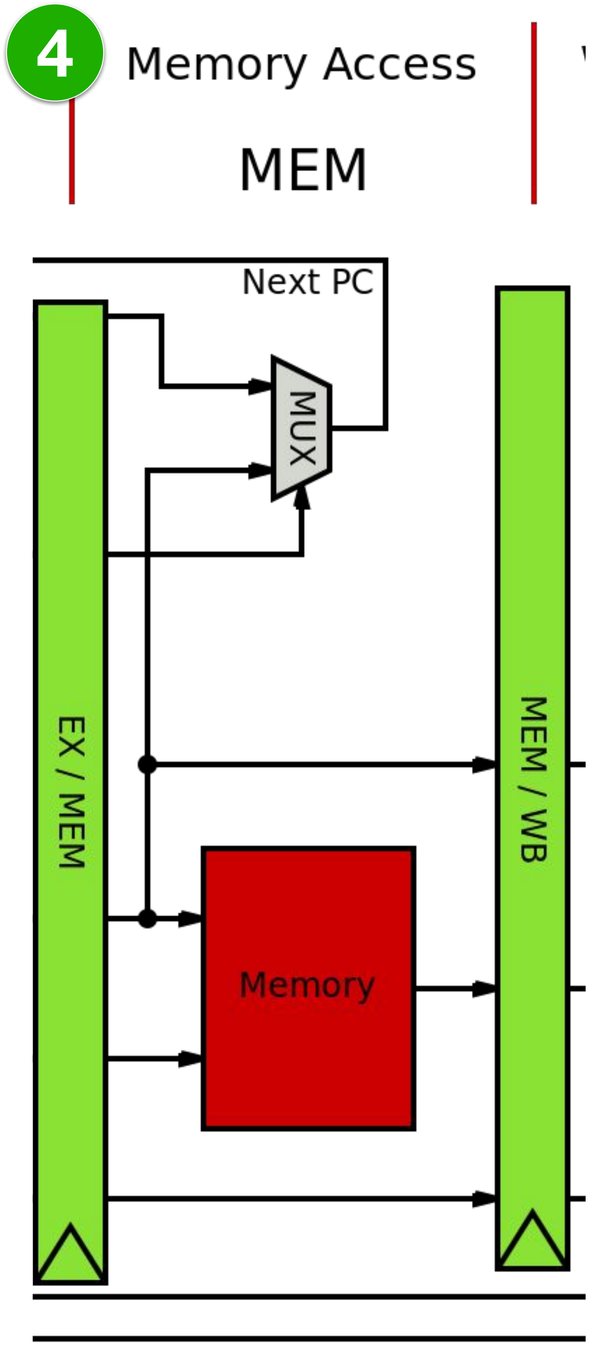
- Execute / **Address Calculation** (EX) Stage
  - **I-type (Immediate-type)** Instructions
    - Purpose: Instructions that use an immediate constant (or a short offset for memory/branching).
    - Format:

      | **opcode** (6 bits) | rs (5 bits) | rt (5 bits) | immediate (16 bits) |
    - **Examples:**
      - lw $t0, 8($sp) → Load from memory at address $sp + 8
      - sw $t0, 8($sp) → Store to memory at address $sp + 8
    - Branching:
      - beq $t0, $t1, label → If equal, branch to label



ISA: MIPS

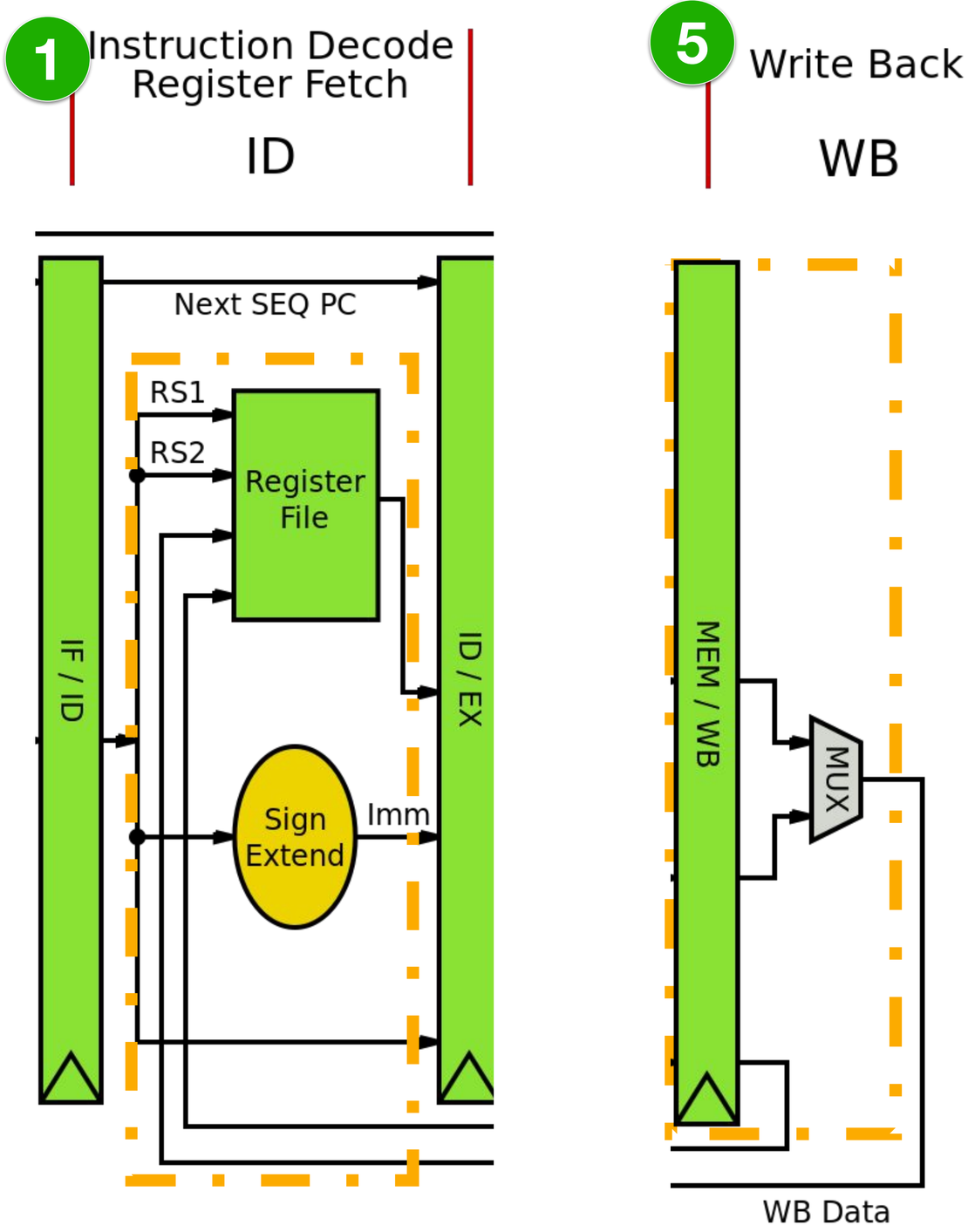
- **Memory Access** (MEM) Stage
  - **Data Memory** (Memory in red) – accessed if instruction is lw (load word) or sw (store word).
    - For lw: reads data from memory.
    - For sw: writes data to memory.
  - **MUX** – selects whether the value going to the next stage comes from the ALU result or the memory output.
  - **MEM/WB Pipeline Register** – holds values from memory or ALU results for the **Write Back** stage.



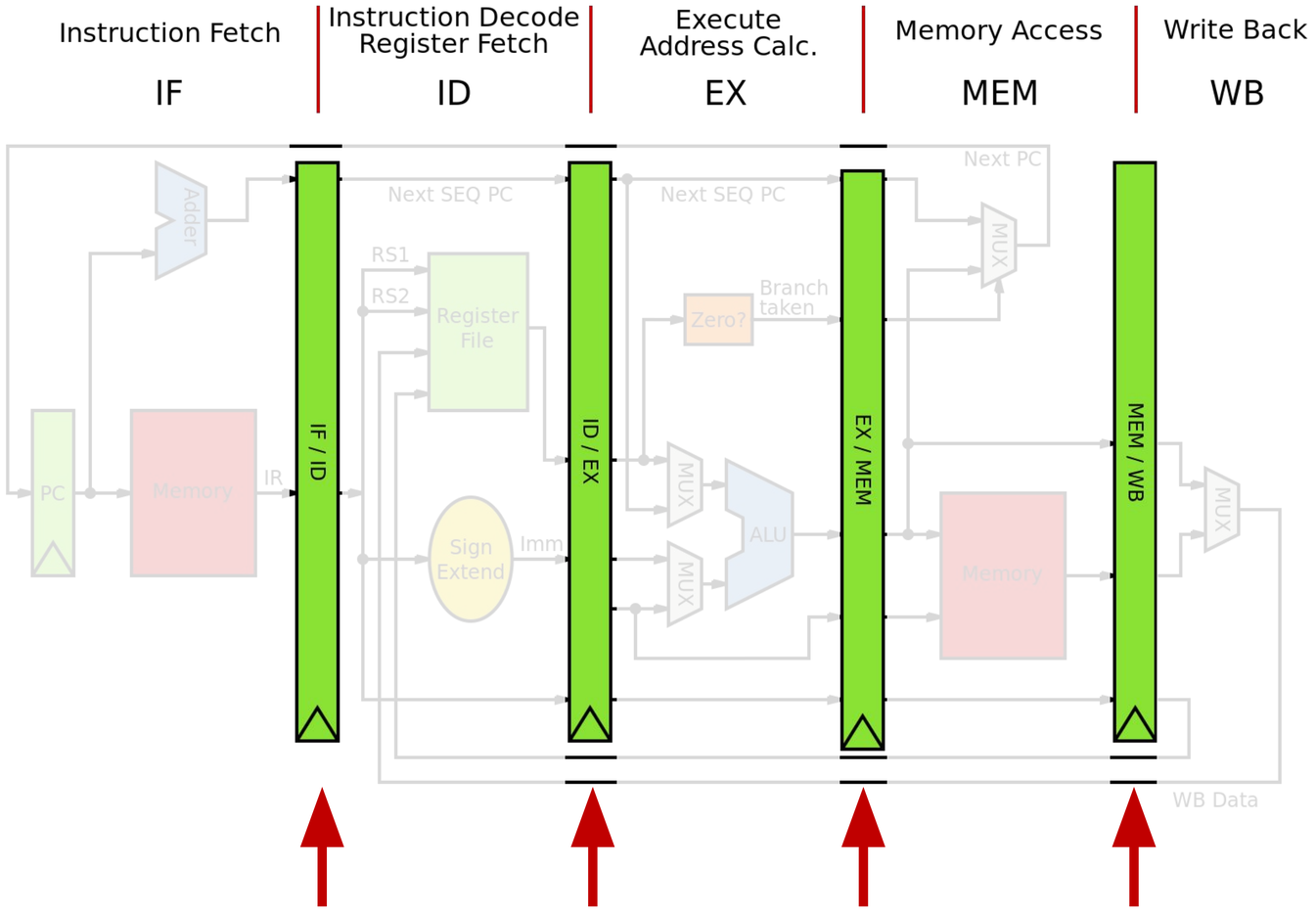
ISA: MIPS

- **Write Back** (WB) Stage
  - **MUX** – chooses between ALU result and memory data to write into the register file.
  - **Register File** (write port) – Writes the final result back into the destination register.

- **Control Signals** (not fully shown here)
  - Throughout the pipeline, control signals (generated in the ID stage) determine:
    - ALU operation
    - Memory read/write
    - Register write enable
    - MUX selections



ISA: MIPS



Pipeline Registers

(vertical green bars)

They store the **intermediate data** and **control signals** between stages.

- **IF/ID** – Between IF and ID
- **ID/EX** – Between ID and EX
- **EX/MEM** – Between EX and MEM
- **MEM/WB** – Between MEM and WB

Allow **multiple instructions** to be in **different stages at the same time.**





Key Points: Instruction Set Architectures (ISAs)

- **MIPS Pipeline Stages**
  - MUX – Multiplexer (for routing instructions)
  - IF → ID → EX → MEM → WB
    1. IF – Instruction Fetch
    2. ID – Instruction Decode (from Register Address)
    3. EX – Execute (at address)
    4. MEM – Memory Access
    5. WB – Write Back (to register/ memory address).

- **Instruction Types**
  - R-type: register-to-register operations
  - I-type: immediate values and memory access

- **Pipeline Registers**
  - Store intermediate data between stages
  - IF/ID, ID/EX, EX/MEM, MEM/WB

PART F.1

Programming Assembly: for ARM Instruction Set Arch.

Writing assembly across instruction sets to see ISAs in action.

Acorn RISC Machine (ARM)

- ARM originally Acorn RISC Machine, now Advanced RISC Machine.
- ARM is a family of Reduced **Instruction Set** Computing (RISC) architectures used in **CPU**s.
- It is designed for efficiency, making it dominant in:
  - Mobile devices (Smartphones, Tablets: Apple Silicon, Snapdragon).
  - Embedded systems (Raspberry Pi, IoT devices).
  - Modern laptops/desktops (Apple M1/M2, Windows on ARM).

- https://cpulator.01xz.net/?sys=arm

Acorn RISC Machine (ARM)

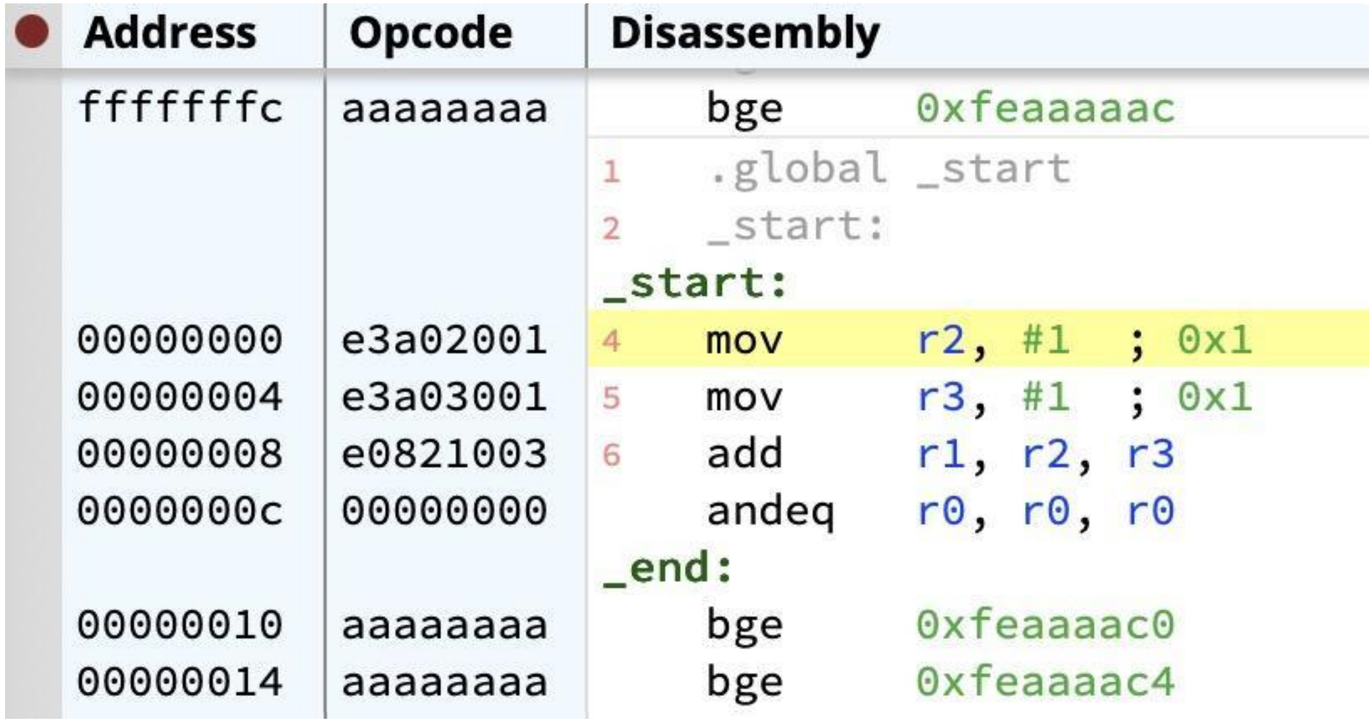
- Origin: Developed by Acorn Computers in the late 1980s (now ARM Ltd.).
- Instruction format:
  - Also fixed-width, also
  - 3-operand style,
  - but with different register names (R0, R1, etc.).
- Registers: R0–R15 (with special ones like PC, LR, SP).
- Use in industry: Dominates smartphones, tablets, and many embedded systems.
  - https://cpulator.01xz.net/?sys=arm

Acorn RISC Machine (ARM)

- For example, write this code in the Editor
  - https://cpulator.01xz.net/?sys=arm

```
.global _start
_start:
    MOV R2, #1
    MOV R3, #1
    ADD R1, R2, R3
```

- Then press “Compile and Load (F5)”
- Then press “Continue”



Acorn RISC Machine (ARM)

- For example, write this code in the Editor
  - https://cpulator.01xz.net/?sys=arm

```
.global _start
_start:
    MOV R2, #5
    MOV R3, #7
    ADD R1, R2, R3
```

- Then press “Compile and Load (F5)”
- Then press “Continue”



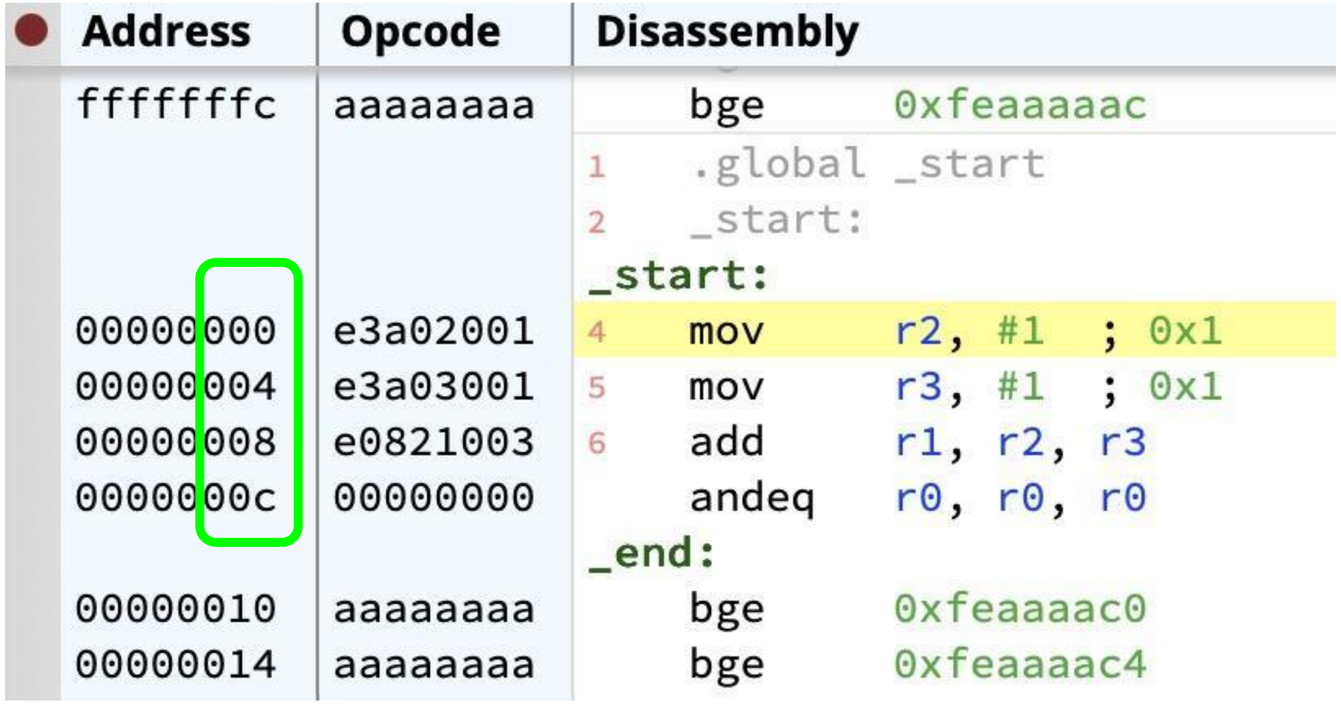
Acorn RISC Machine (ARM)

- In **CPUlator** (ARM mode), we won't see "**output**" printed to the console.

- Instead, we watch the register values change as the program runs.
  (Press "Continue")

Acorn RISC Machine (ARM)

- **00000000, 00000004, etc. are memory addresses** where each instruction is stored.
- ARM instructions are 4 bytes long, so the **address increases by 4** each time.
- These steps walk through:
  - Instruction fetch (Fetch & Decode)
  - Execution
  - Register update



Acorn RISC Machine (ARM)

- Step-by-step decoding (**ARMv7** ARM instruction format)

- ARM data-processing immediate format (for **mov** with an **immediate constant**) is:
  1. **Break into bytes:**

     E3 A0 20 01 in binary: **1110** 0011 1010 **0000** 0010 **0000 0000 0001**

Bits	Purpose
31–28	**Condition code (`cond`)**
27–25	Instruction class (data-processing)
24–21	**Opcode**
20	Set condition flags? (`S`)
19–16	**Destination register (`Rd`)**
15–12	Operand register (`Rn`) — unused in MOV
11–0	**Immediate value (rotated)**

Address	Opcode	Disassembly
fffffffc	aaaaaaaa	bge 0xfeaaaaac
		1 .global _start
		2 _start:
		_start:
00000000	e3a02001	4 mov r2, #1 ; 0x1
00000004	e3a03001	5 mov r3, #1 ; 0x1
00000008	e0821003	6 add r1, r2, r3
0000000c	00000000	andeq r0, r0, r0
		_end:
00000010	aaaaaaaa	bge 0xfeaaaac0
00000014	aaaaaaaa	bge 0xfeaaaac4

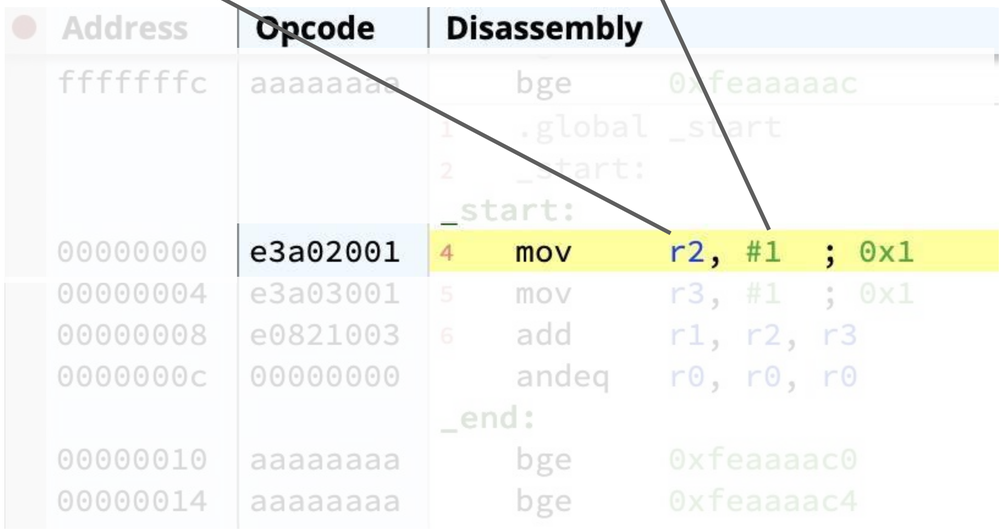
Acorn RISC Machine (ARM)

- Step-by-step decoding (**ARMv7** ARM instruction format)

- ARM data-processing immediate format (for **mov** with an **immediate constant**) is:
  1. Break into bytes:

E3 A0 20 01 in binary: **1110** 0011 1010 **0000** 0010 **0000 0000 0001**

Bits	Purpose
31–28	**Condition code (`cond`)**
27–25	Instruction class (data-processing)
24–21	**Opcode**
20	Set condition flags? (`S`)
19–16	**Destination register (`Rd`)**
15–12	Operand register (`Rn`) — unused in MOV
11–0	**Immediate value (rotated)**



Acorn RISC Machine (ARM)

- Step-by-step decoding (**ARMv7** ARM instruction format)

- ARM data-processing immediate format (for **mov** with an **immediate constant**) is:
  1. Break into bytes:

  E3 **A0** 2**0 01** in binary: **1110** 0011 1010 **0000** 0010 **0000 0000 0001**

Bits	Purpose
31–28	**Condition code (`cond`)**
27–25	Instruction class (data-processing)
24–21	**Opcode**
20	Set condition flags? (`S`)
19–16	**Destination register (`Rd`)**
15–12	Operand register (`Rn`) — unused in MOV
11–0	**Immediate value (rotated)**

```
Address    Opcode     Disassembly
fffffffc   aaaaaaaa       bge     0xfeaaaaac
                       1  .global _start
                       2  _start:
                       _start:
00000000   e3a02001    4  mov     r2, #1  ; 0x1
00000004   e3a03001    5  mov     r3, #1  ; 0x1
00000008   e0821003    6  add     r1, r2, r3
0000000c   00000000       andeq   r0, r0, r0
                       _end:
00000010   aaaaaaaa       bge     0xfeaaaac0
00000014   aaaaaaaa       bge     0xfeaaaac4
```

Acorn RISC Machine (ARM)

- Step-by-step decoding (**ARMv7** ARM instruction format)

- ARM data-processing immediate format (for **mov** with an **immediate constant**) is:

1. Break into bytes:

E3 **A0** 2**0** **01** in binary: **1110** 0011 1010 **0000** 0010 **0000 0000 0001**

```
e3a02001    4     mov     r2, #1  ; 0x1
```

2. Decode **Condition Code** (bits 31–28)

1110 → 0xE → **AL** = Always execute

See MyCourses → Unit-2
→ Cheat Sheet table (Condition Codes)

Acorn RISC Machine (ARM)

Recognizing Hexadecimal from Binary



Is $1110_2 = 14_{10} = E_{16}$?

2. Decode Condition Code (bits 31–28)

1110 → 0xE → **AL** = Always execute

- `0xde0` in hexadecimal is:

$$d \times 16^2 + e \times 16^1 + 0 \times 16^0$$

$$d = 13, e = 14 \quad \text{(hexadecimal digits)}$$

$$0xde0 = 13 \times 256 + 14 \times 16 + 0 = 3328 + 224 + 0 = 3552 \,\text{(decimal)}$$

- `0xdc8` in hexadecimal is:

$$d \times 16^2 + c \times 16^1 + 8 \times 16^0$$

$$c = 12, d = 13, 8 = 8 \quad \text{(hexadecimal digits)}$$

$$0xdc8 = 13 \times 256 + 12 \times 16 + 8 = 3328 + 192 + 8 = 3528 \,\text{(decimal)}$$

Acorn RISC Machine (ARM)

- ARM data-processing immediate format (for mov with an immediate constant) is:
  1. Break into bytes:

E3 A0 20 01 in binary: 1110 0011 1010 0000 0010 0000 0000 0001

```
e3a02001    4    mov     r2, #1  ; 0x1
```

  3. Bits 27–25 (Instruction class)
     001 → Data-processing immediate
  4. Bits 24–21 (**Opcode**)
     1101 → **MOV opcode** (in ARM data-processing instructions)
  5. Bit 20 (S flag)
     0 → MOV Do not update CPSR flags
  6. Bits 19–16 (Rn)
     0000 → R0 (unused for MOV immediate)

Acorn RISC Machine (ARM)

- ARM data-processing immediate format (for mov with an immediate constant) is:
  1. Break into bytes:

E3 A0 20 01 in binary: 1110 0011 1010 0000 0010 0000 0000 0001

```
e3a02001    4    mov     r2, #1  ; 0x1
```

7. Bits 15–12 (Destination register Rd)
   0010 → R2
8. Bits 11–0 (Immediate + rotate)
   0000 0000 0001 → Immediate value 1
   (rotation = 0, so no shift applied)

Acorn RISC Machine (ARM)

- This machine code means:
  - Execute unconditionally
  - Perform **MOV**
    - Rd = R2
  - Immediate = 1

Address	Opcode	Disassembly
fffffffc	aaaaaaaa	bge 0xfeaaaaac
		1 .global _start
		2 _start:
		_start:
00000000	e3a02001	4 mov r2, #1 ; 0x1
00000004	e3a03001	5 mov r3, #1 ; 0x1
00000008	e0821003	6 add r1, r2, r3
0000000c	00000000	andeq r0, r0, r0
		_end:
00000010	aaaaaaaa	bge 0xfeaaaac0
00000014	aaaaaaaa	bge 0xfeaaaac4

PART F.1.1

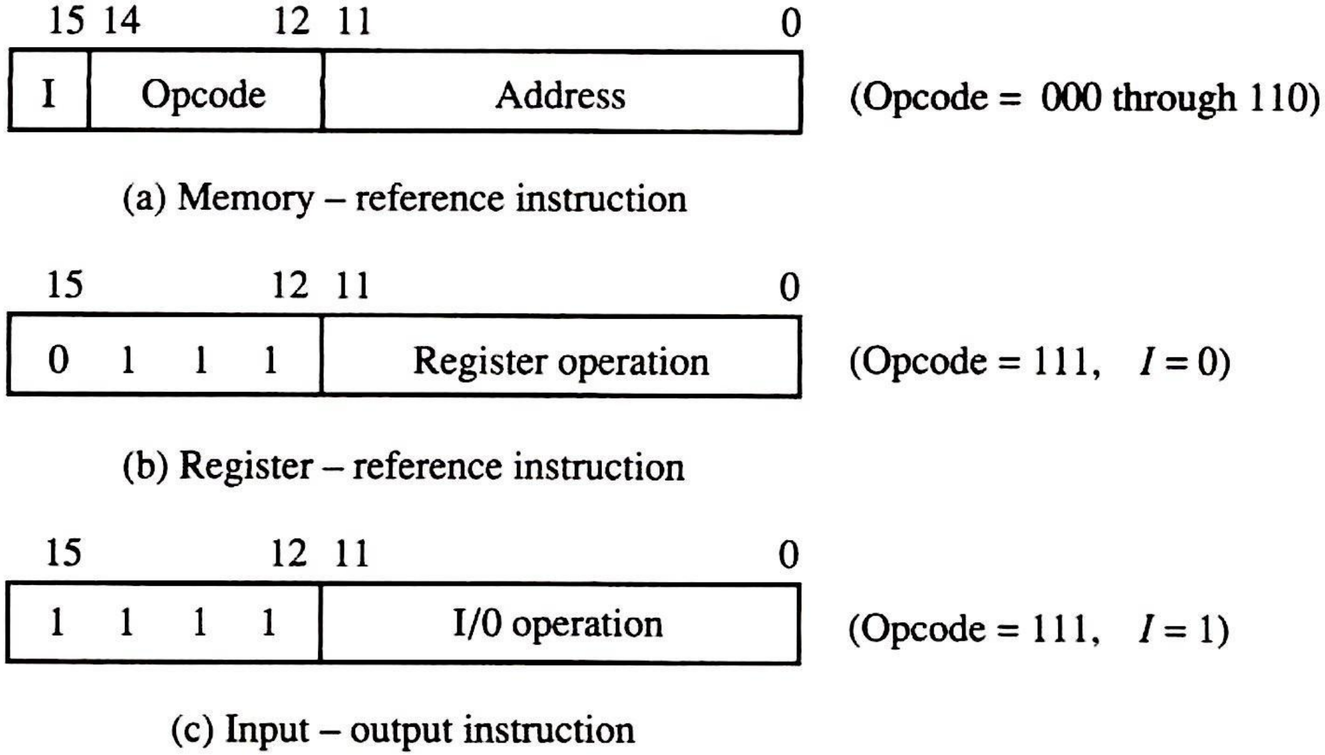
Programming Assembly: ARM

Recognizing Opcodes + Instruction Formats

Writing assembly across instruction sets to see ISAs in action.

Instruction Format

- This image describes the **instruction format** for a **16-bit computer system**, divided into three types:
  - memory-reference instructions,
  - register-reference instructions, and
  - input-output instructions.

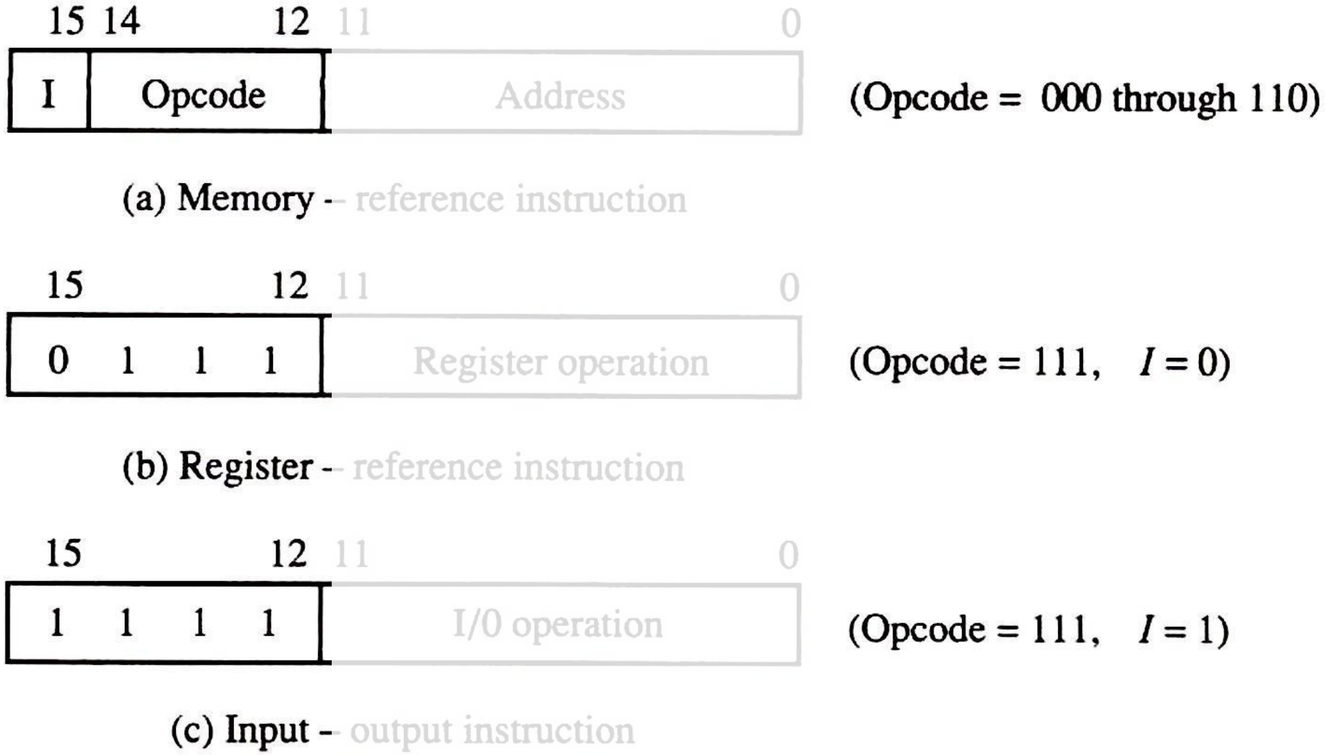


ARM

Instruction Format

Opcodes mean "operation codes".

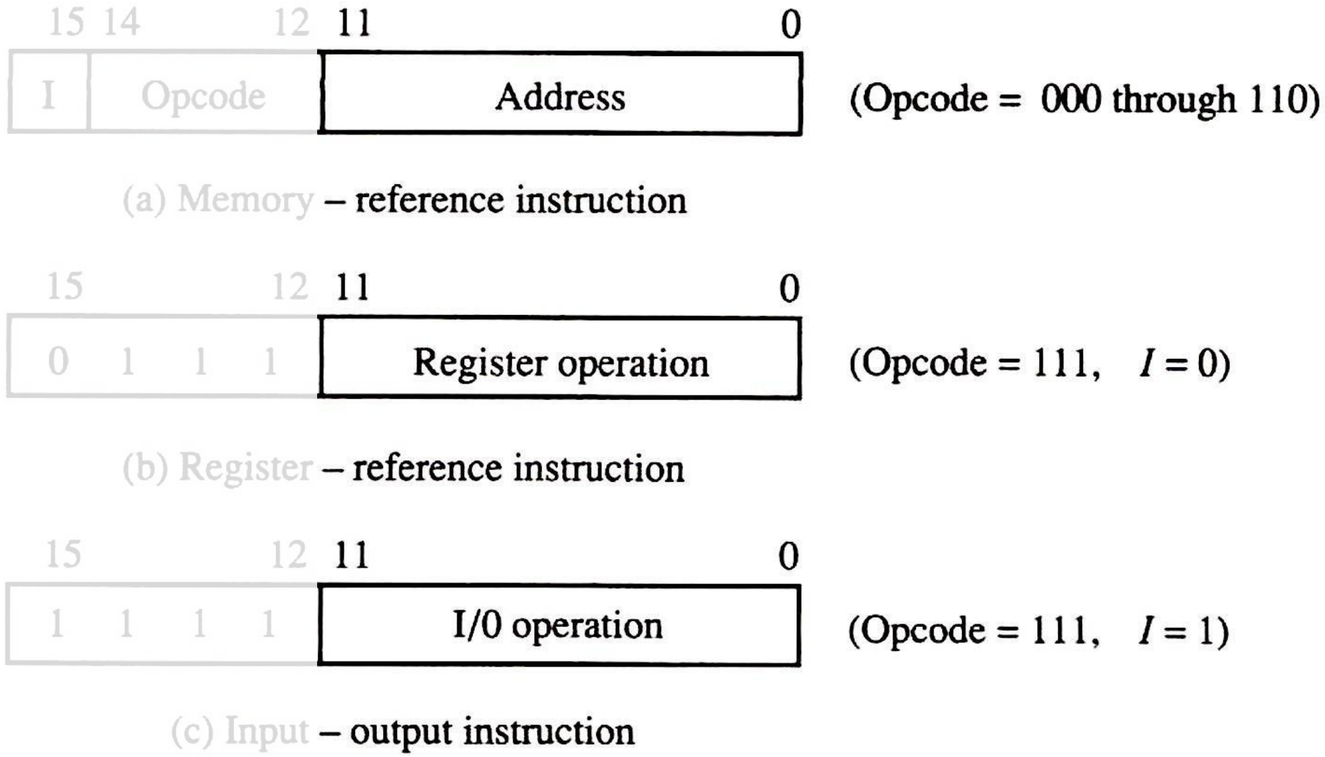
- An **opcode** is the first part of an instruction that tells the computer what function to perform.
- Every computer has an operation code or opcode for each of its functions.
- It is an instruction that tells the processor what to do with the variable or data written beside it.



ARM

Instruction Format

- An operand is the second part of the instruction, which tells the computer where to find or store the data or instructions.
- The number of operands varies among computers.

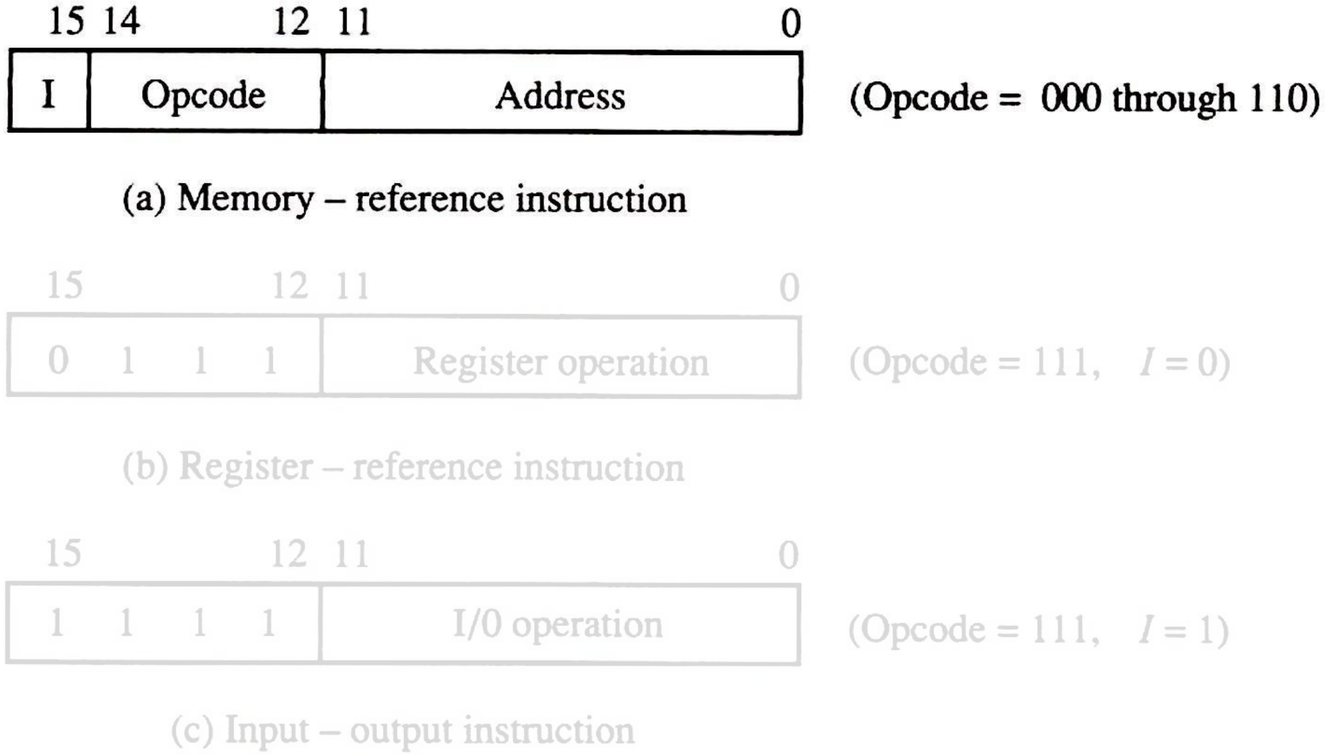


Instruction Format

ARM

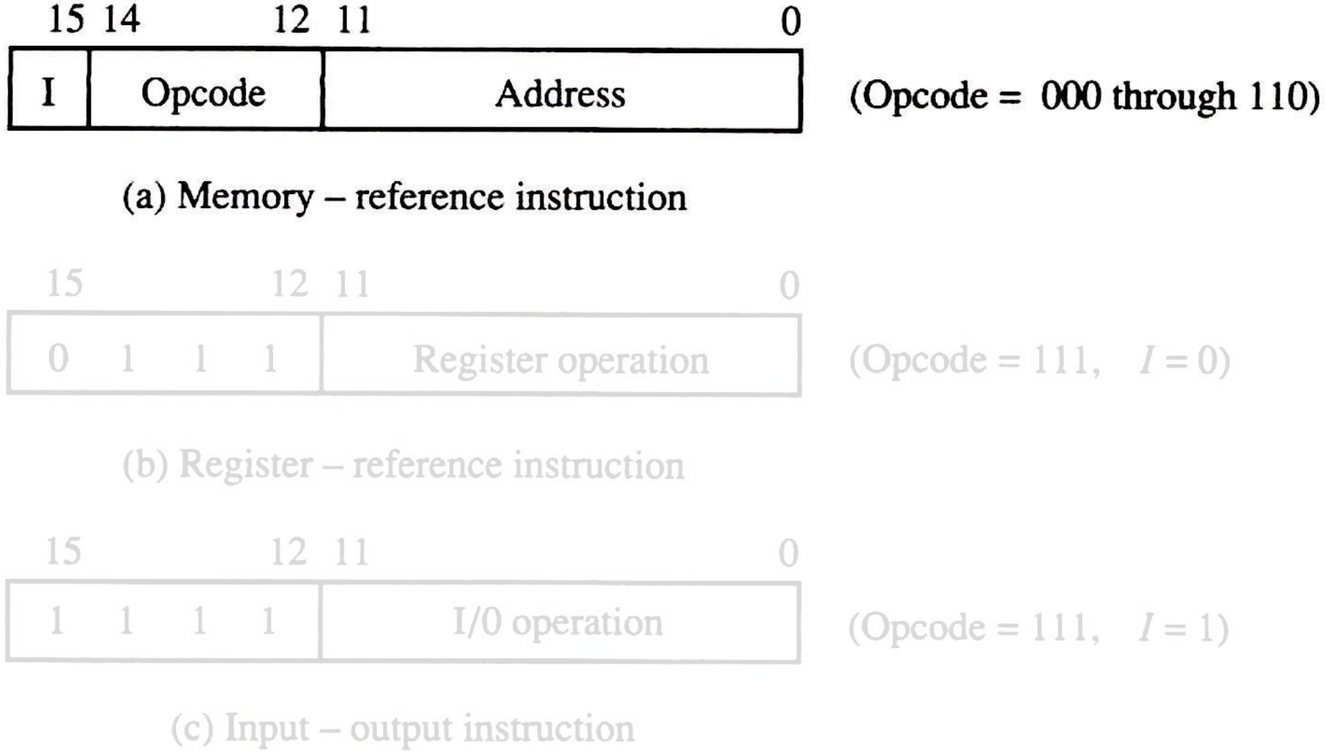
- Each instruction tells the **Control Unit** of the **CPU** what to do and how to do it.

- The operations are **Arithmetic**, **Logical**, **Branch operation**, etc, depending upon the problem that is given to the computers.



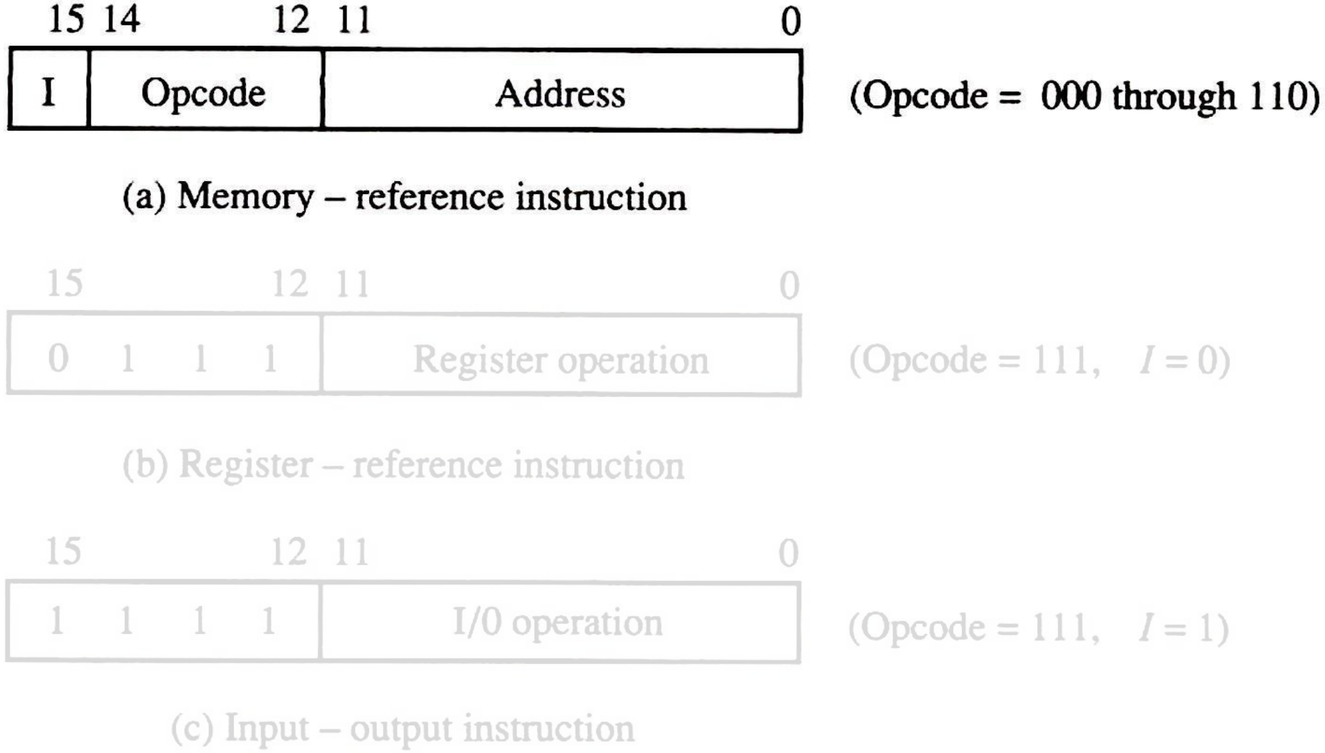
Memory-Reference Instruction

- Bits 15-14:
  - These bits are used for indirect addressing (I) and the **opcode**.
- I (Bit 15):
  - Indicates whether the address is direct (I=0) or indirect (I=1).
- **Opcode** (Bits 14-12):
  - A 3-bit field representing the operation code (values 000 through 110).



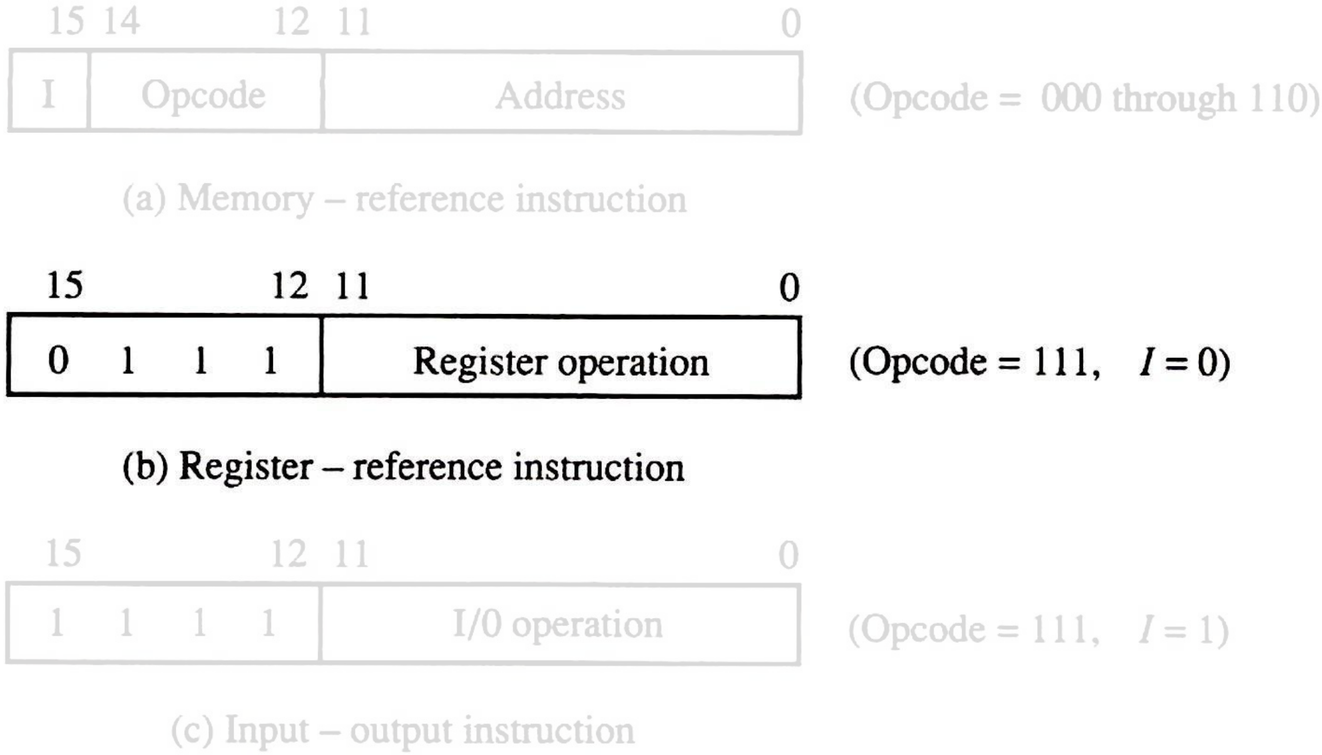
Memory-Reference Instruction

- Bits 11-0: This 12-bit field holds the memory address or operand.
- Example:
  - If the **opcode** is 000 (e.g., **ADD**), the instruction performs an addition using the data at the specified address.



Register-Reference Instruction

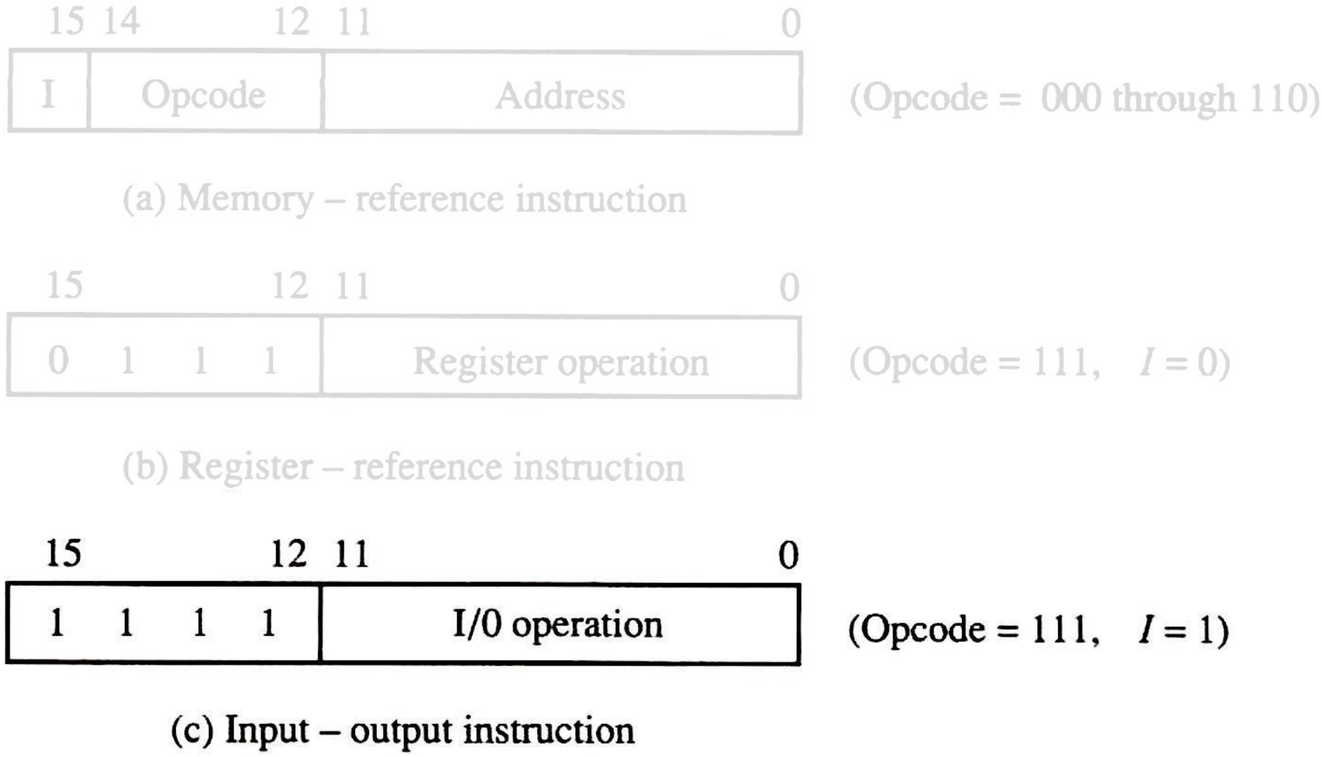
- Bits 15-12: Fixed as 0111 (**opcode** 111 with I=0), indicating this is a register-reference instruction.
- Bits 11-0: These 12 bits specify the operation to be performed on registers, e.g., CLEAR, INCREMENT.
- Each bit or combination of bits in this field corresponds to a specific register operation.
- Example:
  - If bit 11 is 1, it might trigger a “CLEAR ACCUMULATOR” operation.



ARM

Input-Output Instruction

- Bits 15-12: Fixed as 1111 (**opcode** 111 with I=1), indicating this is an input-output instruction.
- Bits 11-0: These 12 bits specify the I/O operation, e.g., READ, WRITE, TEST DEVICE.
- Similar to register-reference, each bit or combination here maps to a specific I/O command.
- Example:
  - If bits 11-8 are 0001, it might command "READ FROM **INPUT** DEVICE."



PART F.1.2

Programming Assembly: ARM

Example Instructions: MOV & ADD Coding Practice

Writing assembly across instruction sets to see ISAs in action.

Acorn RISC Machine (ARM)

- **MOV** Command
  - MOV = Move (really “copy”)
  - It copies a value into a register, without changing the source.
  - It does not move in the “cut-and-paste” sense — the original stays where it was.
    MOV <destination>, <source>

  - Where:
    - <destination> = register to store the result (e.g., R1)
    - <source> = register or immediate value (e.g., R3 or #7)

Acorn RISC Machine (ARM)

- Examples
  - Move an immediate value into a register

    MOV R2, #5 ; R2 = 5

  - Immediate values use #.
  - After execution, R2 contains decimal 5 (0x00000005 in hex).

Acorn RISC Machine (ARM)

- Examples
  - Move value from one register to another

    MOV R2, R1 ; R1 = R2

  - Copy whatever is in R2 into R1.
  - Both registers now hold the same value, but they're independent.

Acorn RISC Machine (ARM)

- Examples
  - Move with flags (**MOV**S)

    MOVS R1, #0 ; R1 = 0 and update status flags

  - MOVS sets the Zero flag (Z=1) in the CPSR because the result is zero.

Acorn RISC Machine (ARM)

- In the **CPU**
  - When we do

    MOVS R2, #5 ; R2 = 5 and update status flags

1. **Instruction Fetch** — CPU fetches E3A02005 from memory.
2. Decode — CPU sees it's a **MOV**, immediate, destination=R2, value=5.
3. Execute — ALU takes the immediate 5, writes it to R2.
4. **Write Back** — Register file stores 5 in

Acorn RISC Machine (ARM)

- Examples
  - Add the values in registers R2 and R3, then store the result in register R1.

    ADD R1, R2, R3 ; R1 = 0 and update status flags

    ADD <destination>, <source1>, <source2>

  - Where:
    - <destination> = register to store the result (e.g., R1)
    - <source1> = register to store the value of the operand 1
    - <source2> = register to store the value of the operand 2

Acorn RISC Machine (ARM)

- Examples
  - Add the values in registers R2 and R3, then store the result in register R1.

```
ADD R1, R2, R3   ; R1 = 0 and update status flags
```

E0 82 10 03 in binary: 1110 0000 1000 0010 0001 0000 0000 0011

In ARM 32-bit assembly, the ADD instruction format is generally:

31-28	27-26	25	24-21	20	19-16	15-12	11-0
cond	00	I	opcode	S	Rn	Rd	Operand2

Acorn RISC Machine (ARM)

- Examples
  - Add the values in registers R2 and R3, then store the result in register R1.

```
ADD R1, R2, R3   ; R1 = 0 and update status flags
```

E0 82 10 03 in binary: 1110 0000 1000 0010 0001 0000 0000 0011

In ARM 32-bit assembly, the ADD instruction format is generally:

31-28	27-26	25	24-21	20	19-16	15-12	11-0
cond	00	I	opcode	S	Rn	Rd	Operand2

- cond (4bits) = 1110 (binary for e hex) → Always
- I (1 bit) = 0 (immediate (I) or register operand)
- **opcode** (4 bits) = 0100 (4 decimal) → **ADD** instruction

Acorn RISC Machine (ARM)

- Examples
  - Add the values in registers R2 and R3, then store the result in register R1.

ADD R1, R2, R3 ; R1 = 0 and update status flags

E0 82 10 03 in binary: 1110 0000 1000 0010 0001 0000 0000 0011

In ARM 32-bit assembly, the ADD instruction format is generally:

31-28	27-26	25	24-21	20	19-16	15-12	11-0
cond	00	I	opcode	S	Rn	Rd	Operand2

- S (1 bit) = 0 (do not update condition flags)
- Rn (4 bits) = first operand register (R2)
- Rd (4 bits) = destination register (R1)
- Operand2 (12 bits) = second operand register (R3) in register form

Acorn RISC Machine (ARM)

- Examples
  - Add the values in registers R2 and R3, then store the result in register R1.

ADD R1, R2, R3 ; R1 = 0 and update status flags

E0 82 10 03 in binary: 1110 0000 1000 0010 0001 0000 0000 0011

In ARM 32-bit assembly, the ADD instruction format is generally:

31-28	27-26	25	24-21	20	19-16	15-12	11-0
cond	00	I	opcode	S	Rn	Rd	Operand2

- Register mapping:
  - Rn = 2 → R2 (first source)
  - Rd = 1 → R1 (destination)
  - Operand2 = 3 → R3 (second source)

EXERCISE

Exercise 2.1: ALU - ARM with Opcode explanation



1. Copy / Rewrite the code from the example screenshot (slides) and find the answers.
2. Write **Assembly code** with **Opcode** explanation for: *[Calculate]*

- **Addition,** when **R2=21** and **R3=12** (add)
- **Subtraction,** when **R2=21** and **R3=12** (sub)
- **Multiplication,** when **R2=21** and **R3=10** (mul)
- **Division,** when **R2=21** and **R3=10** (sdiv) (Integer Division)

https://cpulator.01xz.net/?sys=arm



Key Points: Programming: ISAs with Assembly

- **ARM Assembly**
  - MOV, ADD, SUB instructions
  - Machine code decoding (ARMv7 format)
  - Condition codes (bits 31–28)
- **Instruction Format**
  - Opcode + Operand structure
  - Memory-reference, register-reference, I/O types
- **Architecture Selection**
  - ARM (mobile),
  - MIPS (education/embedded systems)
  - x86 (laptop/ desktop/ server)

PART F.2

Programming Assembly: for MIPS Instruction Set Arch

Writing assembly across instruction sets to see ISAs in action.

MIPS

- MIPS stands for Microprocessor without Interlocked **Pipeline** Stages
- Full name: Microprocessor without Interlocked Pipeline Stages
- Origin: Developed by MIPS Computer Systems in the 1980s.
- Instruction format:
  - Fixed 32-bit instructions,
  - Simple **3-operand** style (**ADD** R1, R2, R3).
- Use in education:
  - Extremely popular in teaching computer architecture because it's **clean** and **easy** to read.
  - Tools like SPIM, MARS simulate it.

MIPS Registers:

- 32 general-purpose registers.

Temporary registers:
- Named **$t0, $t1 — $t9**, etc.

Result value registers:
- Named **$v0, $v1**

Argument registers:
- Named **$a0 – $a3**

… more

#	Name	Notes
0	zero	always contains 0
1	at	reserved for assembler
2	v0	expression evaluation and function results
3	v1	
4	a0	arguments passed to procedures and functions
5	a1	
6	a2	
7	a3	
8	t0	temporary storage – can be overwritten by called routines
…	…	
15	t7	

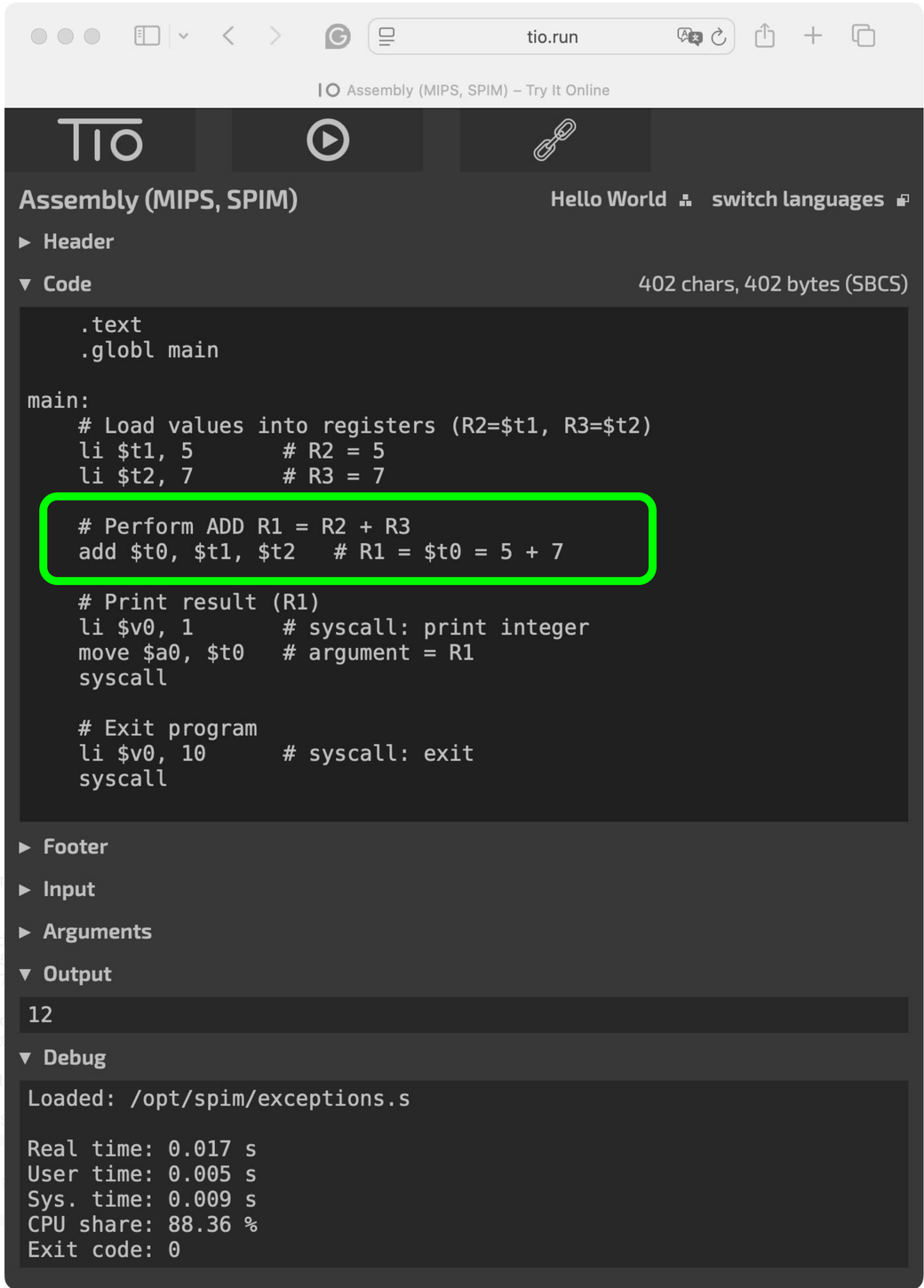
#	Name	Notes
16	s0	variable storage – must be saved by called routines
…	…	
23	s7	
24	t8	more temporary storage
25	t9	
26	k0	reserved for OS use
27	k1	
28	gp	global area pointer
29	sp	stack pointer
30	fp	frame pointer
31	ra	return address register

Source: https://www.cs.rit.edu/~paw/250/notes/135e/,notes.221/02/MIPS%20registers.pdf

We focus on MIPS in this subject, so you will meet all of these registers.

MIPS

- MIPS uses instructions like **ADD** R1, R2, R3 directly.
- This way:
  - $t0 acts as R1
  - $t1 acts as R2
  - $t2 acts as R3
- We can change numbers in li $t1, ... and li $t2, ... to test.
- https://tio.run/# then choose the language "Assembly (MIPS, SPIM)"

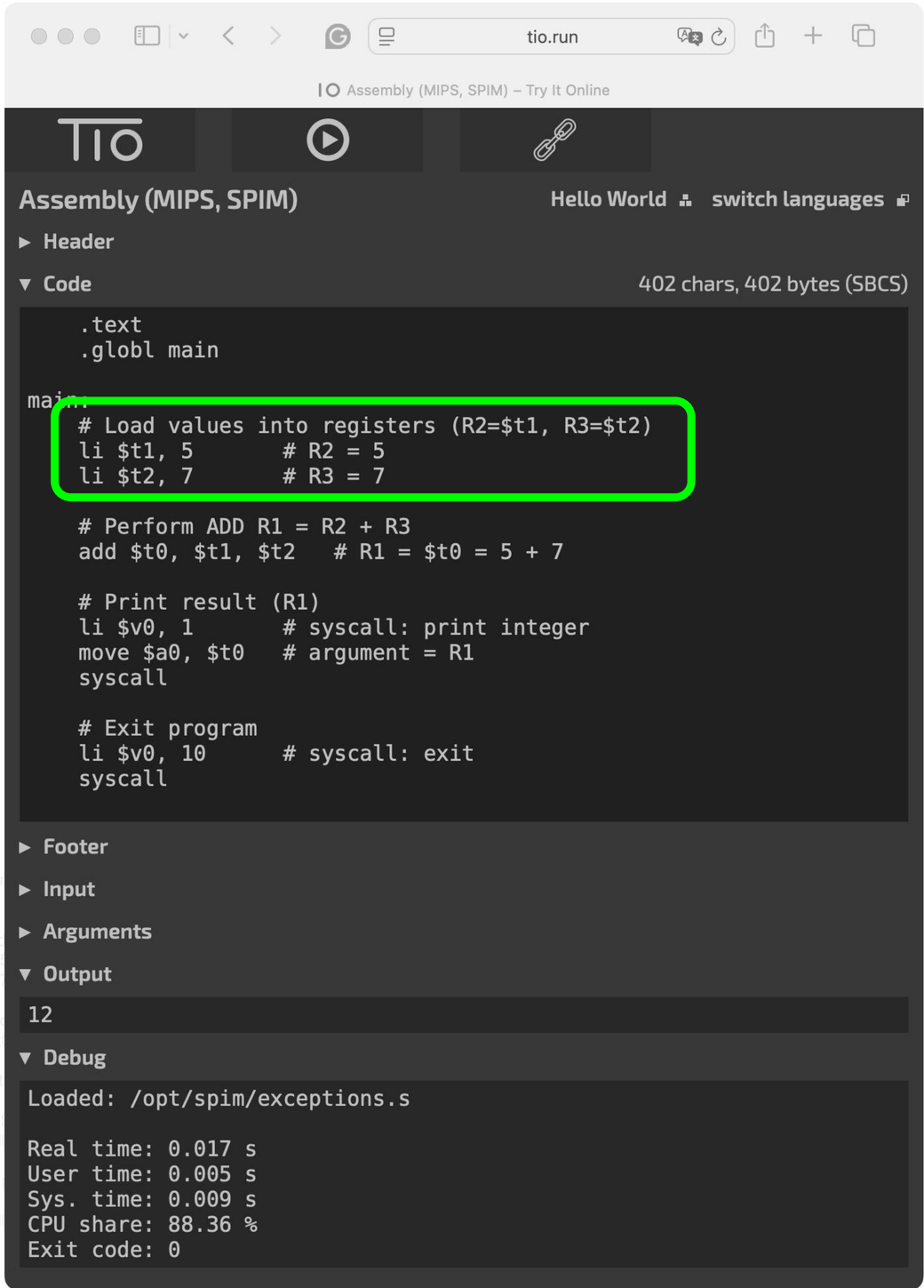


MIPS

- Initialization (Register Loading)

```
li $t1, 5       # R2 = $t1 = 5
li $t2, 7       # R3 = $t2 = 7
```

- li (Load Immediate): Loads a constant value into a register.
- $t1 (temporary register 1) is set to 5.
- $t2 (temporary register 2) is set to 7.
- Note:
  - In the comments, R2 and R3 refer to the logical registers in your original question (mapped to $t1 and $t2 in MIPS).

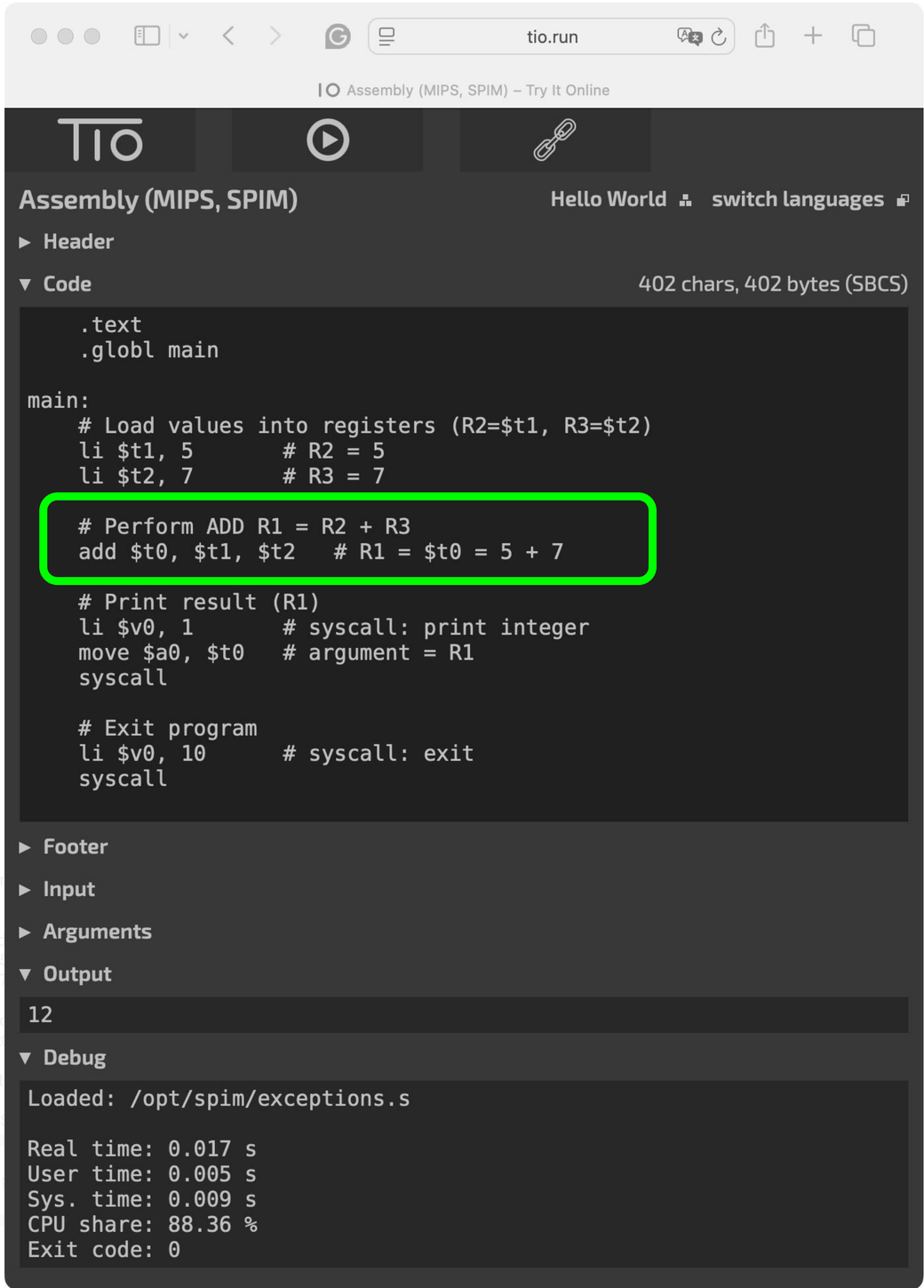


MIPS

- Addition Operation

  add $t0, $t1, $t2 # R1 = $t0 = 5 + 7

- add: Adds the values of $t1 (5) and $t2 (7), storing the result in $t0 (temporary register 0).
- Result: $t0 = 12 (which is referred to as R1 in the comments).

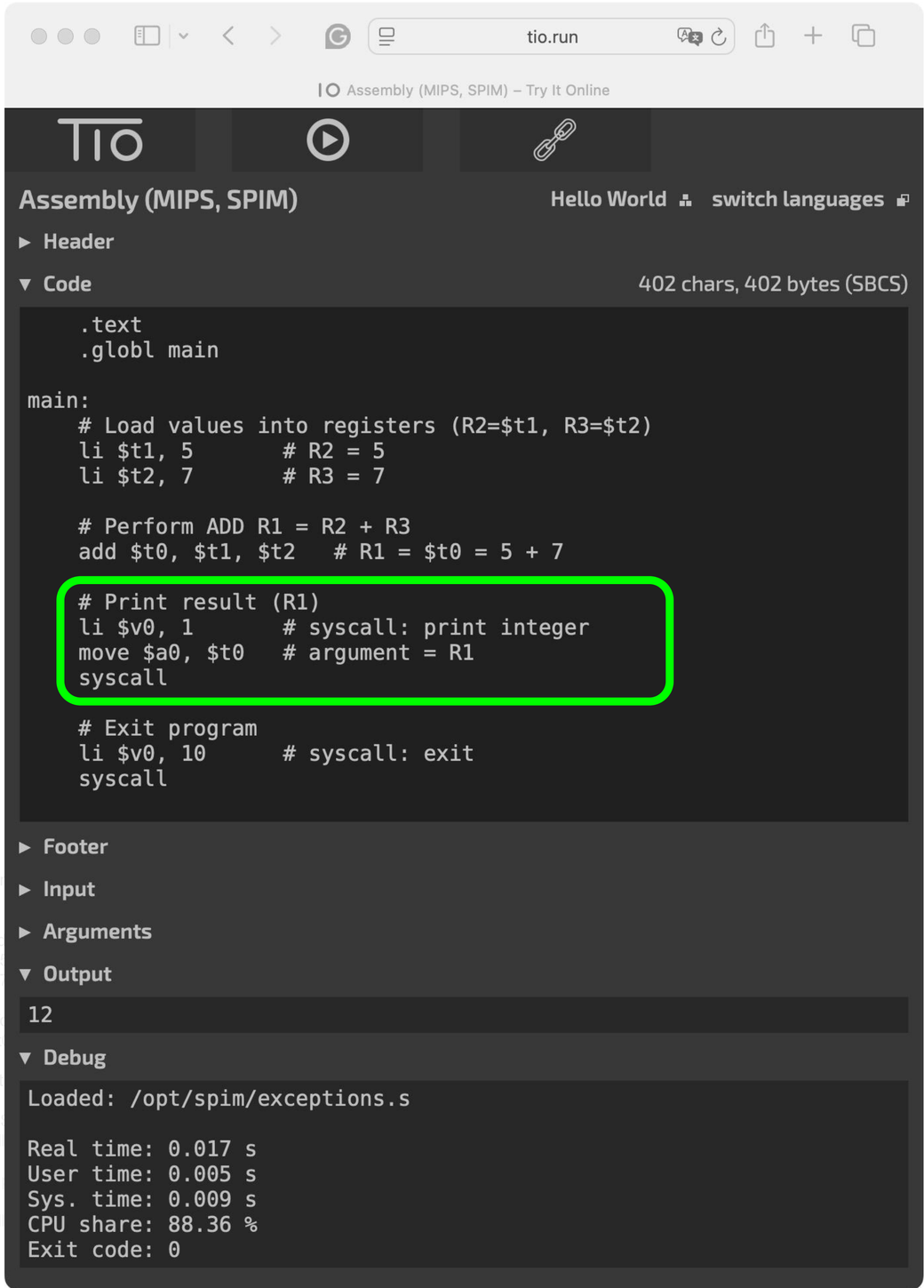


MIPS

- Print the Result

```
li $v0, 1          # Prepare to print an integer
move $a0, $t0      # Set argument = $t0 (R1 = 12)
syscall            # Execute the print
```

- li $v0, 1: Loads the syscall code 1 into $v0 (MIPS uses $v0 to determine the syscall type).
  - 1 = “print integer” service.
- move $a0, $t0: Copies the value of $t0 (12) to $a0 (the register for syscall arguments).
- syscall: Triggers the syscall, printing the integer (12) to the console.

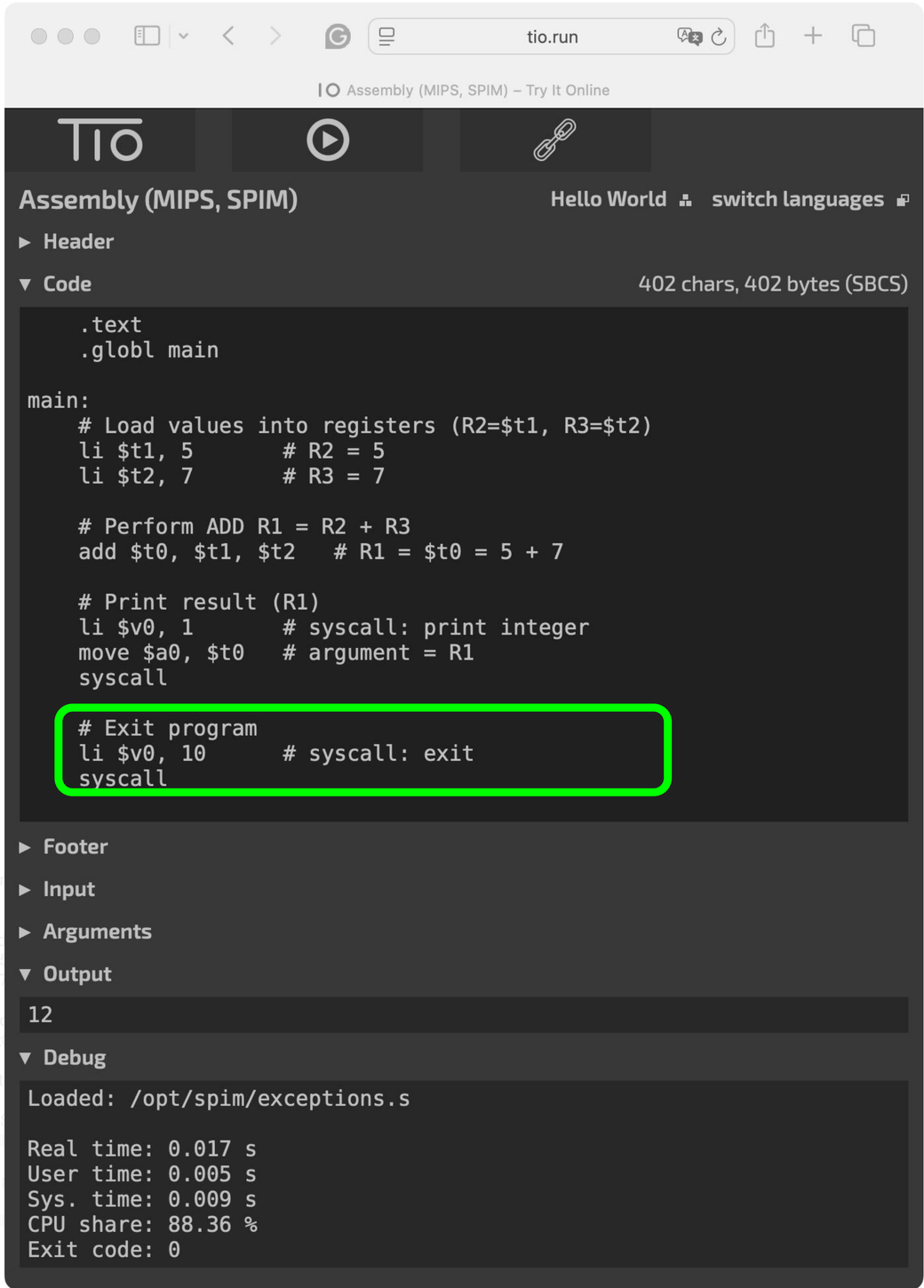


MIPS

- Exit the Program

```
li $v0, 10      # Prepare to exit
syscall         # Terminate the program
```

- li $v0, 10: Loads the syscall code 10 into $v0.
  - 10 = “exit” service.
- syscall: Ends the program execution.



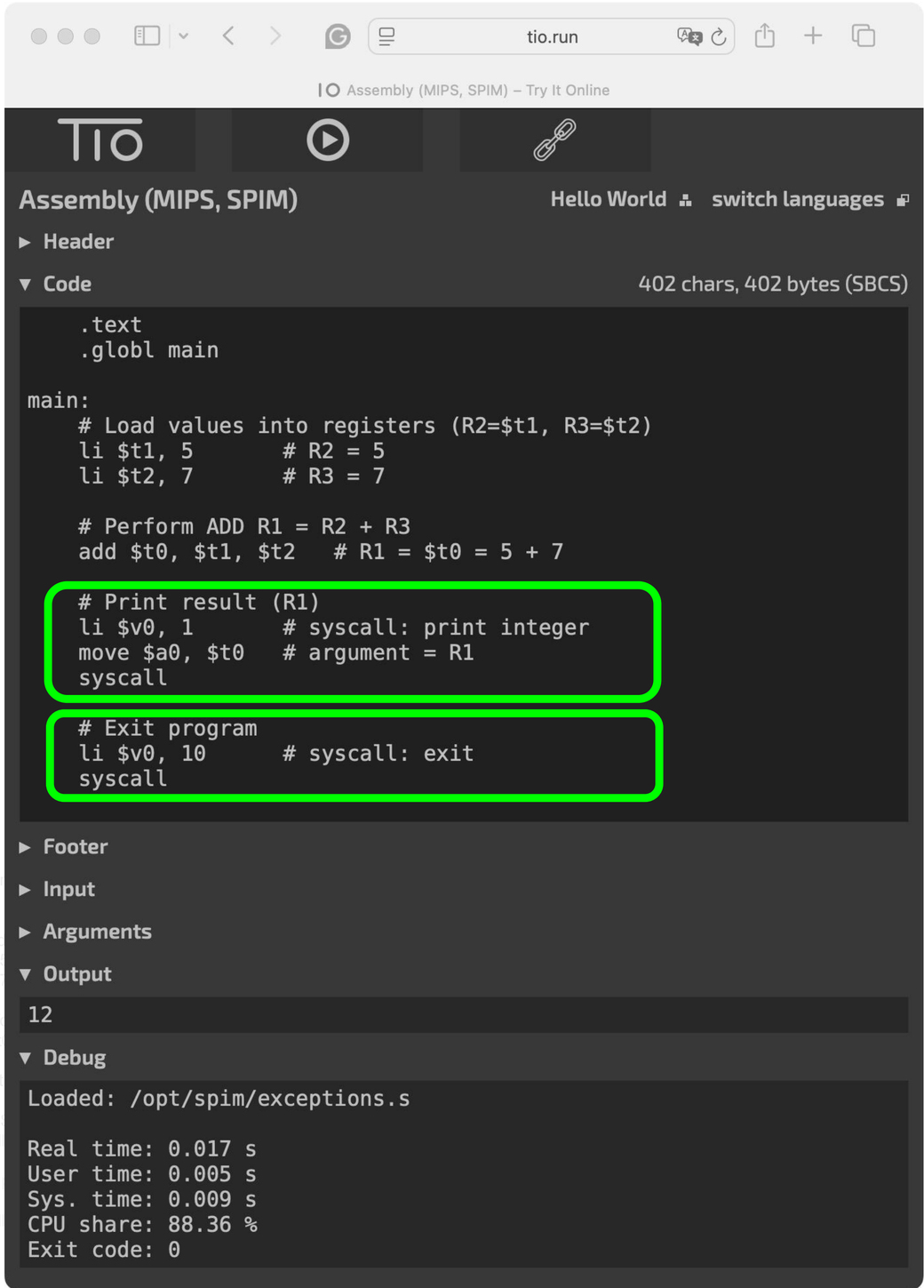
MIPS

MIPS Register Conventions:

- **$t0–$t9**: Temporary registers (caller-saved).
- **$v0**: Holds syscall numbers and return values.
- **$a0–$a3**: Store arguments for syscalls/ functions.

Syscalls:

- MIPS uses **syscall** to interact with the OS, e.g.,
  - Program termination = **10**
  - Print Integer = **1**
  - I/O, = ….



Code to copy/paste

Exercise 2.2: ALU - MIPS

EXERCISE



1. Copy / Rewrite the code from the example screenshot (slides) and find the answers.
2. Write **Assembly code** for: *[Calculate; Print; Exit]*

- **Addition,** when **R2=21** and **R3=12** (add)
- **Subtraction,** when **R2=21** and **R3=12** (sub)
- **Multiplication,** when **R2=21** and **R3=10** (mul)
- **Division,** when **R2=21** and **R3=10** (div) (Integer Division)

https://tio.run/#spim and/or MARS.jar Simulator (Download via MyCourses)



Key Points: Programming: ISAs with Assembly

- **MIPS Assembly**
  - Registers: $t0–$t9, $v0, $a0
  - System calls: print integer, exit
  - Instructions: li, add, syscall

- **Architecture Selection**
  - ARM (mobile),
  - MIPS (education/embedded systems)
  - x86 (laptop/ desktop/ server)

PART F.3

Programming Assembly: for X86 Instruction Set Arch

Writing assembly across instruction sets to see ISAs in action.

x86 Architecture

- x86 architecture refers to a family of instruction set architectures (**ISA**s) initially developed by Intel, based on the 8086 microprocessor.
- It's a **Complex Instruction Set Computer** (**CISC**) design.
- It uses a wide range of instructions that can perform multiple tasks in one operation.
- x86 is the dominant architecture for personal computers and servers.
- The assembly editor below runs on x86.

```
section .data
   result db 0, 10   ; space for one digit + newline

section .text
   global _start

_start:
   mov eax, 5        ; move 5 into eax register
   add eax, 3        ; eax = 8
   add al, '0'       ; convert to ASCII
   mov [result], al  ; store result

   mov eax, 4        ; sys_write
   mov ebx, 1        ; move 1 into ebx register
   mov ecx, result
   mov edx, 2
   int 0x80

   mov eax, 1
   xor ebx, ebx      ; set ebx to 0.
   int 0x80
```

https://www.tutorialspoint.com/compilers/online-assembly-compiler.htm

Registers

x86

- The **CPU registers:**
  - Small, ultra-fast storage locations inside the processor.

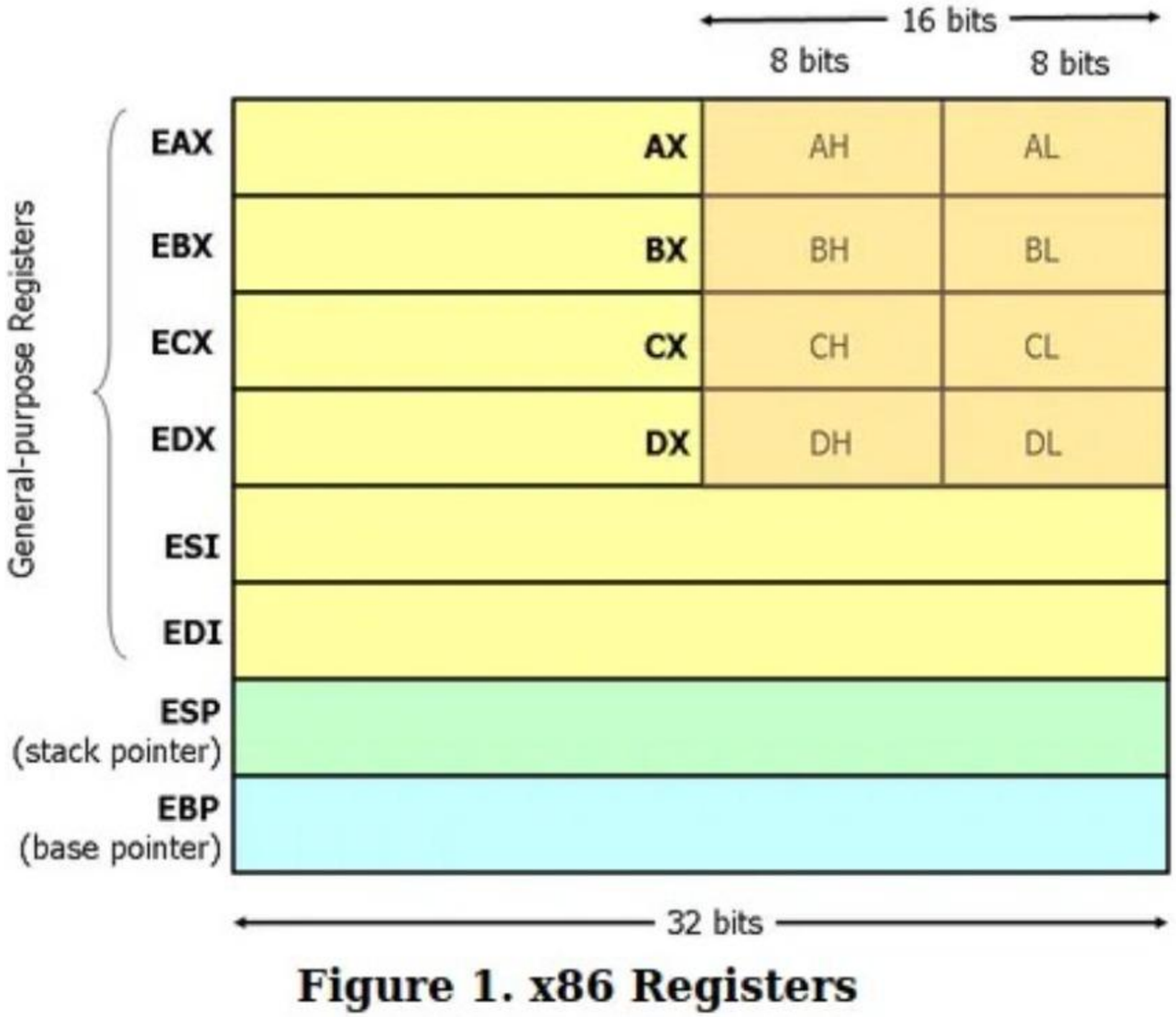


Figure 1. x86 Registers

Registers

x86

- **eax** — “Accumulator Register”
  - Often used to store results of calculations, and in Linux 32-bit syscalls.
  - It holds the system call number, e.g.,
    - 4 for write,
    - 1 for exit.

```
section .data
    result db 0, 10   ; space for one digit + newline

section .text
    global _start

_start:
    mov eax, 5        ; move 5 into eax register
    add eax, 3        ; eax = 8
    add al, '0'       ; convert to ASCII
    mov [result], al  ; store result

    mov eax, 4        ; sys_write
    mov ebx, 1        ; move 1 into ebx register
    mov ecx, result
    mov edx, 2
    int 0x80

    mov eax, 1
    xor ebx, ebx
    int 0x80
```



Registers

x86

- **ebx** — “Base Register”
  - In syscalls, it often holds the first argument to the system call.
  - Example: for writing to the screen, it stores the file descriptor
    - 1 means standard output.

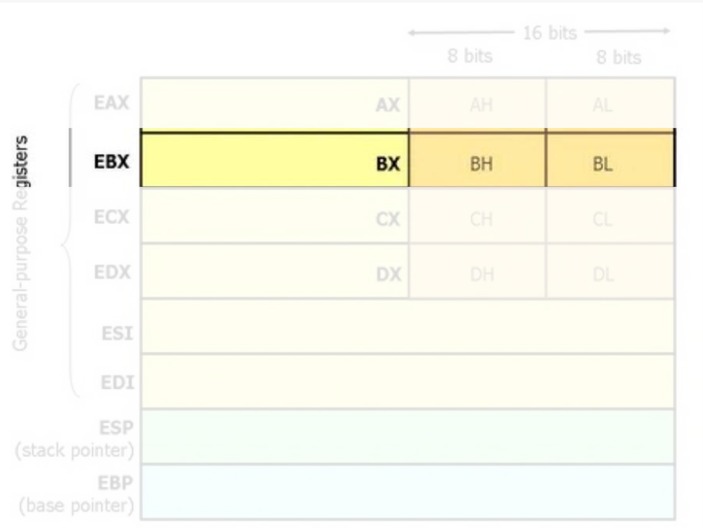
```
section .data
    result db 0, 10   ; space for one digit + newline

section .text
    global _start

_start:
    mov eax, 5        ; move 5 into eax register
    add eax, 3        ; eax = 8
    add al, '0'       ; convert to ASCII
    mov [result], al  ; store result

    mov eax, 4        ; sys_write
    mov ebx, 1        ; move 1 into ebx register
    mov ecx, result
    mov edx, 2
    int 0x80

    mov eax, 1
    xor ebx, ebx
    int 0x80
```



Registers

x86

- **ecx** — “Counter Register”
  - Often used as a counter in loops.
  - For syscalls,
    - it stores the second argument (pointer to the data to write).
- **edx** — “Data Register”
  - Stores the third argument to a syscall
    - For write, that’s the number of bytes to print.

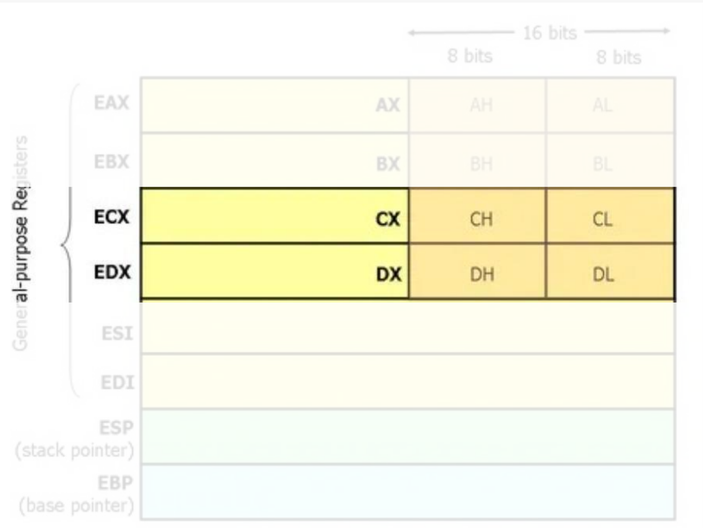
```
section .data
    result db 0, 10   ; space for one digit + newline

section .text
    global _start

_start:
    mov eax, 5        ; move 5 into eax register
    add eax, 3        ; eax = 8
    add al, '0'       ; convert to ASCII
    mov [result], al  ; store result

    mov eax, 4        ; sys_write
    mov ebx, 1        ; move 1 into ebx register
    mov ecx, result
    mov edx, 2
    int 0x80

    mov eax, 1
    xor ebx, ebx
    int 0x80
```



Registers

- All of these names:
  - eax,
  - ebx,
  - ecx,
  - edx

  are for 32-bit registers.

- Their 8-bit low parts are named
  - al, bl, cl, dl.

- Using the al in this code to convert to ASCII.

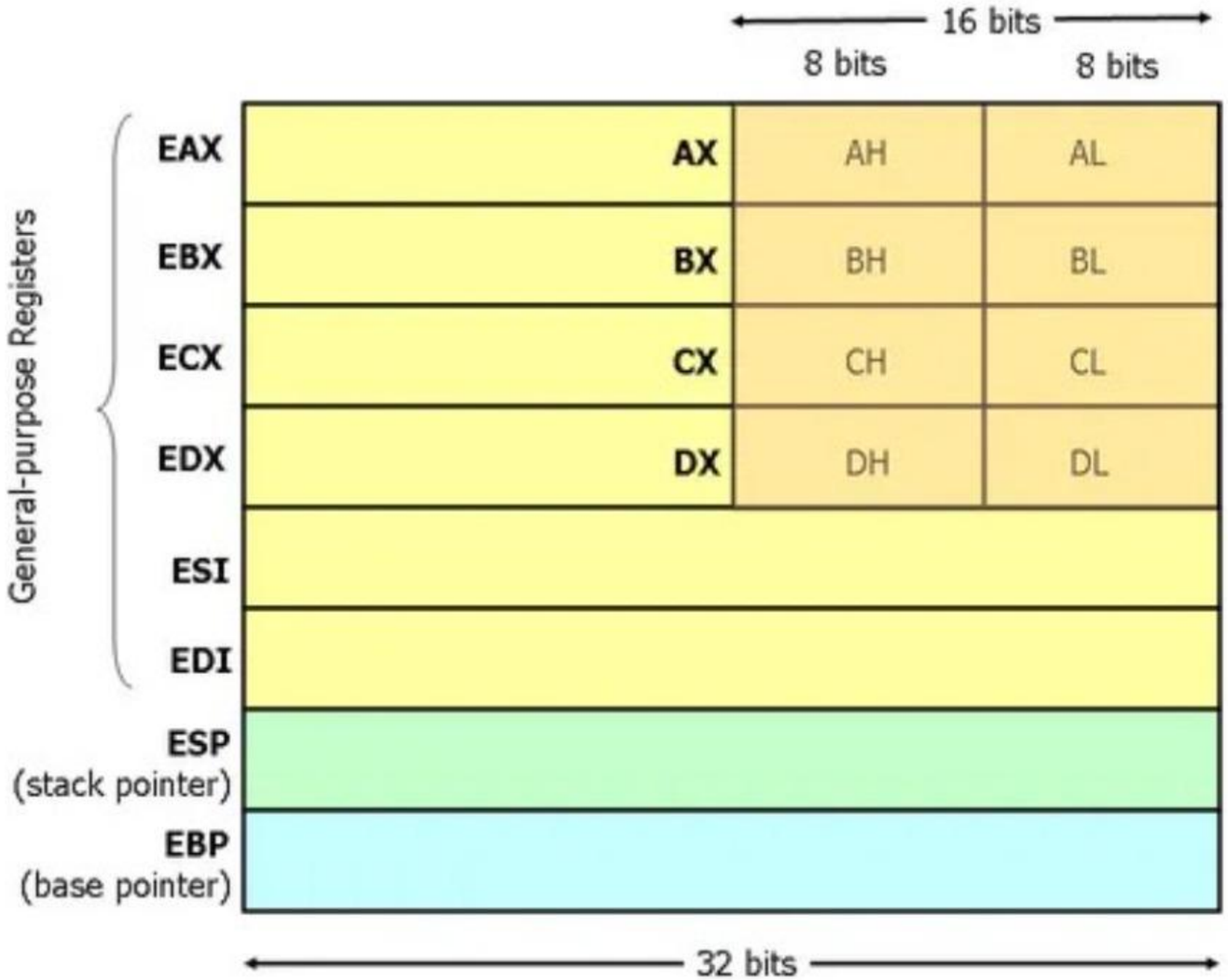


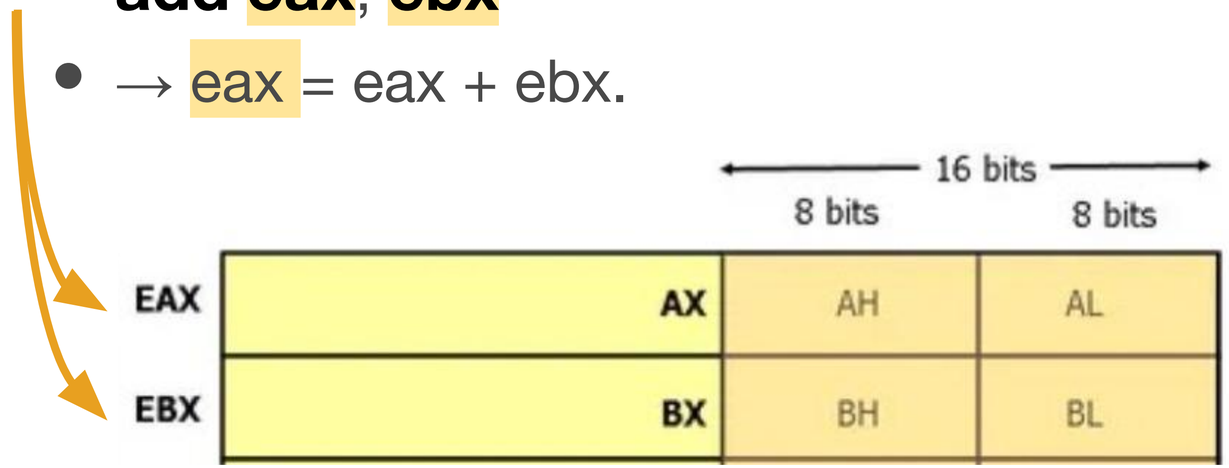
Figure 1. x86 Registers

x86

Instructions

- **add** — Adds values.
- Example:
  - mov eax, 5
  - **add eax**, **3**
  - → eax = 5 + 3.

- **add eax**, **ebx**
- → eax = eax + ebx.




```
section .data
    result db 0, 10   ; space for one digit + newline

section .text
    global _start

_start:
    mov eax, 5          ; move 5 into eax register
    add eax, 3          ; eax = 8
    add al, '0'         ; convert to ASCII
    mov [result], al    ; store result

    mov eax, 4          ; sys_write
    mov ebx, 1          ; move 1 into ebx register
    mov ecx, result
    mov edx, 2
    int 0x80

    mov eax, 1
    xor ebx, ebx        ; set ebx to 0.
    int 0x80
```

x86

Instructions

- **mov** — Moves data from one place to another
  - register ↔ register,
  - register ↔ memory,
  - etc.
- Example:
  - **mov eax**, **5**
  - → puts the number **5** into **eax**.

```
section .data
    result db 0, 10   ; space for one digit + newline

section .text
    global _start

_start:
    mov eax, 5          ; move 5 into eax register
    add eax, 3          ; eax = 8
    add al, '0'         ; convert to ASCII
    mov [result], al    ; store result

    mov eax, 4          ; sys_write
    mov ebx, 1          ; move 1 into ebx register
    mov ecx, result
    mov edx, 2
    int 0x80

    mov eax, 1
    xor ebx, ebx        ; set ebx to 0.
    int 0x80
```

x86

Instructions

- **xor** — Bitwise XOR.
  - A common trick:
    - **xor** ebx, ebx
    - sets ebx to zero without using an immediate **mov** 0.

```
section .data
    result db 0, 10   ; space for one digit + newline

section .text
    global _start

_start:
    mov eax, 5        ; move 5 into eax register
    add eax, 3        ; eax = 8
    add al, '0'       ; convert to ASCII
    mov [result], al  ; store result

    mov eax, 4        ; sys_write
    mov ebx, 1        ; move 1 into ebx register
    mov ecx, result
    mov edx, 2
    int 0x80

    mov eax, 1
    xor ebx, ebx      ; set ebx to 0.
    int 0x80
```

Exercise 2.3: ALU - x86

EXERCISE



1. Copy / Rewrite the code from the example screenshot (slides) and find the answers.
2. Write **Assembly code** for: *[Calculate; Print; Exit]*

- **Addition,** when **R2=21** and **R3=12** (add)
- **Subtraction,** when **R2=21** and **R3=12** (sub)
- **Multiplication,** when **R2=21** and **R3=10** (imul)
- **Division,** when **R2=21** and **R3=10** (idiv) (Integer Division)

https://www.tutorialspoint.com/compilers/online-assembly-compiler.htm



Key Points: Programming: ISAs with Assembly

- **x86 Assembly**
  - CISC architecture (Intel 8086 family)
  - Registers: eax, ebx, ecx, edx
  - Instructions: mov, add, xor, int
- **Architecture Selection**
  - ARM (mobile),
  - MIPS (education/embedded systems)
  - x86 (laptop/ desktop/ server)

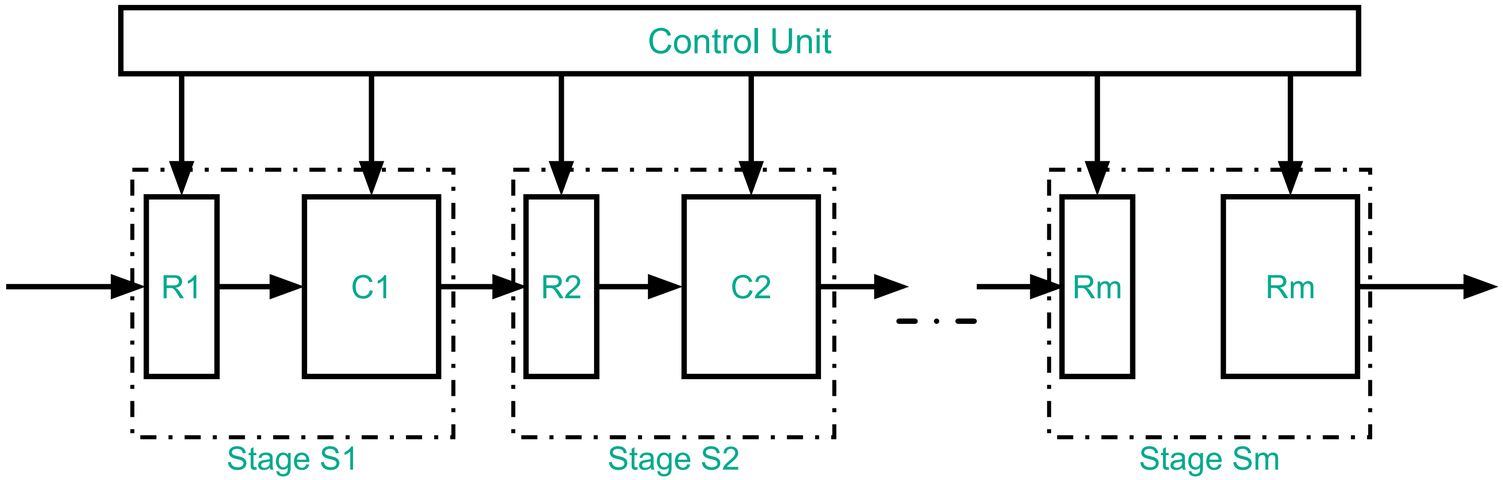
PART G

Pipelined Processors

How pipelines **overlap** **instruction execution** to boost throughput.

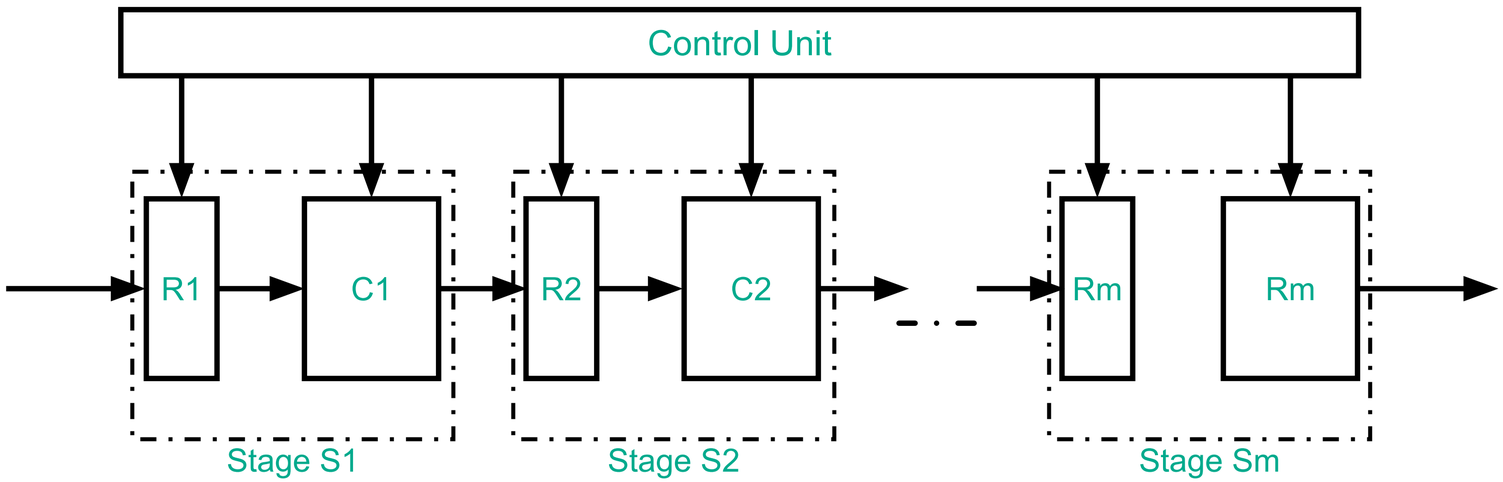
Pipelining or pipeline processing

- Pipelining is the process of collecting instructions from a processor using a pipeline.
- Pipelining allows us to save and execute instructions through a systematic process.
- Pipelining is also called pipeline processing.



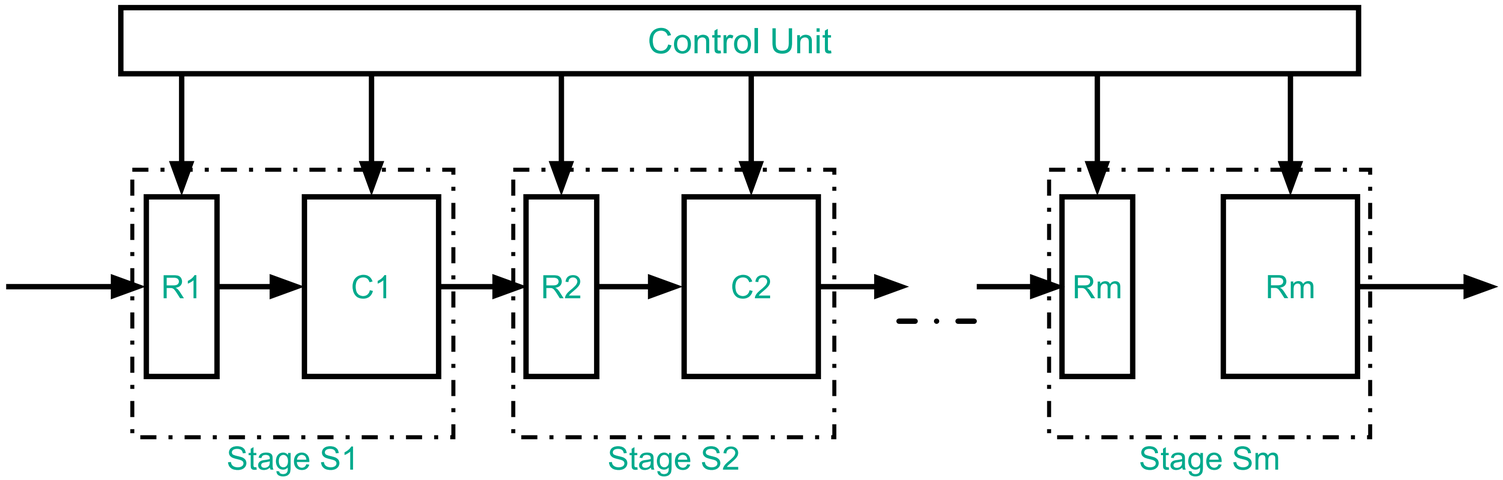
Pipelining or pipeline processing

- Commands come in and out of another.
- Instructions come in from one end of the pipe and out of another end.
- It increases the total instruction throughput.



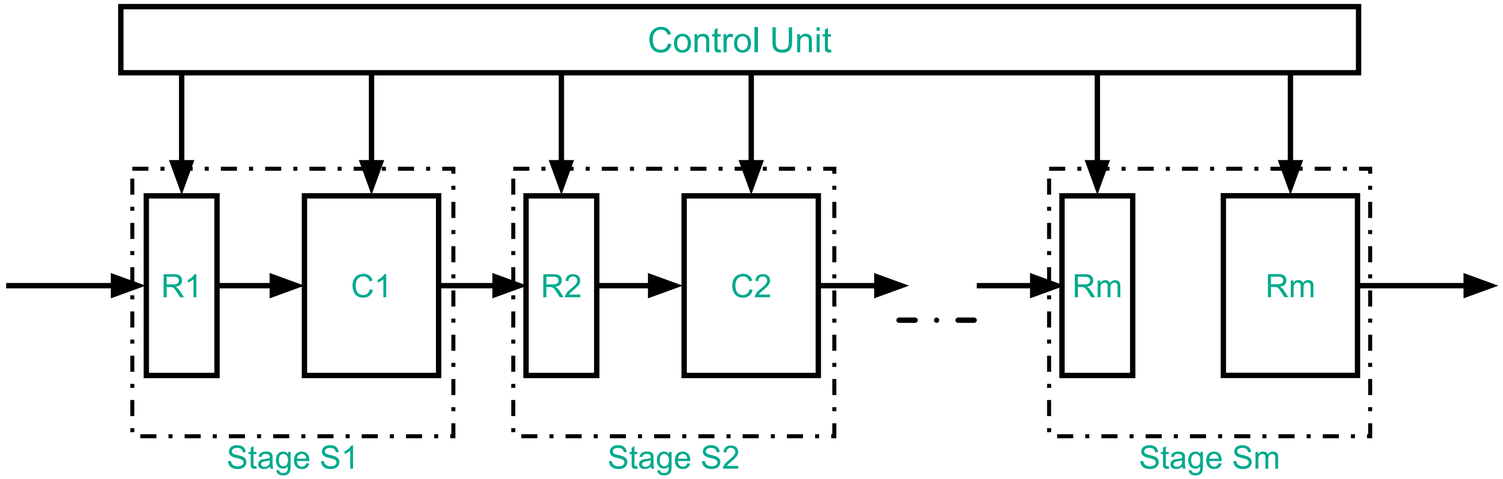
Pipelining or pipeline processing

- While one instruction is being executed, others are moving through the earlier stages, allowing multiple instructions to be processed at the same time.
- This is similar to an assembly line in a factory, where different workers (stages) handle different tasks on multiple products (instructions) simultaneously.



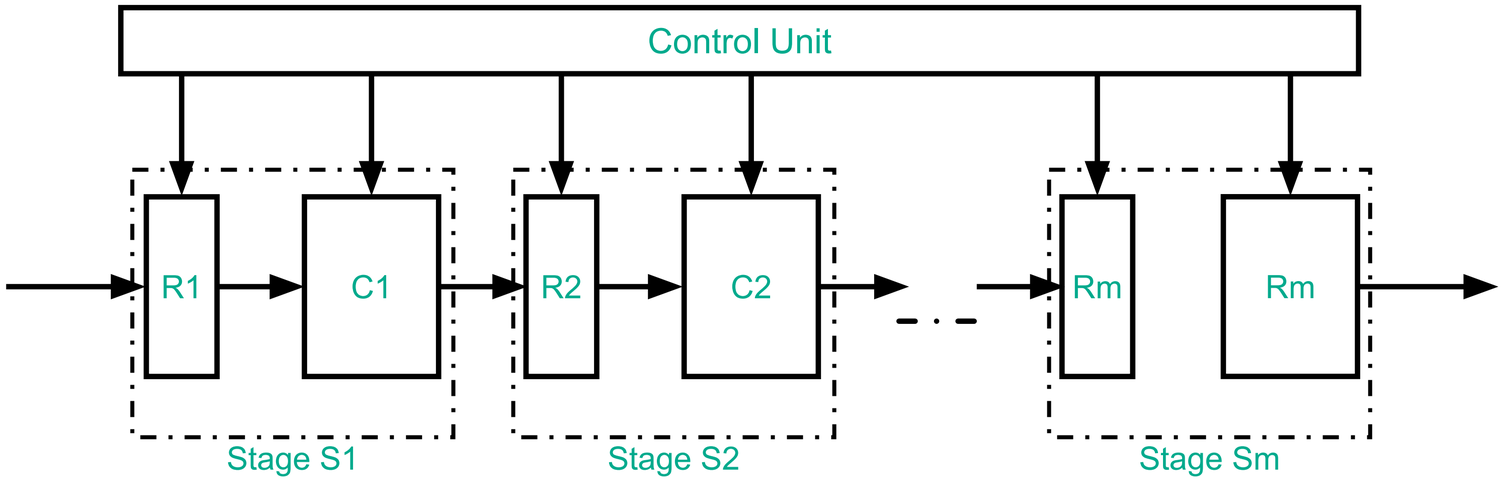
Pipeline Processor

- An assembly line in a factory, for example, consider how cars are built in a factory:
  - One worker installs the engine.
  - The next one adds the wheels.
  - Another paints the car.
  - The last one performs final checks.



Pipeline Processor

- This overlap increases the overall efficiency and throughput of the processor.
- In clock cycle 1, data enters Stage S1 (R1 → C1).
- In clock cycle 2, the data moves to Stage S2 (R2 → C2), while new data enters Stage S1.



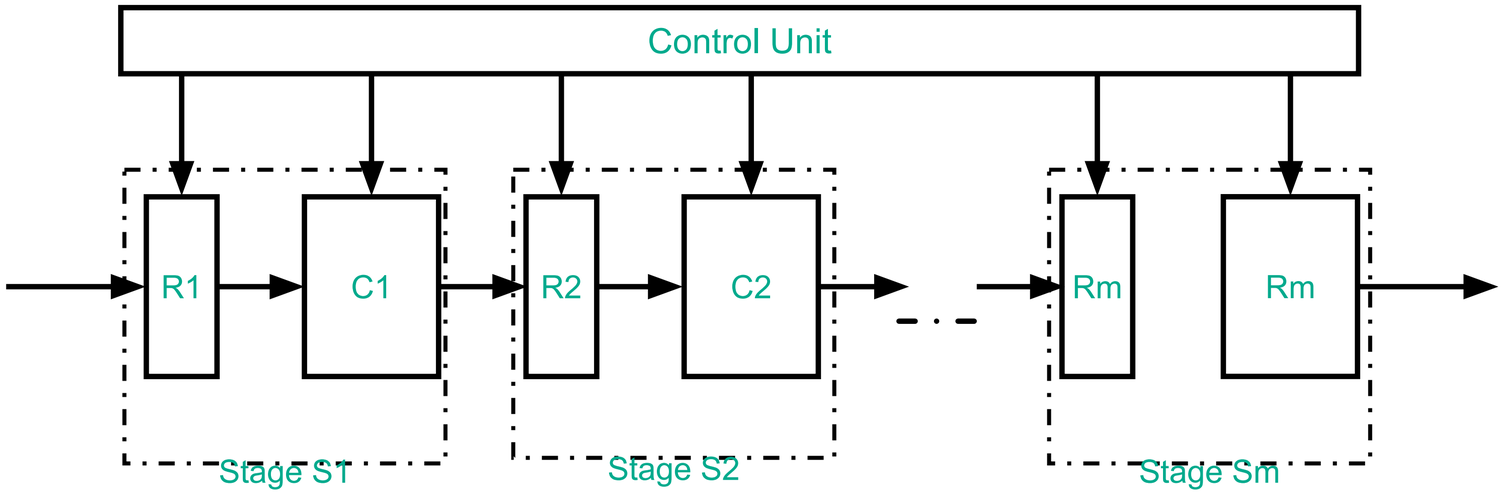
Pipeline Processor

- This process continues, and each stage is simultaneously processing a different piece of data.
- Eventually, the output appears at the end after passing through all stages.

Clock Cycle	Stage S1 (R1→C1)	Stage S2 (R2→C2)	Stage S3 (R3→C3)	...	Stage Sm (Rm→Cm)
1	Data A	-	-	...	...
2	Data B	Data A	-	...	...
3	Data C	Data B	Data A	...	...
4	Data D	Data C	Data B	...	...
5	Data E	Data D	Data C	...	...
...	...	...	...	...	...

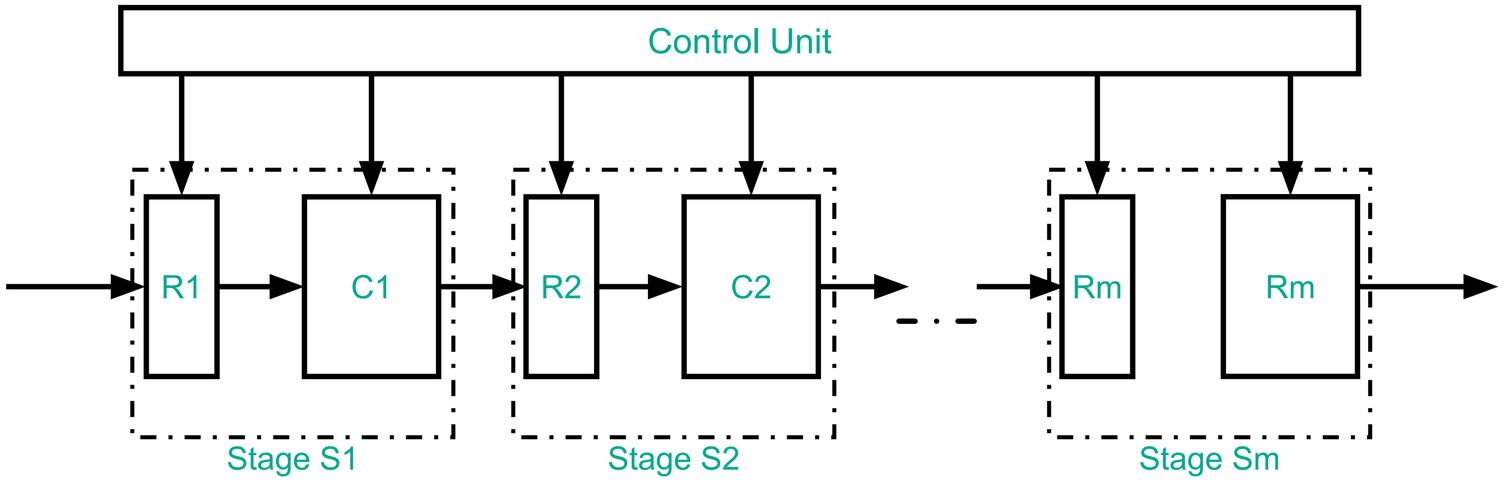
Pipelining

- In the pipeline system, each segment contains an input register followed by a combinational circuit.
- The register holds data, and the combinational circuit performs functions on it.
- The output of the combinational circuit is used in the input register of the next segment.



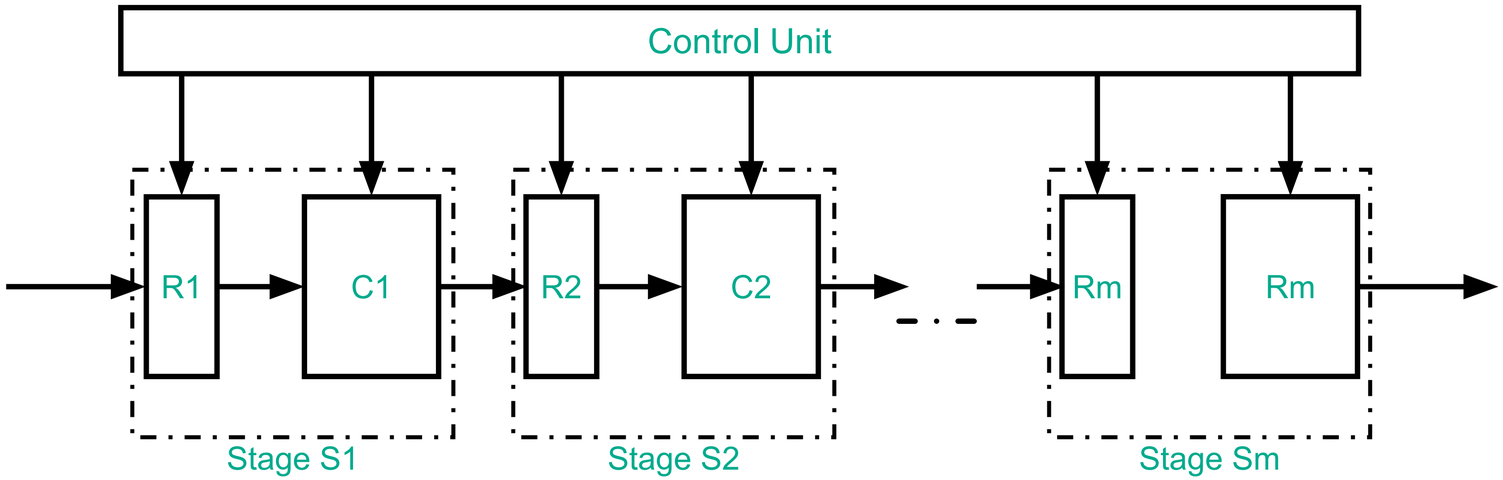
Components in the Diagram

- Data In: This is the input data that enters the pipeline.
- Stages (S1, S2, ..., Sm): Each stage performs part of the operation.
  - The pipeline is divided into multiple stages (S1, S2, ..., Sm), and each stage handles a specific operation.
- Data Out: The final output after processing is complete across all pipeline stages.



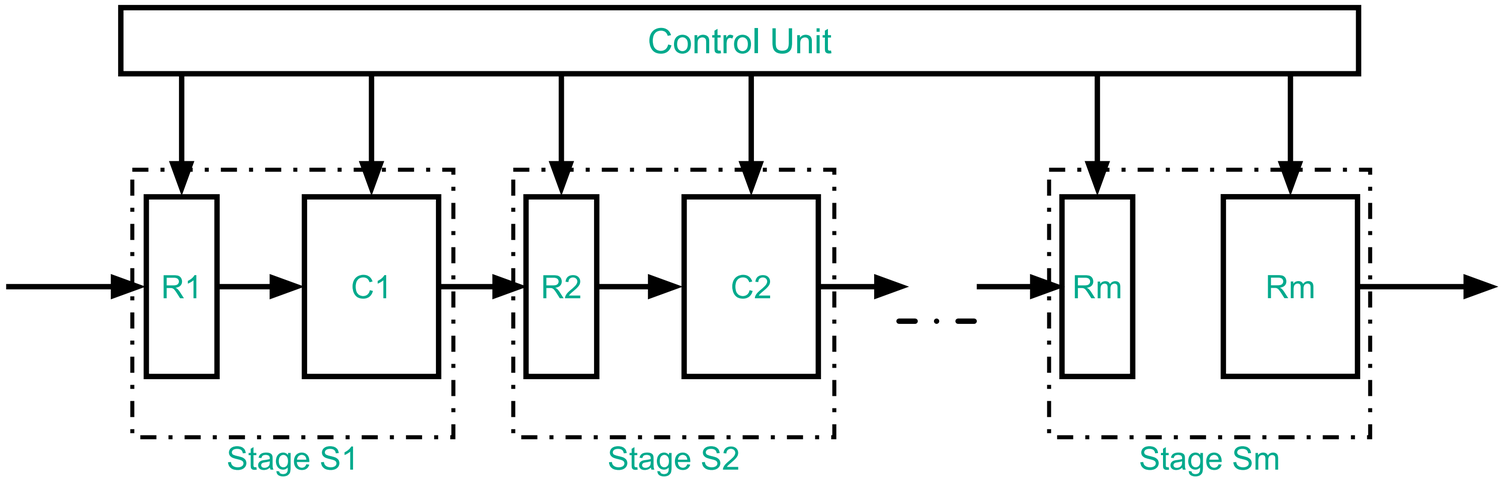
Components in the Diagram

- Registers (R1, R2, ..., Rm): These are pipeline registers.
  - They temporarily hold data between stages to ensure smooth transfer and isolation between operations.
  - Example: R1 stores the output of Stage S1 before passing it to S2.



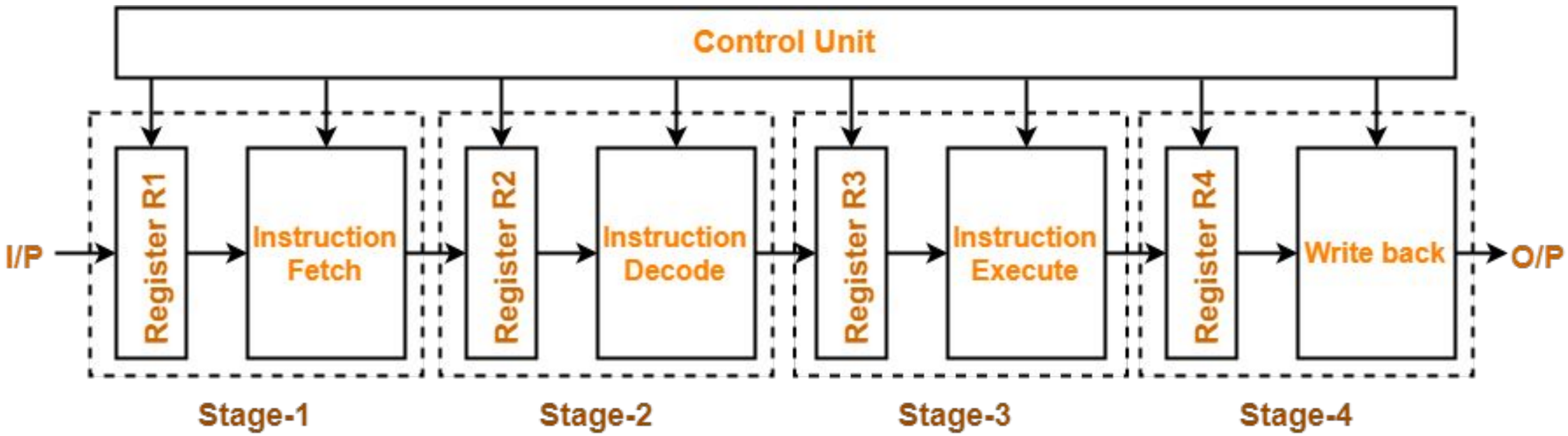
Components in the Diagram

- **Computation Units** (C1, C2, ..., Cm): These perform the actual processing (like arithmetic or logical operations by ALU).
  - Each computation unit corresponds to a specific stage.
- **Control Unit**: Manages the timing and control signals for each stage.
  - Ensures that each stage operates in sync and processes the correct data at the right time.



Pipeline Processor

- A pipeline processor is a type of **CPU** architecture that improves processing speed by dividing instruction execution into separate stages.
- Each stage handles a specific part of the instruction, such as fetching, decoding, executing, memory access, and writing results.

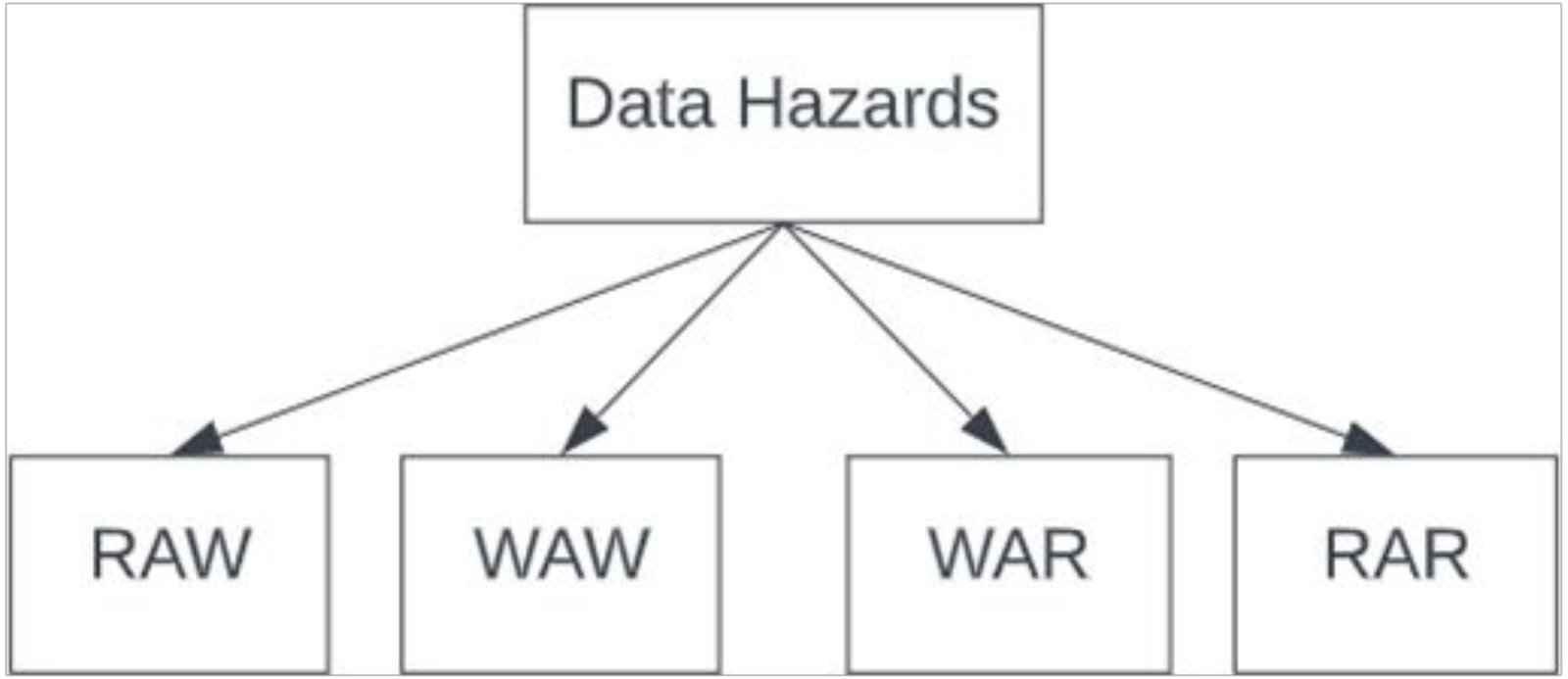


Four Stage Instruction Pipeline

source: https://www.gatevidyalay.com/pipelining-in-computer-architecture/

Pipeline Hazards

- Pipeline instruction execution comes with potential **conflicts** and **breaking cases**.
  - These are called **Data or Pipeline Hazards**.
  - We will see **four** types.



source: https://medium.com/@shrutika.gade22/pipeline-hazards-f6c317824b9f

More in future classes.



Key Points: Pipelined Processors

- **Pipelining Concept**
  - Assembly line analogy
  - Multiple instructions in different stages
- **Pipeline Structure**
  - Stages (S1, S2, ..., Sm)
  - Registers (R1, R2, ..., Rm)
  - Computation Units (C1, C2, ..., Cm)
- **Benefits**
  - Increased throughput
  - Improved efficiency through overlap
- **Pipeline Hazards**
  - Data, control, and structural hazards
- **Control Unit**
  - Manages timing and control signals
  - Coordinates between stages

References

References

Reference Materials

- William Stallings, “Computer Organization and Architecture Designing for Performance,” Tenth Edition, *Pearson*, 2016
- Tools:
  - https://cpulator.01xz.net/?sys=arm
  - https://tio.run/#
  - https://www.tutorialspoint.com/compile_assembly_online.php

Mahidol University

Logic Gates

x \ y	∧ 0	∧ 1
0	0	0
1	0	1

x \ y	∨ 0	∨ 1
0	0	1
1	1	1

x \ y	→ 0	→ 1
0	1	1
1	0	1

x \ y	⊕ 0	⊕ 1
0	0	1
1	1	0

Figure 1. Truth tables

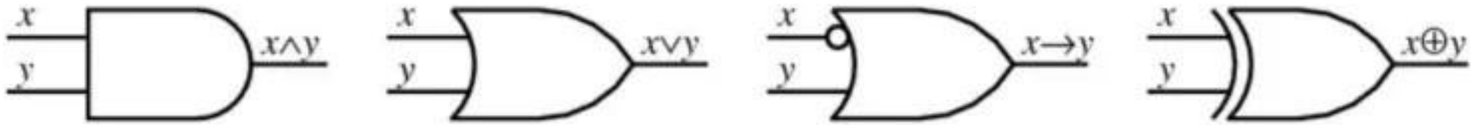


Figure 2. Logic gates

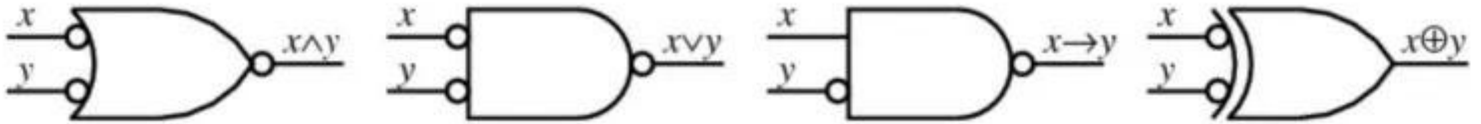


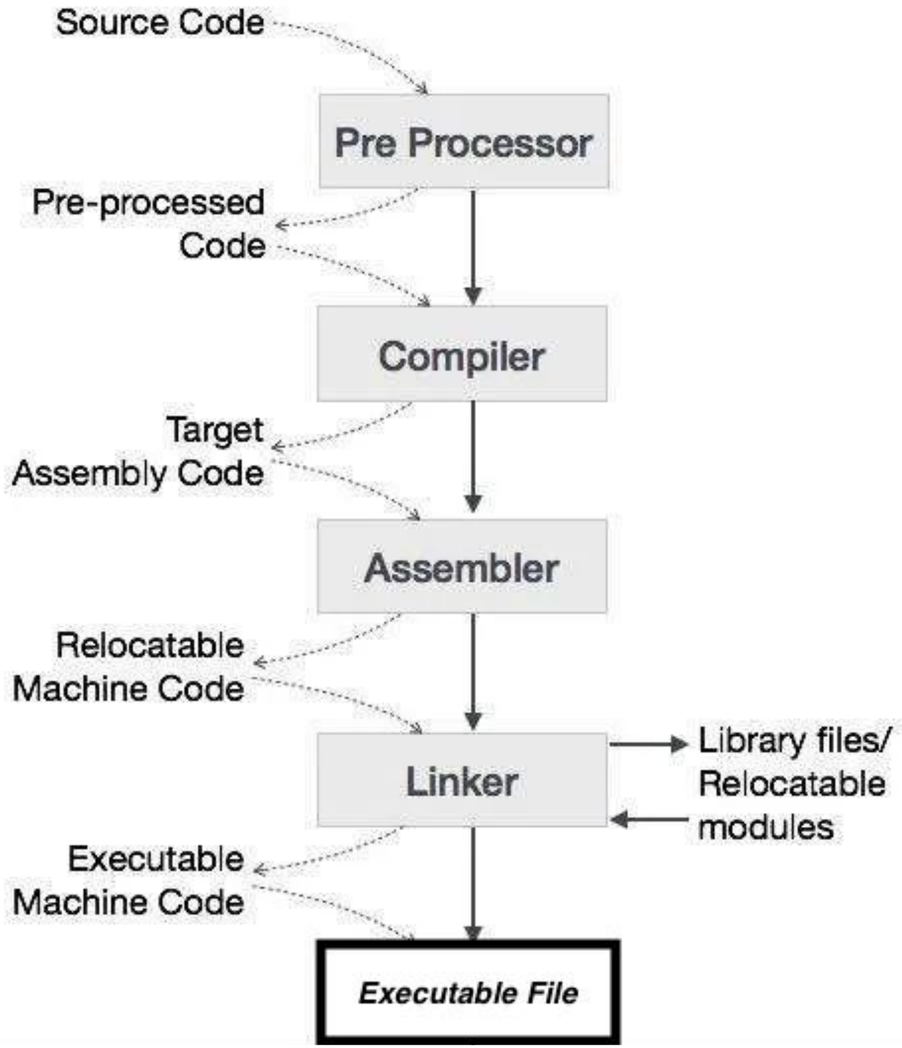
Figure 3. De Morgan equivalents



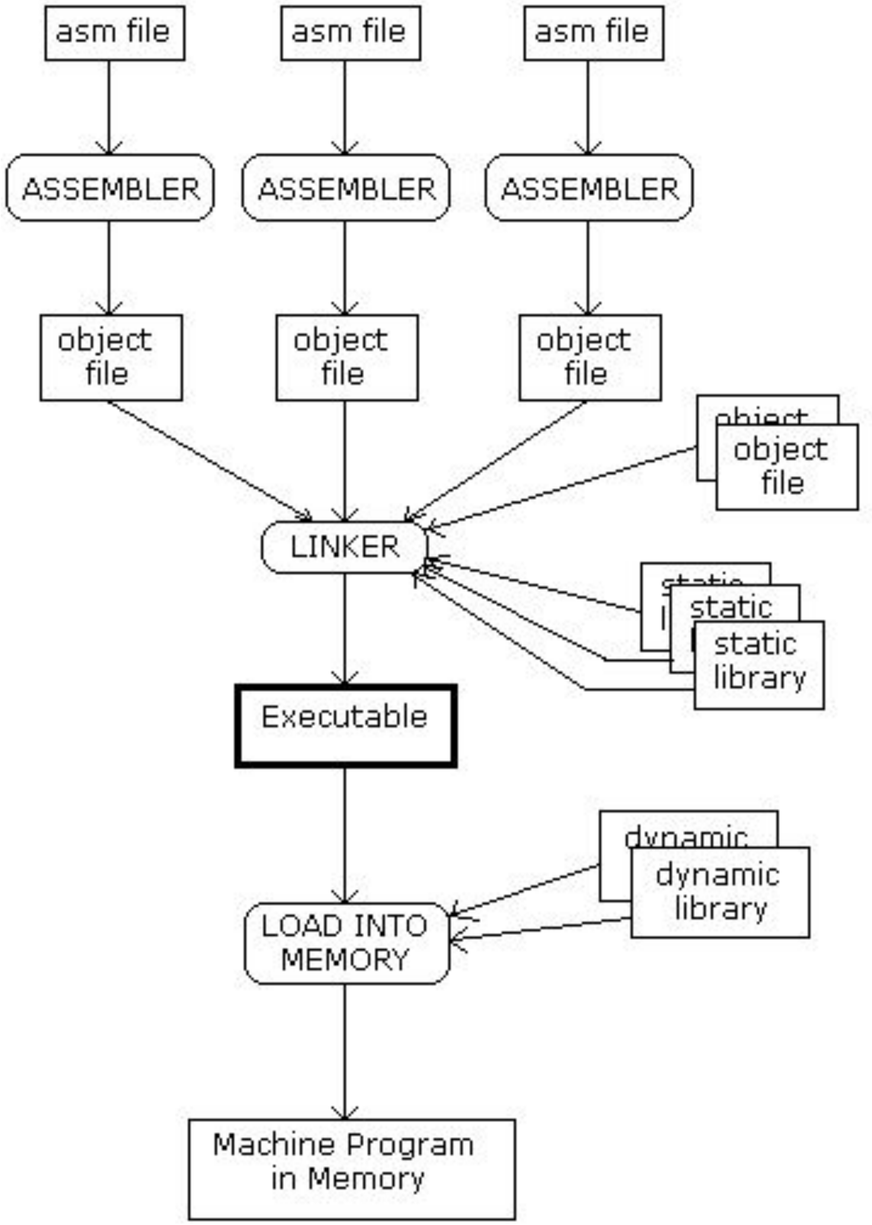
Figure 4. Venn diagrams

source: https://pablocorbalann.medium.com/programming-a-hello-world-in-assembly-from-the-first-line-to-the-end-**x86**-9c48fb499238

Assembly Execution



source: https://pablocorbalann.medium.com/programming-a-hello-world-in-assembly-from-the-first-line-to-the-end-**x86**-9c48fb499238



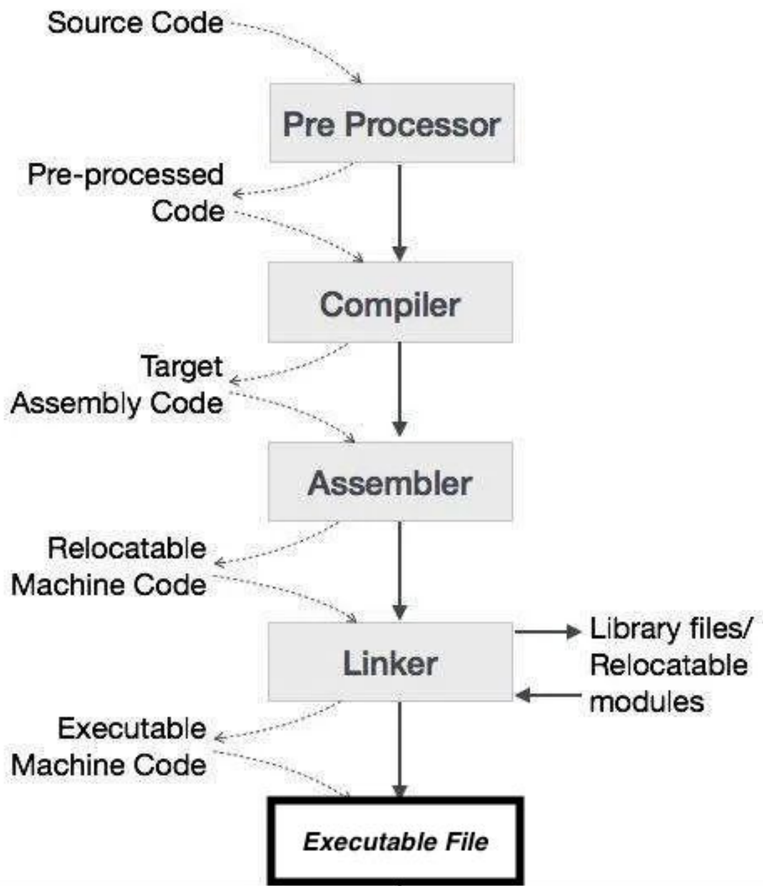
source: https://cs.lmu.edu/~ray/notes/x86assembly/

X86 Processor Registers

Processor Registers

8-bits	16-bits	32-bits	64-bits
al	ax	eax	rax
bl	bx	ebx	rbx
cl	cx	ecx	rcx
dl	dx	edx	rdx
sil	si	esi	rsi
dil	di	edi	rdi
bpl	bp	ebp	rbp
spl	sp	esp	rsp
r8b	r8	r8d	r8
r9b	r9	r9d	r9
r10b	r10w	r10d	r10
r11b	r11w	r11d	r11
r12b	r12w	r12d	r12
r13b	r13w	r13d	r13
r14b	r14w	r14d	r14
r15b	r15w	r15d	r15

source: https://medium.com/@notarioairlangga/simple-looping-in-assembly-summing-numbers-from-1-to-20-without-printing-an-introductory-aa6ecb7e98ed



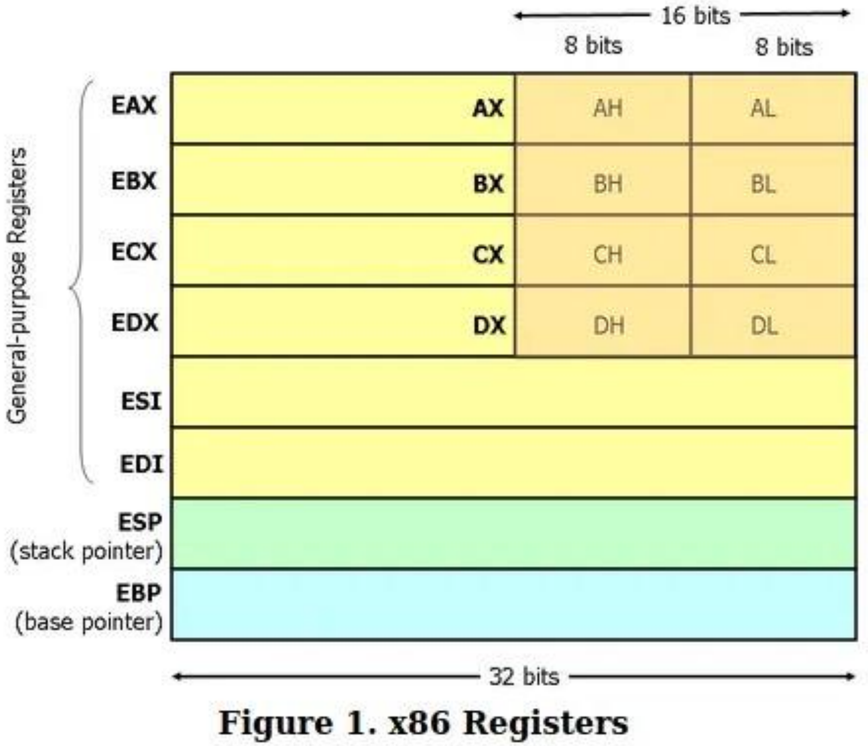


Figure 1. x86 Registers

source: https://pablocorbalann.medium.com/programming-a-hello-world-in-assembly-from-the-first-line-to-the-end-x86-9c48fb499238